14 haftalık yol haritasında 13. hafta

ALGORİTMA VE PROGRAMLAMA – II · DERS NOTU

HAFTA 13

Dinamik Programlama

Örtüşen Alt Problemlerden Tablolara

MODÜL V · OPTİMİZASYON VE SENTEZ · H13–H14

> **HAFTANIN BÜYÜK SORUSU**
>
> *Açgözlü yöntemin başarısız olduğu bir bozuk para sisteminde (H12), tüm olasılıkları denemeden en iyi yanıtı nasıl buluruz?*

Dr. Öğr. Üyesi Mehmet Sevri
Recep Tayyip Erdoğan Üniversitesi · Mühendislik ve Mimarlık Fakültesi · Bilgisayar Mühendisliği Bölümü
Bilgisayar Mühendisliği 1. sınıf · 2026–2027 Bahar yarıyılı · Sürüm 1.0 · Ekim 2026
`Ortam: gcc -std=c17 · g++ -std=c++17 (-Wall -Wextra -pedantic -O2) · Python 3.10+`

İçindekiler

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları 5
1.1 Öğrenme çıktıları 5
2. Geçen Haftadan ve AP-1'den Bağlantı 6
3. Kavramsal Anlatım: Örtüşen Alt Problemlerden Tablolara 6
3.1 Fibonacci'ye dönüş: dört sürüm 6
3.2 DP'nin iki koşulu 7
3.3 DP reçetesi: altı adım 8
3.4 İlk durum–geçiş tanımı: merdiven çıkma 9
3.5 Yukarıdan aşağıya ve aşağıdan yukarıya 9
3.6 Alt problem DAG'ı ve hesaplama sırası 9
3.7 Not tutmanın iki aracı: dizi ve karma tablosu 10
3.8 Kadane'ye (H2) DP gözüyle bakış 10
3.9 Kaputun Altında 11
4. Algoritmalar 12
4.1 Fibonacci: memoization, tablolama, iki değişken 12
4.2 Bozuk para: en az para ve kaç farklı yol 14
4.3 En uzun artan alt dizi (LIS) 17
4.4 En uzun ortak alt dizi (LCS) 19
4.5 Düzenleme (Levenshtein) uzaklığı ve “Bunu mu demek istediniz?” 21
4.6 0/1 sırt çantası: tam tablo, tek satır ve sözde polinom zaman 23
4.7 Aynı problem, dört strateji: alt küme toplamı 26
5. Çözümlü Örnekler ve Uygulamalar 28
5.1 Izgarada yol sayısı ve en küçük maliyetli yol (Örnek 5) 29
5.2 2B tabloyu kurmak: tek blok, satır vektörleri ve Python tuzağı (Örnek 6) 29
5.3 Meydan okuma: kelime bölme, önek DP'si (Örnek 14) 30
5.4 Meydan okuma: matris zinciri çarpımı, aralık DP'si (Örnek 15) 31
6. Haftanın Deneyi: Kaba Kuvvet, Memoization ve Tablolama 32
6.1 Ölçüm düzeni 32
6.2 (a) Fibonacci ve alt küme toplamı: üstelden sözde polinoma 33
6.3 (b) Python: lru_cache ile elle yazılmış not tablosu 34
6.4 (c) Doldurma sırası: satır ya da sütun öncelikli 34
6.5 Sınırlılıklar 34
7. Hata Avcısı 35
8. Dil Penceresi: C · C++ · Python 36

9. Mühendis Gözüyle: Bellek–Zaman Ödünleşimi 36
10. Mini-OBS 2 v2.12: Ders Seçimi (0/1 Sırt Çantası) ve “Bunu mu demek istediniz?” 37
10.1 Gereksinimler 37
10.2 İpuçları 38
10.3 Başlangıç noktası 38
10.4 Kabul ölçütleri ve test senaryoları 38
11. Alıştırmalar ve Atölyeler 39
11.1 İzleme ve çıktı tahmini (A) 39
11.2 Karmaşıklık (B) 40
11.3 Değişmez yazma (C) 41
11.4 Hata bulma (D) 41
11.5 Parsons (E) 42
11.6 Kod yazma (F) 43
11.7 Diller arası okuma (G) 43
11.8 Atölye 1 · DP reçetesi kartları (eşli, ≈ 25 dk) 44
11.9 Atölye 2 · Yazım düzeltici (gruplar, ≈ 30 dk) 45
11.10 Atölye 3 · Strateji turnuvası (dört grup, ≈ 30 dk) 45
12. Kendini Değerlendirme 45
13. İleri Bakış ve Merak Kutusu 46
13.1 Merak Kutusu: LIS'i O(n log n)'de bulmak — sabır sıralaması (Örnek 13) 46
14. Özet, Kavram Haritası ve Terim Sözlüğü 47
15. Kaynaklar 48
Ek A. Dil Karşılıkları 50
A.1 Örnek 1 · Fibonacci: memoization 50
A.2 Örnek 2 · Merdiven çıkma 51
A.3 Örnek 3 · Bozuk para: en az para 51
A.4 Örnek 4 · Bozuk para: kaç farklı yol (kombinasyon) 52
A.5 Örnek 5 · Izgara: yol sayısı ve en küçük maliyet 53
A.6 Örnek 6 · 2B tablo kurma (Δ) 54
A.7 Örnek 7 · LIS O(n^2) 56
A.8 Örnek 8 · LCS tablosu 57
A.9 Örnek 9 · Düzenleme uzaklığı 57
A.10 Örnek 10 · 0/1 sırt çantası 58
A.11 Örnek 11 · 0/1 sırt çantası: tek satır 60
A.12 Örnek 12 · Alt küme toplamı 60
A.13 Örnek 13 · LIS O(n log n): sabır sıralaması 61

A.14 Örnek 14 · Kelime bölme (meydan okuma)..... 63
A.15 Örnek 15 · Matris zinciri (meydan okuma)..... 64

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları

Açgözlü yöntemin başarısız olduğu bir bozuk para sisteminde (H12), tüm olasılıkları denemeden en iyi yanıtı nasıl buluruz?

H12'de açgözlü para üstünün {1, 3, 4} sisteminde yanıldığını gördük: 6 birimi "her adımda sığan en büyük para" kuralı 4 + 1 + 1 = 3 parayla öder, oysa 3 + 3 = 2 para yeter (Örnek 3). Bütün kombinasyonları denemek (H12 Örnek 1'in kâhini) her zaman doğrudur ama tutar ve para türü büyüdükçe olanaklar patlar. Bu hafta üçüncü bir yol öğreniyoruz: **dinamik programlama** (dynamic programming, DP). Fikir şudur: 6'nın en iyi ödemesi, son paradan sonra kalan tutarın (5, 3 ya da 2) en iyi ödemesine bir para eklemektir; o kalan tutarların yanıtı ise daha küçük tutarlardan gelir. Her tutarın yanıtını **bir kez** hesaplayıp bir tabloya yazarsak 0'dan S'ye kadar S + 1 hücre ve hücre başına para türü kadar deneme yeter: Θ(n·S) iş.

Aynı düşünce, AP-1 H9'da üstel çağrı ağacıyla tanıştığımız özyinelemeli Fibonacci'yi doğrusal zamana indirir, iki metnin en uzun ortak alt dizisini (diff), bir yazım hatasının en yakın sözcüğünü ("Bunu mu demek istediniz?") ve AKTS sınırı altında en değerli ders kümesini (0/1 sırt çantası) bulur. Hepsinin ortak yapısı: **örtüşen alt problemler** (overlapping subproblems) ve **en iyi alt yapı** (optimal substructure). Haftanın sorusu, haftanın sonunda iki satırla yanıtlanacak: durum enaz[t], geçiş enaz[t] = min(enaz[t − c] + 1).

> **GERÇEK DÜNYADAN · "Dinamik programlama" adı nereden geliyor?**
>
> Bellman'ın (1984) anlattığına göre, 1950 sonbaharını RAND Corporation'da geçirirken ilk işi "çok aşamalı karar süreçleri" (multistage decision processes) için bir ad bulmaktı (aktaran Dreyfus, 2002). Bellman'a göre Washington'da Savunma Bakanı Wilson, "araştırma" sözcüğünden patolojik biçimde korkuyor ve nefret ediyordu; RAND, Hava Kuvvetleri için çalışıyordu ve Hava Kuvvetleri'nin başında da fiilen Wilson bulunuyordu. Wilson'ın "matematiksel" sözcüğüne ne hissedeceğini tahmin etmek de zor değildi. Bellman bu yüzden "programming" sözcüğünü seçti; işin dinamik, çok aşamalı ve zamana bağlı olduğunu anlatan ve "kötü anlamda kullanılması olanaksız" gördüğü "dynamic" sözcüğünü ekledi. Kendi deyişiyle bu, "bir Kongre üyesinin bile itiraz edemeyeceği" bir addı; onu bütün çalışmalarının şemsiyesi yaptı.[1]
>
> Aynı yöntem bugün başka alanların da temelidir. Needleman ve Wunsch (1970), iki proteinin amino asit dizilerindeki benzerlikleri aramak için genel bir yöntem önerdi; dizi hizalama, biyoinformatiğin temel araçlarından biri oldu. Wagner ve Fischer (1974) bir dizgiyi ötekine değiştirme, silme ve ekleme işlemleriyle çeviren en kısa işlem dizisini iki dizginin uzunluklarının çarpımıyla orantılı zamanda bulan algoritmayı verdi ve uygulama olarak otomatik yazım düzeltmeyi ve en uzun ortak alt diziyi andı. Unix'in `diff` aracının merkezindeki algoritma da "en uzun ortak alt dizi problemini" çözerek dosyalar arasında değişmeyen satırları bulur; Hunt ve McIlroy'un (1976) belirttiği gibi doğrudan dinamik program hem zamanda hem bellekte O(mn)'dir.
>
> [1] Kronoloji notu: Bellman olayı 1950 sonbaharına yerleştirir; oysa Charles E. Wilson ABD Savunma Bakanı olarak 28 Ocak 1953'te göreve başlamıştır (Office of the Secretary of Defense Historical Office, t.y.) ve Bellman terimi bir makale başlığında Ağustos 1952'de kullanmıştır (Bellman, 1952). Anlatı bu yüzden Bellman'ın kendi hatırası olarak okunmalıdır; aktaran Dreyfus (2002) da tarihleri ayrıca doğrulamaz.

1.1 Öğrenme çıktıları

- **ÖÇ 13.1** Bir problemin dinamik programlamaya uygunluğunu örtüşen alt problemler ve en iyi alt yapı ölçütleriyle belirler. (Bölüm 3.1–3.2, 4.1)
- **ÖÇ 13.2** Özyinelemeli bir çözümü memoization ile yukarıdan aşağıya, tablolamayla aşağıdan yukarıya dinamik programlamaya dönüştürür. (Bölüm 3.5, 4.1, Örnek 1–2)

- **ÖÇ 13.3** Durum tanımı, geçiş bağıntısı, temel durumlar ve hesaplama sırasını açıkça yazarak klasik DP problemlerini (bozuk para, LIS, LCS, düzenleme uzaklığı, 0/1 sırt çantası) çözer. (Bölüm 3.3, 4.2–4.6, Örnek 3–11)
- **ÖÇ 13.4** DP tablosundan en iyi çözümün kendisini geri çıkarır (reconstruction); bellek kullanımını satır tamponlarıyla azaltır. (Bölüm 4.3–4.6, 3.9, Örnek 7–11)
- **ÖÇ 13.5** Kaba kuvvet, geri izleme, açgözlü ve DP çözümlerini aynı problem üzerinde doğruluk ve karmaşıklık açısından karşılaştırır. (Bölüm 4.7, 6, Örnek 12)

2. Geçen Haftadan ve AP-1'den Bağlantı

Bu hafta beş eski konuyu birleştiriyoruz. **H12:** açgözlü bir algoritma her adımda tek bir seçim yapar ve geri dönmez; doğru olması için **açgözlü seçim özelliği** ve **en iyi alt yapı** gerekir. DP yalnız ikincisini ister: bütün seçenekleri dener ama her alt problemi bir kez çözer. **H3:** özyinelemeli Fibonacci'nin ve alt küme toplamının üstel arama ağaçları DP'nin hareket noktasıdır. **H11–H12:** topolojik sıra ve kenar gevşetme, DP'nin graf diliyle anlatımıdır (Bölüm 3.6). **AP-1 H9 ve H11:** özyineleme ve 2B diziler (satır öncelikli yerleşim) tablonun kendisidir.

> TAHMİN ET · **Geri çağırma: dört soru (yanıtlar aşağıda)**
>
> - **S1 (H12).** {1, 3, 4} sisteminde 1 … 100 tutarlarının kaçında açgözlü para üstü en az parayı verir? Yanıldığı en küçük tutar kaçtır?
> - **S2 (H3).** Saf özyinelemeli fib(5) kaç çağrı yapar? Bunların kaçı daha önce çözülmüş bir alt problemin tekrarıdır?
> - **S3 (H12).** Kapasite 50: A(10 kg, 60), B(20 kg, 100), C(30 kg, 120). Yükler bölünemiyorsa birim değere göre açgözlü kaç değer toplar, en iyi kaç?
> - **S4 (H11).** Bir DAG'ın topolojik sırası her kenar u → v için neyi güvence eder? Döngülü bir grafta ne olur?

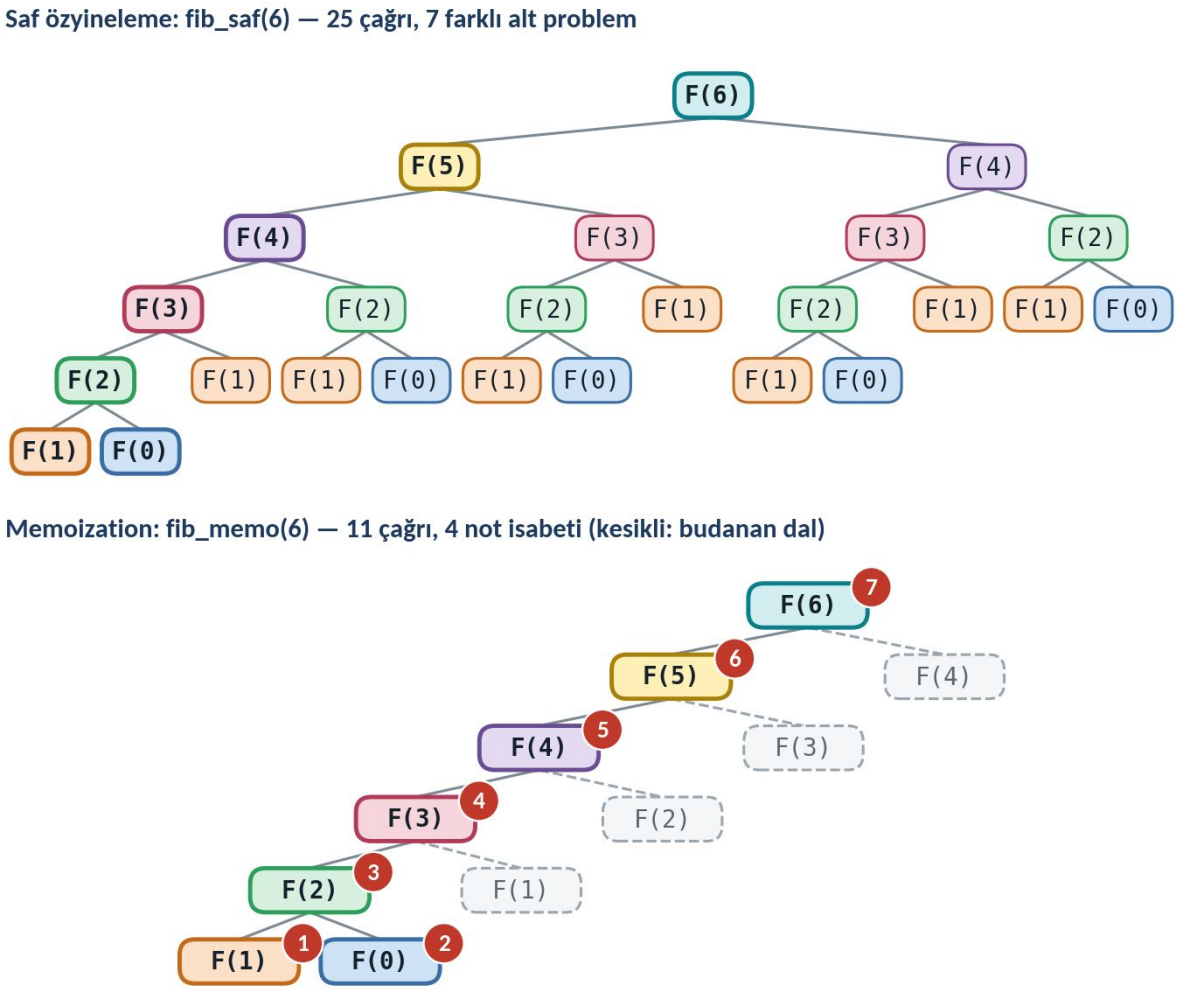
Yanıtlar. S1: 76'da en iyidir; yanıldığı ilk tutar 6 (H12 Örnek 1). S2: 15 çağrı; yalnız 6 farklı alt problem vardır, 9 çağrı tekrardır (H3 Örnek 1) — DP'nin “bir kez çöz” ilkesi tam bu tekrarları atar. S3: açgözlü AB → 160, en iyi BC → 220 (H12 Örnek 4); 0/1 sürümde açgözlü seçim özelliği yoktur, Bölüm 4.6 bunu DP ile çözer. S4: u sırada v'den önce gelir; döngülü grafta topolojik sıra yoktur (H11 Örnek 11'in sırası: 1 2 4 3 5 6 7 0 8). DP'de “önce bağımlı olunan alt problemler” kuralı tam olarak alt problem DAG'ının topolojik sırasıdır.

3. Kavramsal Anlatım: Örtüşen Alt Problemlerden Tablolara

3.1 Fibonacci'ye dönüş: dört sürüm

AP-1 H9'da F(n) = F(n − 1) + F(n − 2) tanımını doğrudan özyinelemeye çevirmiştik. fib_saf(6)'nın çağrı ağacında 25 çağrı vardır ama yalnız 7 farklı alt problem (F(0) … F(6)) çözülür; F(1) tek başına 8 kez, F(2) 5 kez hesaplanır (Şekil 13.1, üstte). Örnek 1 çağrı sayısının tam olarak 2·F(n + 1) − 1 olduğunu denetler: n = 25'te 242 785 çağrı, ardışık oran çağrı(25)/çağrı(24) = 1,6180, yani altın oran φ ≈ 1,618: büyüme Θ(φⁿ)'dir.

Şekil 13.1
Örnek 1: fib_saf(6) çağrı ağacı (25 çağrı; aynı alt problem aynı renkte, ince kenarlı kutu tekrar) ve fib_memo(6) (11 çağrı; kesikli gri: nottan gelen, budanan dal; rozet: memo'ya yazılış sırası)



Saf ağaçta çağrı sayısı: F(0) 5 · F(1) 8 · F(2) 5 · F(3) 3 · F(4) 2 · F(5) 1 · F(6) 1

Kırmızı rozet: memo'ya yazılış (hesaplama) sırası 1 → 7; aynı alt problem her yerde aynı renkte

Aynı değer dört maliyetle hesaplanabilir. **Memo** (memoization, not alma) özyinelemeyi korur ama her F(i)'yi ilk hesaplandığında bir diziye yazar ve sonraki çağrılarda oradan döndürür: n = 25'te 49 çağrı, 26 hesaplanan hücre, 23 not isabeti (Θ(n) zaman, Θ(n) dizi + Θ(n) yığın). **Tablo** (tablolama, tabulation) özyinelemeyi bırakır, tablo[0..n]'i küçükten büyüğe bir döngüyle doldurur: 26 hücre, yığın yok. Tablonun her satırı yalnız son iki değere baktığı için **iki değişken** yeter: O(1) bellek. Saf sürüm ise Θ(φⁿ) çağrı ve Θ(n) derinlikte yığın kullanır.

3.2 DP'nin iki koşulu

Örtüşen alt problemler: özyinelemeli çözüm aynı alt problemleri tekrar tekrar ister. Fibonacci'de F(n − 1) de F(n − 2)'yi çağırır; bozuk parada 6 tutarı 5'i, 3'ü ve 2'yi, 5 de 4'ü, 2'yi ve 1'i ister. Farklı alt problemlerin sayısı küçüktür (n + 1 ya da S + 1), çağrıların sayısı üsteldir. **En iyi alt yapı:** problemin en iyi çözümü, alt problemlerin en iyi çözümlerinden kurulur. 6'nın en iyi ödemesinin son parası c ise geri kalanı 6 − c'nin en iyi ödemesidir; daha kötü bir ödeme kullanılsaydı onu iyileştirerek 6'nın ödemesini de iyileştirebilirdik (“kes ve yapıştır” kanıtı; Cormen vd., 2022).

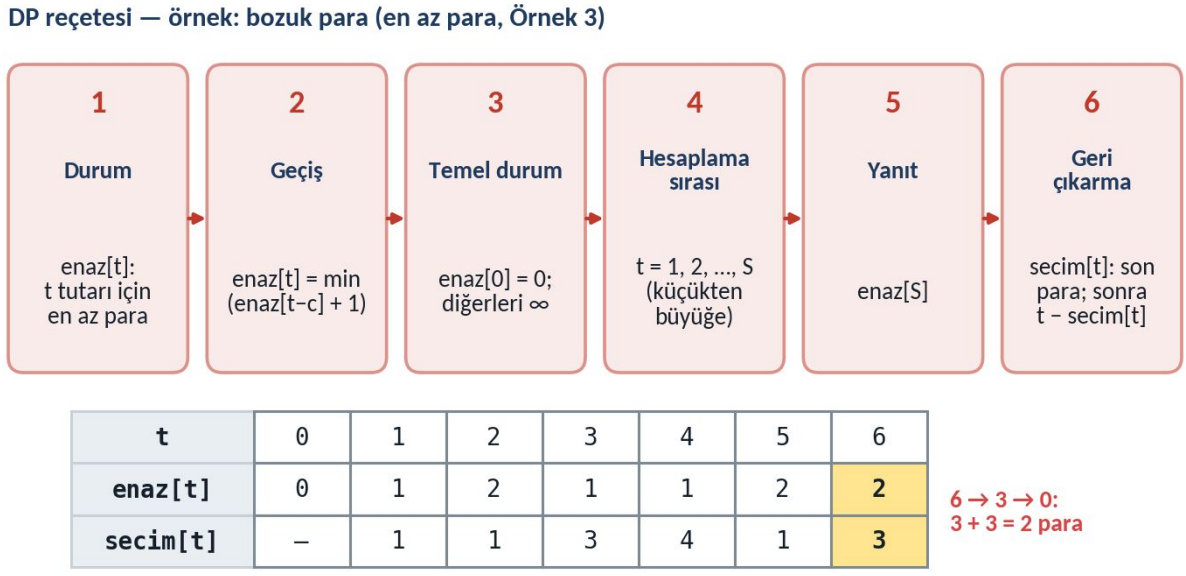
Böl ve fethetten farkı (H6): birleştirmeli sıralamanın alt problemleri **ayrıktır**: sol yarı ile sağ yarı ortak öğe paylaşmaz ve her alt dizi bir kez sıralanır; not almak kazanç sağlamaz. DP'de alt problemler örtüşür; not almadan çözersek aynı alt problem üstel sayıda çözülür. **Açgözlüden farkı (H12):** açgözlü her adımda tek seçeneği dener (açgözlü seçim özelliği varsayımıyla); DP her seçeneği dener ama her alt problemi bir kez çözer. En iyi alt yapının olmadığı bir örnek: ağırlıksız grafta iki köşe arasındaki **en uzun basit yol** — alt yolların en uzunları birleşince yol basit kalmayabilir; bu problem DP'nin bu biçimine uymaz (Cormen vd., 2022).

3.3 DP reçetesi: altı adım

Her DP çözümü altı soruya verilen yanıtlarla tanımlanır. Kod yazmadan önce bu altı satırı yazmak, haftanın en önemli alışkanlığıdır (Atölye 1). Örneklerin hepsinin dosya başlığında bu reçete bir **DP** satırı olarak yazılıdır.

1. **Durum** (state): bir hücre neyi anlatır? enaz[t] = t tutarı için en az para; sonucu belirleyen bütün parametreleri içerir (Hata Avcısı 6).
2. **Geçiş** (transition): "son adımda ne yaptım?" sorusundan çıkar: enaz[t] = min{enaz[t − c] + 1 : c ≤ t}.
3. **Temel durum** (base case): geçişsiz bilinen hücreler: enaz[0] = 0, ulaşılamayan tutar ∞ (Hata Avcısı 1 ve 5).
4. **Hesaplama sırası:** her hücre bağlı olduğu hücrelerden sonra (t artan); yanlış sıra boş hücre okur (Hata Avcısı 2).
5. **Yanıt:** aranan değer hangi hücrededir? enaz[S]; LIS'te ise bütün uz[i]'lerin en büyüğü.
6. **Geri çıkarma** (reconstruction): çözümün kendisi için kazanan seçimi tutarız (secim[t]) ya da tablodan geriye okuruz. "Geri izleme" (backtracking) H3'ün arama yöntemidir.

Şekil 13.2
DP reçetesinin altı adımı; Örnek 3'ün tablosu ve geri çıkarma (6 → 3 → 0)



t	0	1	2	3	4	5	6
enaz[t]	0	1	2	1	1	2	**2**
secim[t]	–	1	1	3	4	1	**3**

6 → 3 → 0:
3 + 3 = 2 para

3.4 İlk durum–geçiş tanımı: merdiven çıkma

Bir merdivende her adımda 1 ya da 2 basamak çıkılıyor; n. basamağa kaç farklı yolla çıkılır? **Durum:** yol[i] = i. basamağa çıkış sayısı. **Geçiş:** son adım ya 1 basamaktı (öncesinde i − 1'deydik) ya 2 (i − 2'deydik): yol[i] = yol[i − 1] + yol[i − 2]. **Temel durum:** yol[0] = 1 (hiç adım atmamak bir yoldur), yol[1] = 1. **Sıra:** i artan. **Yanıt:** yol[n]. Örnek 2:

c/H13_02_merdiven.c · satır 26–33

```c
static void merdiven_tablo(int n)
{
    yol[0] = 1;                              /* temel durum */
    yol[1] = 1;
    for (int i = 2; i <= n; i++) {
        yol[i] = yol[i - 1] + yol[i - 2]; /* son adım 1 ya da 2 */
    }
}
```

Tablo Fibonacci'nin bir kaydırılmışıdır: yol[n] = F(n + 1) (`n = 0..40 için yol[n] = F(n+1): evet; yol[40] = 165580141`). Program bunu listelemeyle de doğrular (n = 5'te sekiz yol), {1, 2, 3} değişkesinde geçişe üçüncü terim eklenir (`yol3 : 1 1 2 4 7 13 24 44 81 149 274`) ve büyük n için her toplamadan sonra mod $10^9 + 7$ alır. Alıştırma 13.1 aynı reçeteyi {1, 3} için ister.

3.5 Yukarıdan aşağıya ve aşağıdan yukarıya

Bir DP iki biçimde yazılır. **Memoization (yukarıdan aşağıya)**: özyinelemeli tanım aynen kalır, yalnız sonuçlar bir not tablosunda saklanır ve "hesaplanmadı" işaretiyle (C'de −1) denetlenir. Yalnız gerçekten **gereken** alt problemler hesaplanır: Kavram 6'da alt küme toplamının (n = 30) 8 835 hücreli tablosundan yalnız 6 005 tanesi (%68,0) ziyaret edilir. **Tablolama (aşağıdan yukarıya)**: hesaplama sırası açıkça seçilir ve tablonun tamamı bir döngüyle doldurulur; çağrı ve yığın yoktur, bellek azaltma (satır tamponu) kolaydır. Tablo 13.1 ikisini karşılaştırır.

Tablo 13.1
Memoization ile tablolamanın karşılaştırması

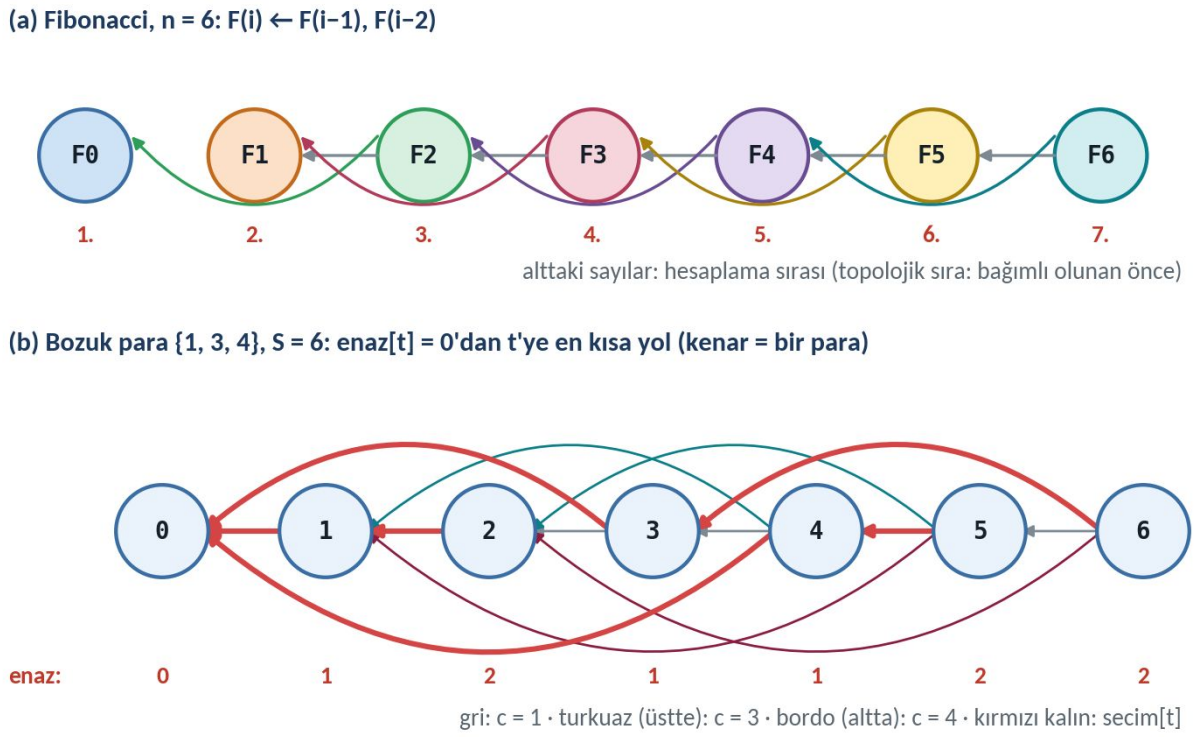
	Memoization (yukarıdan aşağıya)	Tablolama (aşağıdan yukarıya)
Yazım	özyinelemeli tanım + not tablosu	döngüler; doldurma sırası elle seçilir
Hesaplanan durumlar	yalnız gerekenler (seyrek durumda kazanç)	tablonun tamamı
Çağrı yığını	derinlik kadar (Python'da sınır: Bölüm 3.9)	yok
Bellek azaltma	zor (hangi notun artık gerekmediği belirsiz)	kolay: satır tamponu, iki değişken
Geri çıkarma	not tablosundan ya da ayrı seçim dizisinden	tablodan geriye doğru
Ne zaman?	durum uzayı büyük ama seyrek; tanım doğal olarak özyinelemeli	durum uzayı yoğun; en iyi sabit çarpan; derin zincirler

3.6 Alt problem DAG'ı ve hesaplama sırası

Alt problemleri düğüm, "u, v'nin sonucunu kullanır" ilişkisini kenar yapalım. Örtüşen alt problemler bu grafta birden çok yoldan ulaşılan düğümlerdir ve graf **döngüsüzdür** (DAG): bir alt problem kendine bağlı olamaz. Hesaplama sırası bu DAG'ın bir **topolojik sırasıdır** (H11): bağımlı olunan önce. Bozuk para DAG'ında 0'dan t'ye her yol bir ödemedir; enaz[t] 0'dan t'ye **en kısa yoldur** (her kenar bir para). Kavram

7 aynı tabloyu H12'nin diliyle hesaplar: düğümleri topolojik sırayla (t artan) ziyaret eder ve her kenarı **gevşetir**; 13 kenarda 7 başarılı gevşetmeyle Örnek 3'ün enaz satırını verir. LIS'te ise kenar j → i (j < i, a[j] < a[i]) ve uz[i], i'de biten **en uzun yoldur** (19 kenar, en uzun yol 4 düğüm); sayma DP'leri (merdiven, Örnek 4) ise yolların **sayısını** toplar. Dijkstra'dan farkı: DAG'da topolojik sıra bilindiği için öbek gerekmez ve negatif kenar sorun değildir.

Şekil 13.3
Alt problem DAG'ı: (a) Fibonacci, n = 6 ve hesaplama sırası; (b) bozuk para {1, 3, 4}, S = 6 — enaz[t], 0'dan t'ye en kısa yoldur (kırmızı: seçilen son paralar)



3.7 Not tutmanın iki aracı: dizi ve karma tablosu

Durum küçük tamsayılarla tanımlanıyorsa not tablosu bir **dizidir**: (i, t) → memo[i·(T + 1) + t], başlangıçta −1. Durum uzayı seyrekse ya da sınırları önceden bilinmiyorsa H10'un **karma tablosu** yalnız ziyaret edilen durumları saklar. Kavram 6 iki aracı aynı problemde karşılaştırır: ikisi de 7 107 232 sonucunu verir; karma tablosu yalnız ziyaret edilen 6 005 durumu saklar. Yoğun durum uzayında dizi daha hızlı ve yalındır; Python'da sözlük tabanlı notların (`dict`, `functools.cache`) bedeli Haftanın Deneyi'nde ölçülür (Bölüm 6.3). Not anahtarı durumun **tamamını** içermelidir (Hata Avcısı 6).

3.8 Kadane'ye (H2) DP gözüyle bakış

H2'nin en büyük toplamlı alt dizi algoritması (Kadane) aslında bir DP'dir. **Durum:** biten[i] = a[i] ile **biten** en büyük toplamlı alt dizinin toplamı. **Geçiş:** ya a[i]'de yeni başlarız ya önceki diziyi uzatırız: biten[i] = max(a[i], biten[i − 1] + a[i]). **Temel:** biten[0] = a[0]. **Yanıt:** bütün biten[i]'lerin en büyüğü. Tablonun yalnız son hücresi kullanıldığından tek değişkene iner — Fibonacci'nin iki değişkenli sürümüyle aynı fikir. Kavram 5, H2'nin dizisinde biten[i] satırını (−2, 1, −2, 4, 3, 5, 6, 1, 5) yazar: en büyük toplam 6, aralık a[3..6] — H2 Örnek 13'ün sonucuyla aynı.

3.9 Kaputun Altında

KAPUTUN ALTINDA · Satır öncelikli yerleşim ve doldurma sırası

C'de 2B tablo (ya da tek bloktaki t[i·m + j]) satır öncelikli saklanır: L[i][j] ile L[i][j + 1] bellekte komşudur (AP-1 H11). LCS tablosu satır satır doldurulursa iç döngü belleği sırayla tarar ve her önbellek satırı (64 bayt) 16 `int` getirir; sütun sütun doldurulursa her adım m + 1 hücre ileri atlar. Değer ikisinde de aynıdır (bağımlılıklar her iki sırada da önce hesaplanır), süre değil: bu makinede C -O2'de tablo ≈ 1 MB iken iki sıra eşit (n = 500: 1,07 kat), tablo önbelleği aşınca sütun öncelikli sıra en çok 6,15 kat yavaş (n = 3 000, 36 MB). Python'da oran 0,85–1,08: hücre başına yorumlayıcı maliyeti önbellek etkisini örter (Bölüm 6.4).

KAPUTUN ALTINDA · İki satıra inmek ve geri çıkarmayı kaybetmek

LCS ve düzenleme uzaklığında satır i yalnız satır i − 1'e bakar; iki satırlık tampon yeter ve kısa dizgi sütun yapılırsa bellek O(min(n, m)) olur. Kavram 1, n = m = 20 000 rastgele DNA dizisinde LCS = 13 034 değerini iki satırla 160 008 bayt kullanarak bulur; tam tablo 1 600 160 004 bayt gerektirirdi (10 000 kat fazla). Bedeli: geri çıkarma (n, m)'den başlayıp her adımda bir üst satıra bakar; o satırlar artık üzerine yazılmıştır: değer var, yol yok (Şekil 13.4). Hem az bellek hem çözümün kendisi gerekiyorsa Hirschberg'in (1975) böl ve fethet yöntemi kullanılır (kapsam dışı).

Şekil 13.4
Kavram 1: düzenleme uzaklığının tam tablosu ve iki satırlı sürümü (kitten → sitting); değer aynı, geri çıkarma için gereken satırlar silinmiş



K01: n = m = 20 000 LCS = 13 034: tam tablo 1 600 160 004 bayt, iki satır 160 008 bayt (10 000 kat az)

değer aynı (D[6][7] = 3), ama yol yok: çare tam tablo ya da Hirschberg (1975)

KAPUTUN ALTINDA · Memoization'da özyineleme derinliği

Yukarıdan aşağıya memo **özyinelemelidir**: fib(n) → fib(n − 1) → … zinciri n derinliğe iner. Python'un varsayılan sınırı `sys.getrecursionlimit()` = 1000'dir. Bu yorumlayıcıda (CPython 3.11) `functools.cache` ile soğuk önbellekte fib(600) `RecursionError` verdi; elle yazılmış sözlük memo aynı n'de çalıştı. Haftanın Deneyi'nde çalışan en büyük n elle liste ve sözlükte 996, `lru_cache` ve `cache` ile 498 çıktı: dekoratörlü fonksiyonun her düzeyi özyineleme sayacında yaklaşık iki düzey sayılıyor (bu sürümdeki gözlem; başka sürümde farklı olabilir). Sınırı `sys.setrecursionlimit` ile yükseltmek C yığınını taşırıp süreci çökertebilir; doğru çare tablolamadır (döngü, yığın yok): Kavram 2 fib(100 000) mod $10^9 + 7$'yi böyle hesaplar. C'de yığın taşması çoğunlukla uyarısız bir çöküşle (segmentation fault) sonlanır; derin zincirlerde de tablo tercih edilir.

4. Algoritmalar

Amiral algoritmalar on bir adımlı standart anlatımla işlenir (problem, fikir, elle izleme, sözde kod, doğruluk, analiz ve kimlik kartı, C, üç dil, test, deney, varyantlar). Kod kutuları dosya satır numaralı çekirdek döngülerdir; tam fonksiyonlar `kodlar/` klasöründe ve Ek A'dadır.

4.1 Fibonacci: memoization, tablolama, iki değişken

1 · Problem. F(0) = 0, F(1) = 1, F(n) = F(n − 1) + F(n − 2); 64 bitlik işaretsiz tamsayıya F(93)'e kadar sığar. **2 · Fikir.** Saf özyineleme aynı soruları tekrar sorar (Bölüm 3.1). Her yanıtı ilk bulduğumuzda bir deftere yazıp sonra defterden okursak (**not alma**) her F(i) bir kez hesaplanır; F(0)'dan yukarı doğru doldurursak özyineleme de gerekmez. **Reçete:** durum F(i) · geçiş F(i − 1) + F(i − 2) · temel F(0), F(1) · sıra i artan · yanıt tablo[n].

3 · Elle izleme. fib_memo(6), memo başta −1 (Tablo 13.2; "not": nottan dönen çağrı; ağaç Şekil 13.1, altta). Özyineleme önce F(1)'e iner; geri dönerken her F(i) bir kez hesaplanır, F(i − 2) için yapılan ikinci çağrı nottan döner: n ≥ 2 için 2n − 1 çağrı, n + 1 hesaplanan hücre (Örnek 1'in memo tablosu bunu n = 2 … 25 için gösterir).

Tablo 13.2
fib_memo(6): 11 çağrı (ön sıra), 4 not isabeti; memo'ya yazılış sırası 1, 0, 2, 3, 4, 5, 6

çağrı	1	2	3	4	5	6	7	8	9	10	11
n	6	5	4	3	2	1	0	1	2	3	4
ne olur?	hesap	hesap	hesap	hesap	hesap	temel	temel	not	not	not	not
yazılış	7.	6.	5.	4.	3.	1.	2.	–	–	–	–

SÖZDE KOD · 4 · Sözde kod: yukarıdan aşağıya ve aşağıdan yukarıya

```
algoritma FİB-MEMO(i)                    ▷ memo[0..n] başta −1
    eğer memo[i] ≠ −1 ise döndür memo[i]   ▷ not isabeti
    eğer i < 2 ise memo[i] ← i
    değilse memo[i] ← FİB-MEMO(i − 1) + FİB-MEMO(i − 2)
    döndür memo[i]
algoritma FİB-TABLO(n)
    tablo[0] ← 0;  tablo[1] ← 1
    i ← 2'den n'ye kadar:  tablo[i] ← tablo[i − 1] + tablo[i − 2]
    döndür tablo[n]
```

5 · Doğruluk. Tablolamanın değişmezi: i. adımdan sonra tablo[0..i] = F(0..i). Başlatma i = 1'de, koruma tanımdan, sonlanma i = n'de; memo için aynı tümevarım: memo[i] ya −1'dir ya F(i). **6 · Analiz.** Tablo n − 1 toplama: Θ(n) zaman ve bellek; iki değişken Θ(1) bellek; memo Θ(n) not + Θ(n) derinlikte yığın. Saf sürüm 2·F(n + 1) − 1 çağrı, $\Theta(\varphi^n)$.

Tablo 13.3
Fibonacci kimlik kartı

ALGORİTMA KİMLİK KARTI **Fibonacci: memoization, tablolama, iki değişken** *(Fibonacci: memoization, tabulation, constant space)*	
Strateji	DP: F(i − 1) ve F(i − 2) bir kez çözülür (memo ya da tablo)

Girdi önkoşulu	n ≥ 0; uint64_t'ye F(93)'e kadar sığar (sonrası mod $10^9 + 7$)
Zaman (en iyi · ortalama · en kötü)	Θ(n) · Θ(n) · Θ(n)
Ek bellek	O(n) / O(1)
Özellikler	n = 25: saf 242 785 çağrı; memo 49 çağrı; tablo 26 hücre
Değişmez / doğruluk	tablo[0..i] = F(0..i); iki değişkende (onceki, simdiki) = (F(i − 1), F(i))
Kütüphane karşılığı	C: yok · C++: yok · Python: `functools.cache`
Ne zaman kullanılır	Doğrusal yinelemeli bağıntılar (merdiven, döşeme, sayma)
Ne zaman kullanılmaz	Tek ve çok büyük n (matris üssü, O(log n)); saf özyineleme asla

7 · C gerçekleştirimi. Not tablosunda −1 "hesaplanmadı" demektir; F(i) ≥ 0 olduğu için geçerli bir yanıtla karışmaz (0 seçilseydi F(0) sürekli yeniden hesaplanırdı: Alıştırma 13.9):

C `c/H13_01_fibonacci.c · satır 38–45`

```c
    if (memo[i] != -1) {
        return memo[i];                    /* not alınmış: yeniden çözme */
    }
    if (i < 2) {
        memo[i] = i;
    } else {
        memo[i] = fib_memo(i - 1) + fib_memo(i - 2);
    }
```

8 · Üç dil. Memo C++'ta aynı satırlardır; tablolama döngüsü üç dilde aynıdır, yalnız Python'un `range`'i üst sınırı dışarıda bırakır (`n + 1`). Python'da `functools.cache` memo'yu tek satıra indirir ama derin zincirde özyineleme sınırına erken takılır (Bölüm 3.9):

C++ `H13_01_fibonacci.cpp · satır 60–62`

```cpp
    for (int i = 2; i <= n; i++) {
        tablo[i] = tablo[i - 1] + tablo[i - 2];
    }
```

Python `H13_01_fibonacci.py · satır 49–50`

```python
    for i in range(2, n + 1):
        tablo[i] = tablo[i - 1] + tablo[i - 2]
```

9 · Test. Program saf çağrı sayısını 2·F(n + 1) − 1 ile, dört sürümü birbiriyle karşılaştırır (`n = 0..90 için iki değişken = tablo: evet`). Sınır: `F(93) = 12200160415121876738`; F(94)'te toplama yapılmadan **önce** `onceki > UINT64_MAX - simdiki` denetlenir; büyük n için her adımda mod $10^9 + 7$ alınır. **10 · Deney.** C -O2'de n = 35: saf 131,2 ms, tablo 0,0005 ms: yaklaşık 265 bin kat (tablo süresi ölçüm çözünürlüğüne yakın olduğundan kat kabadır; Bölüm 6.2). **11 · Varyantlar.** Merdiven ve döşeme sayma (Bölüm 3.4), tek ve çok büyük n için matris üssü (O(log n); H2'nin hızlı üs alması).

4.2 Bozuk para: en az para ve kaç farklı yol

1 · Problem. Para değerleri $c_1 < \ldots < c_n$ (her birinden sınırsız) ve tutar S: S en az kaç parayla, hangileriyle ödenir? Ödenemiyorsa bildirilmeli. **2 · Fikir.** Son para c ise kalan S − c'nin **en iyi** ödemesi kullanılmalıdır (en iyi alt yapı). Son parayı bilmediğimiz için hepsini deneriz, S − c'nin yanıtını tablodan okuruz. **Reçete:** durum enaz[t] · geçiş min{enaz[t − c] + 1} · temel enaz[0] = 0, diğerleri ∞ · sıra t = 1 … S · yanıt enaz[S] · geri çıkarma secim[t].

3 · Elle izleme. {1, 3, 4}, S = 6 (Şekil 13.5): t = 4'te adaylar 1:2, 3:2, 4:1 → 1 para; t = 6'da 1:3, 3:2, 4:3 → c = 3 ile enaz[3] + 1 = 2 para. Açgözlü 4'ü seçer ve kalan 2'yi yalnız 1 + 1 ile öder. **Tahmin edin** (şekle bakmadan; sonra Şekil 13.5'nin secim satırından denetleyin): t = 5'in adayları 1:2, 3:3, 4:2. Paralar artan sırayla denenir ve yalnız **kesin** küçükte güncellenir. secim[5] kaçtır, 5'in geri çıkarılan ödemesi nedir?

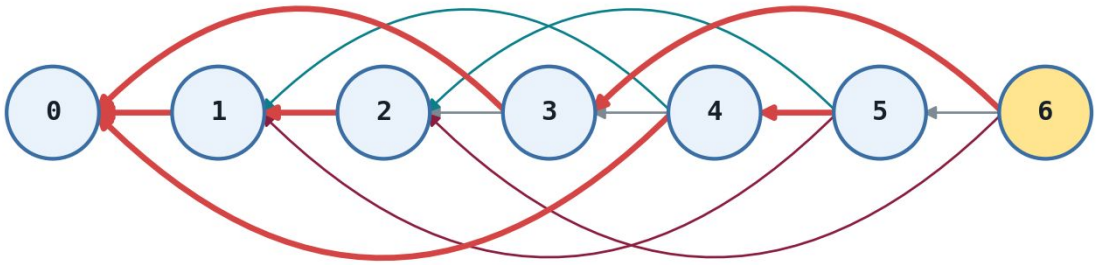
SÖZDE KOD · 4 · Sözde kod: en az para ve geri çıkarma

```
algoritma EN-AZ-PARA(para[1..n], S)        ▷ para artan sıralı
    enaz[0] ← 0
    t ← 1'den S'ye kadar her t için
        enaz[t] ← ∞;  secim[t] ← 0
        her c ∈ para için
            eğer c ≤ t ve enaz[t − c] ≠ ∞ ve enaz[t − c] + 1 < enaz[t] ise
                enaz[t] ← enaz[t − c] + 1;  secim[t] ← c
    döndür enaz[S]                          ▷ ∞: ödenemez
GERİ-ÇIKAR(t):  iken t > 0:  yaz secim[t];  t ← t − secim[t]
```

Şekil 13.5
Bozuk para {1, 3, 4}, S = 6: durum–geçiş diyagramı (ok t → t − c; kırmızı: secim[t]) ve adaylar, enaz, secim satırları (Örnek 3)

Durum–geçiş diyagramı: paralar {1, 3, 4}, S = 6 (düğüm t, ok t → t − c)

gri: c = 1 · turkuaz (üstte): c = 3 · bordo (altta): c = 4 · kırmızı kalın: secim[t]



t	0	1	2	3	4	5	6
adaylar*	–	1:1	1:2	1:3 3:1	1:2 3:2 4:1	1:2 3:3 4:2	1:3 3:2 4:3
enaz[t]	0	1	2	1	1	2	2
secim[t]	–	1	1	3	4	1	3

* adaylar c:v — para c ile enaz[t − c] + 1 = v; en küçük v seçilir (eşitlikte küçük c)

5 · Doğruluk. Değişmez: t artan doldurulurken t'den küçük her u için enaz[u] en az para sayısıdır (ya da ∞). Koruma en iyi alt yapıdan gelir: t'nin en iyi ödemesinin son parası c ise t − c daha az parayla ödenebilseydi t de öyle ödenirdi; bütün c'ler denendiği için en iyi kaçmaz. ∞'a 1 eklenmez: anlam

(ulaşılamaz + 1) ve C'de işaretli taşma (Hata Avcısı 5). **6 · Analiz.** Θ(n·S) zaman, Θ(S) bellek; S girdinin **değeridir**, uzunluğu değil: sözde polinom (Bölüm 4.6.2).

Tablo 13.4
Bozuk para (en az para) kimlik kartı

ALGORİTMA KİMLİK KARTI	**Bozuk para: en az para sayısı** *(coin change (minimum number of coins))*
Strateji	Dinamik programlama (1B, en iyileme): enaz[t] = min(enaz[t − c] + 1)
Girdi önkoşulu	Pozitif tamsayı paralar ve tutar; her paradan sınırsız sayıda
Zaman (en iyi · ortalama · en kötü)	Θ(n·S) · Θ(n·S) · Θ(n·S)
Ek bellek	O(S)
Özellikler	{1, 3, 4} ile 6: açgözlü 3 para, DP 2 para; kanonik sistemde ikisi aynı
Değişmez / doğruluk	t artarken her u < t için enaz[u] = u'nun en az para sayısı (ya da ∞)
Kütüphane karşılığı	C: yok · C++: yok · Python: yok
Ne zaman kullanılır	Para sistemi kanonik değilse ya da bilinmiyorsa; en az adım problemleri
Ne zaman kullanılmaz	S çok büyükse (Θ(n·S): sözde polinom); kanonik sistemde açgözlü yeter

7 · C gerçekleştirimi. ∞ `INT_MAX`'tır ve toplamadan önce denetlenir:

C `c/H13_03_bozuk_para_en_az.c · satır 33–44`

```c
    for (int t = 1; t <= S; t++) {
        enaz[t] = SONSUZ;                  /* henüz ulaşılamadı */
        secim[t] = 0;
        for (int i = 0; i < k; i++) {
            int c = paralar[i];
            if (c <= t && enaz[t - c] != SONSUZ &&    /* taşma koruması */
                enaz[t - c] + 1 < enaz[t]) {
                enaz[t] = enaz[t - c] + 1;
                secim[t] = c;
            }
        }
    }
```

8 · Üç dil. ∞ üç dilde aynı tamsayı sabitidir (Python'da `math.inf` yazdırılınca `inf` çıkar ve çıktı farklılaşırdı):

C++ `H13_03_bozuk_para_en_az.cpp · satır 40–43`

```cpp
            if (c <= t && enaz[t - c] != SONSUZ &&    // taşma koruması
                enaz[t - c] + 1 < enaz[t]) {
                enaz[t] = enaz[t - c] + 1;
                secim[t] = c;
```

Python `H13_03_bozuk_para_en_az.py · satır 32–35`

```python
            if (c <= t and enaz[t - c] != SONSUZ
                    and enaz[t - c] + 1 < enaz[t]):
                enaz[t] = enaz[t - c] + 1
                secim[t] = c
```

9 · Test. Açgözlü ile DP t = 1 … 20 için karşılaştırılır: `farklı tutar sayısı: 4 / 20` (6, 10, 14, 18); {4, 6} ile `tutar 7: ulaşılamaz (enaz = ∞); açgözlü: çözüm bulamaz`; kanonik {1, 5, 10, 25, 50, 100} sisteminde açgözlü kâhindir: `t = 1..1000 için açgözlü = DP: 1000 / 1000`. **10 · Deney.** S = 1 000 ve altı para türünde 1 001 hücre ve en çok 6 000 deneme. **11 · Varyantlar.** Her paradan en çok bir tane (0/1, 4.6), ödemeleri saymak (4.2.1).

4.2.1 Kaç farklı yolla ödenir? Döngü sırası tuzağı

Aynı tablo **sayma** için de kullanılır: yol[t] = Σ yol[t − c], yol[0] = 1 (boş ödeme). “2 + 1 ile 1 + 2 aynı ödeme mi?” sorusunun yanıtını **döngülerin sırası** verir (Örnek 4, Şekil 13.6). Dış döngü **para** ise c işlenirken yol[t] yalnız ilk paralarla yapılan ödemeleri sayar, her ödeme bir kez: **kombinasyon** ({1, 2, 5} ile 5: 4). Dış döngü **tutar** ise her para her tutarın son adımı olabilir, sıralı diziler sayılır: **permütasyon** (9). Program iki sayıyı ödemeleri tek tek listeleyerek doğrular (`listelenen = 9; tablo ile aynı: evet`); yanlış sıra Hata Avcısı 3'tür. Kombinasyon döngüsü aşağıdadır; `permutasyon` aynı iki döngünün yer değiştirmiş hâlidir:

Şekil 13.6
Örnek 4: {1, 2, 5}, tutar 5 — dış döngü para (kombinasyon: her paradan sonraki satır) ve dış döngü tutar (permütasyon: yol[t]'ye katkılar)

Dış döngü PARA → kombinasyon

t	0	1	2	3	4	5
başta	1	0	0	0	0	0
+ para 1	1	**1**	**1**	**1**	**1**	**1**
+ para 2	1	1	**2**	**2**	**3**	**3**
+ para 5	1	1	2	2	3	**4**

yol[5] = 4: 5, 2+2+1, 2+1+1+1, 1+1+1+1+1

her ödeme bir kez (sıra önemsiz)

Dış döngü TUTAR → permütasyon

t	Σ yol[t − c] ← c	yol
1	1 ← c: 1	**1**
2	1 + 1 ← c: 1,2	**2**
3	2 + 1 ← c: 1,2	**3**
4	3 + 2 ← c: 1,2	**5**
5	5 + 3 + 1 ← c: 1,2,5	**9**

yol[5] = 9: 1+2 ile 2+1 ayrı sayılır

her paranın son adım olabildiği diziler

C `c/H13_04_bozuk_para_yollar.c · satır 44–48`

```c
    for (int i = 0; i < k; i++) {          /* DIŞ: para */
        int c = paralar[i];
        for (int t = c; t <= S; t++) {     /* İÇ: tutar */
            yol[t] = (yol[t] + yol[t - c]) % MOD;
        }
```

4.3 En uzun artan alt dizi (LIS)

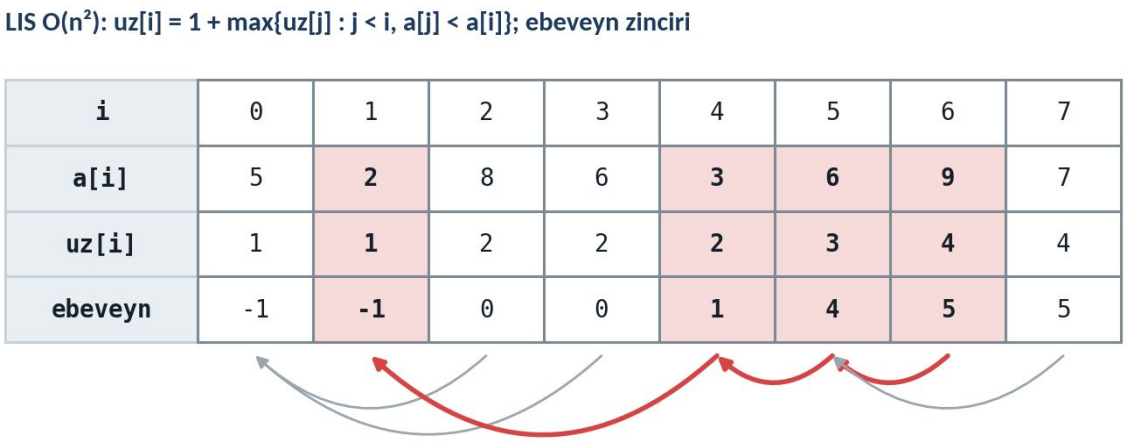
1 · Problem. Öğeleri **kesin** artan, sırası korunan (ardışık olması gerekmeyen) en uzun alt dizi: a = {5, 2, 8, 6, 3, 6, 9, 7} için 4. **2 · Fikir.** i'de biten en uzun artan alt dizinin bir önceki öğesi, a[j] < a[i] olan bir j'de biten en uzun dizinin sonudur. **Reçete:** durum uz[i] = a[i] ile biten en uzun artan alt dizi · geçiş 1 + max{uz[j] : j < i, a[j] < a[i]} · temel uz[i] = 1 · sıra i artan · yanıt max uz[i] · geri çıkarma ebeveyn[i].

3 · Elle izleme. i = 5'te a[5] = 6; küçük önceki öğeler j = 0, 1, 4, adaylar uz[j] + 1 = 2, 2, 3: uz[5] = 3, ebeveyn[5] = 4 (eşitlikte en küçük j). Sonda en büyük uz'un ilk indisinden ebeveyn zinciri izlenir (Şekil 13.7). **Tahmin edin** (şekle bakmadan): a[7] = 7. Önceki öğelerden hangileri daha küçüktür, adayların uz[j] + 1 değerleri nedir? Eşitlikte hangi j tutulur?

SÖZDE KOD · 4 · Sözde kod: O(n²) LIS ve ebeveyn dizisi

```
algoritma LIS(a[0..n−1])
    son ← 0
    i ← 0'dan n−1'e kadar her i için
        uz[i] ← 1;  ebeveyn[i] ← −1
        j ← 0'dan i−1'e kadar her j için
            eğer a[j] < a[i] ve uz[j] + 1 > uz[i] ise
                uz[i] ← uz[j] + 1;  ebeveyn[i] ← j
        eğer uz[i] > uz[son] ise son ← i
    döndür son          ▷ zincir: son, ebeveyn[son], …
```

Şekil 13.7
Örnek 7: a, uz ve ebeveyn dizileri; ebeveyn okları ve geri çıkarılan zincir (kırmızı)



en uzun: uz = 4, son indis 6; zincir 6 → 5 → 4 → 1; alt dizi 2 3 6 9 (kırmızı: geri çıkarılan zincir)

5 · Doğruluk. Değişmez: i. dış adımdan sonra uz[0..i] doğrudur. i'de biten en iyi alt dizinin sondan bir önceki öğesi bir j < i'dedir ve o j'de biten kısım da en iyi olmalıdır; iç döngü bütün j'leri dener. **6 · Analiz.** İç döngü n(n − 1)/2 kez döner: her durumda Θ(n²): n = 8'de program tam 28 karşılaştırma sayar; ek bellek Θ(n).

Tablo 13.5
En uzun artan alt dizi (LIS) kimlik kartı

ALGORİTMA KİMLİK KARTI	**En uzun artan alt dizi (LIS)** *(longest increasing subsequence)*
Strateji	DP (1B): uz[i] = 1 + max uz[j] (j < i, a[j] < a[i]); ebeveyn dizisi

Girdi önkoşulu	Karşılaştırılabilir öğeler; “artan” = KESİN artan (a[j] < a[i])
Zaman (en iyi · ortalama · en kötü)	$\Theta(n^2)$ · $\Theta(n^2)$ · $\Theta(n^2)$
Ek bellek	O(n)
Özellikler	Rastgele n = 16: uzunluk 7 (2^{16} alt kümeli kâhinle aynı)
Değişmez / doğruluk	i. adımdan sonra her k ≤ i için uz[k] = a[k] ile biten en uzun artanın boyu
Kütüphane karşılığı	C: yok · C++: yok · Python: yok
Ne zaman kullanılır	Sırası korunan en uzun zincir: sürüm dizileri, kutu iç içe geçirme
Ne zaman kullanılmaz	n büyükse $O(n^2)$ sürüm (n = 10^5 → $5{\cdot}10^9$ karşılaştırma)

7 · C gerçekleştirimi. uz ve ebeveyn çağıranın dizileridir:

C `c/H13_07_lis_n2.c · satır 28–41`

```c
    for (int i = 0; i < n; i++) {
        uz[i] = 1;                              /* temel: yalnız a[i] */
        ebeveyn[i] = -1;
        for (int j = 0; j < i; j++) {
            karsilastirma++;
            if (a[j] < a[i] && uz[j] + 1 > uz[i]) {
                uz[i] = uz[j] + 1;
                ebeveyn[i] = j;
            }
        }
        if (uz[i] > uz[son]) {                  /* eşitlikte küçük i */
            son = i;
        }
    }
```

8 · Üç dil. C++'ta diziler `assign` ile, Python'da `uz[:] = …` dilim atamasıyla yerinde doldurulur; iç döngü aynıdır:

C++ `H13_07_lis_n2.cpp · satır 38–41`

```cpp
            if (a[j] < a[i] && uz[j] + 1 > uz[i]) {
                uz[i] = uz[j] + 1;
                ebeveyn[i] = j;
            }
```

Python `H13_07_lis_n2.py · satır 29–31`

```python
            if a[j] < a[i] and uz[j] + 1 > uz[i]:
                uz[i] = uz[j] + 1
                ebeveyn[i] = j
```

9 · Test. Kâhin bütün 2^n alt kümeyi tarar (n = 16'da 2^{16} alt küme: kâhin de 7 bulur); sınırlar: artan dizi 8, azalan dizi 1, hepsi eşit 1 (“kesin” artan). **10 · Deney.** n = 10^5'te $O(n^2)$ sürüm 4 999 950 000 karşılaştırma isterdi; ikili aramalı sürüm 858 287 yoklamayla uzunluğu 618 bulur (Örnek 13). **11 · Varyantlar.** Azalmayan (≤) ve azalan alt dizi, kutuları iç içe geçirme, O(n log n) sabır sıralaması (Bölüm 13.1).

4.4 En uzun ortak alt dizi (LCS)

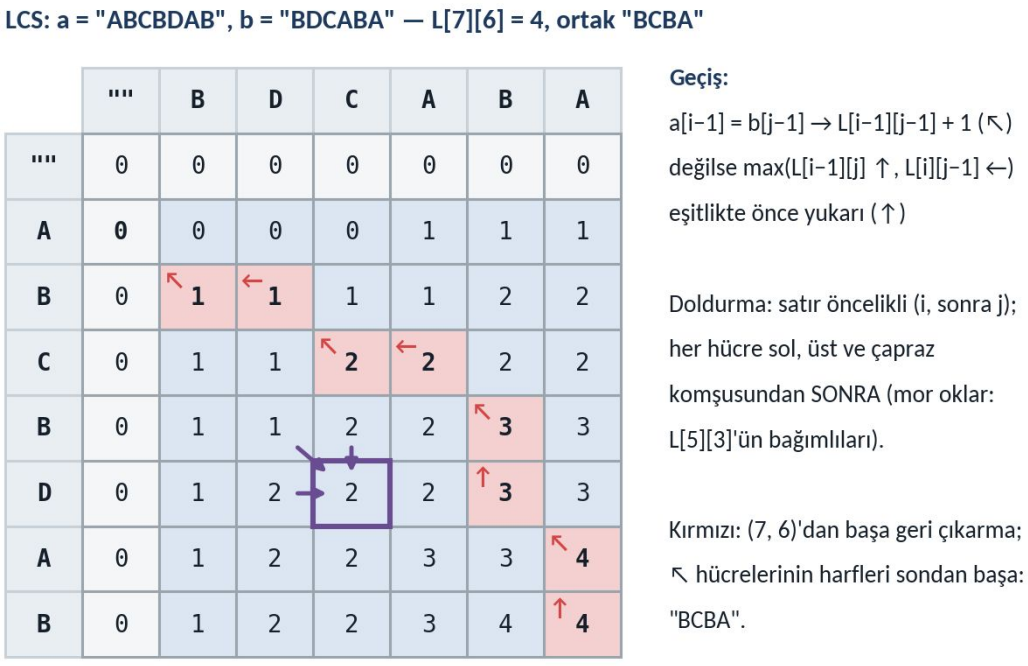
1 · Problem. İki dizinin (karakter, satır, DNA bazı) ikisinde de aynı sırayla geçen en uzun alt dizisi: “ABCBDAB” ile “BDCABA” için 4, bir yanıt “BCBA” (Cormen vd., 2022). **2 · Fikir.** Önekleri karşılaştırırız: son öğeler eşitse ortak alt dizinin sonuna konabilir; değilse en az biri kullanılmaz ve iki küçük problemin iyisi alınır. **Reçete:** durum L[i][j] = a[0..i) ile b[0..j)'nin LCS'si · geçiş a[i−1] = b[j−1] ise L[i−1][j−1] + 1, değilse max(L[i−1][j], L[i][j−1]) · temel L[0][j] = L[i][0] = 0 · sıra satır öncelikli · yanıt L[n][m] · geri çıkarma (n, m)'den.

3 · Elle izleme. Tablo (n + 1)(m + 1) = 56 hücre (Şekil 13.8); her hücre üst, sol ve çapraz komşusundan sonra doldurulur. Geri çıkarma (7, 6)'dan başlar: harfler eşitse çapraza gidip harfi yazar, değilse büyük komşuya (eşitlikte yukarı) gider; çapraz adımların harfleri sondan başa “BCBA”yı verir. **Tahmin edin** (şekle bakmadan): a'nın ilk dört harfi “ABCB”, b'nin ilk beş harfi “BDCAB”. Ortak alt dizilerinden en uzunu hangisi? Bu hücreye hangi komşudan gelinir?

SÖZDE KOD · 4 · Sözde kod: LCS tablosu

```
algoritma LCS(a[0..n−1], b[0..m−1])
    her i için L[i][0] ← 0;  her j için L[0][j] ← 0
    i ← 1'den n'ye, j ← 1'den m'ye kadar (satır öncelikli)
        eğer a[i−1] = b[j−1] ise L[i][j] ← L[i−1][j−1] + 1
        değilse L[i][j] ← max(L[i−1][j], L[i][j−1])
    döndür L[n][m]
```

Şekil 13.8
Örnek 8: LCS tablosu (satır: a'nın önekleri, sütun: b'nin önekleri), bir hücrenin bağımlılıkları (mor) ve geri çıkarma yolu (kırmızı)



5 · Doğruluk. Değişmez: L[i][j], a[0..i) ile b[0..j)'nin LCS uzunluğudur. a[i−1] = b[j−1] ise bu harfi sona koyan bir en iyi çözüm vardır; değilse en iyi çözüm a[i−1]'i ya da b[j−1]'i kullanmaz ve ilgili alt problem en iyi olmalıdır. **6 · Analiz.** Θ(n·m) zaman, tam tabloyla Θ(n·m) bellek (yalnız değer için iki satır: Bölüm 3.9); geri çıkarma O(n + m).

Tablo 13.6
LCS ve düzenleme (Levenshtein) uzaklığı kimlik kartı

ALGORİTMA KİMLİK KARTI **LCS ve düzenleme (Levenshtein) uzaklığı** *(longest common subsequence / edit (Levenshtein) distance)*	
Strateji	DP (2B, önek çiftleri): L[i][j], D[i][j]; satır öncelikli; geri çıkarma
Girdi önkoşulu	İki dizi (karakter, satır, kod noktası); eşitlikte yön sabit
Zaman (en iyi · ortalama · en kötü)	Θ(n·m) · Θ(n·m) · Θ(n·m)
Ek bellek	O(n·m) / O(min(n, m))
Özellikler	"BCBA" (4); kitten → sitting 3; iki satırla O(min(n, m)) bellek
Değişmez / doğruluk	L[i][j] (D[i][j]) = a[0..i) ve b[0..j) öneklerinin LCS'si (uzaklığı)
Kütüphane karşılığı	C: yok · C++: yok · Python: yok
Ne zaman kullanılır	diff, yazım düzeltme ("Bunu mu demek istediniz?"), DNA hizalama
Ne zaman kullanılmaz	Çok uzun dizilerde tam tablo (yalnız değer gerekiyorsa iki satır)

7 · C gerçekleştirimi. Tablo tek bir `malloc` bloğudur, L[i·(m + 1) + j] ile indislenir (satır öncelikli; Örnek 6):

C `c/H13_08_lcs.c · satır 34–45`

```c
    for (int i = 1; i <= n; i++) {
        L[i * s] = 0;                                   /* L[i][0] = 0 */
        for (int j = 1; j <= m; j++) {
            if (a[i - 1] == b[j - 1]) {
                L[i * s + j] = L[(i - 1) * s + j - 1] + 1;
            } else if (L[(i - 1) * s + j] >= L[i * s + j - 1]) {
                L[i * s + j] = L[(i - 1) * s + j];
            } else {
                L[i * s + j] = L[i * s + j - 1];
            }
        }
    }
```

8 · Üç dil. C++'ta `lcs_tablo` bir şablondur: aynı kod `std::string` ile karakterlerde, `std::vector<std::string>` ile satırlarda çalışır; Python'da aynı fonksiyon `str` ve `list` alır:

C++ `H13_08_lcs.cpp · satır 35–38`

```cpp
            if (a[i - 1] == b[j - 1]) {
                L[i][j] = L[i - 1][j - 1] + 1;
            } else if (L[i - 1][j] >= L[i][j - 1]) {
                L[i][j] = L[i - 1][j];
```

Python `H13_08_lcs.py · satır 24–27`

```python
            if a[i - 1] == b[j - 1]:
                L[i][j] = L[i - 1][j - 1] + 1
            elif L[i - 1][j] >= L[i][j - 1]:
                L[i][j] = L[i - 1][j]
```

9 · Test. Geri çıkarılan dizinin iki girdinin de alt dizisi olduğu iki işaretçiyle denetlenir (“BCBA” iki girdinin de alt dizisidir: evet). Aynı DP satırlara uygulanınca bir **diff** çıkar: LCS'de olmayan satırlar silinmiş (−) ya da eklenmiştir (+); çay demleme tarifinin iki sürümünde 4 satır ortak, 1 satır silinmiş, 2 satır eklenmiştir.

10 · Deney. Rastgele iki DNA dizisinde (n = m = 300) LCS 199 (oran 0,6633), 90 601 hücre; doldurma sırasının süreye etkisi Bölüm 6.4'tedir. **11 · Varyantlar.** En uzun ortak alt dizgi (ardışık), üç dizinin LCS'si (3B tablo), en kısa ortak üst dizi (n + m − LCS), `diff` (Hunt & McIlroy, 1976).

4.5 Düzenleme (Levenshtein) uzaklığı ve “Bunu mu demek istediniz?”

1 · Problem. a'yı b'ye çeviren en az tek karakterlik **silme**, **ekleme** ve **değiştirme** sayısı (Levenshtein, 1965/1966; Wagner & Fischer, 1974): “kitten” → “sitting” = 3. **2 · Fikir.** LCS'nin tablosu, ama en küçük **maliyet**: a[i−1]'i silebilir, b[j−1]'i ekleyebilir ya da a[i−1]'i b[j−1]'e çevirebiliriz (eşitse maliyetsiz eşleşme). **Reçete:** durum D[i][j] · geçiş min(D[i−1][j] + 1, D[i][j−1] + 1, D[i−1][j−1] + [a[i−1] ≠ b[j−1]]) · temel D[i][0] = i, D[0][j] = j · sıra satır öncelikli · yanıt D[n][m] · geri çıkarma: eşitlikte eşleşme > silme > ekleme > değiştirme.

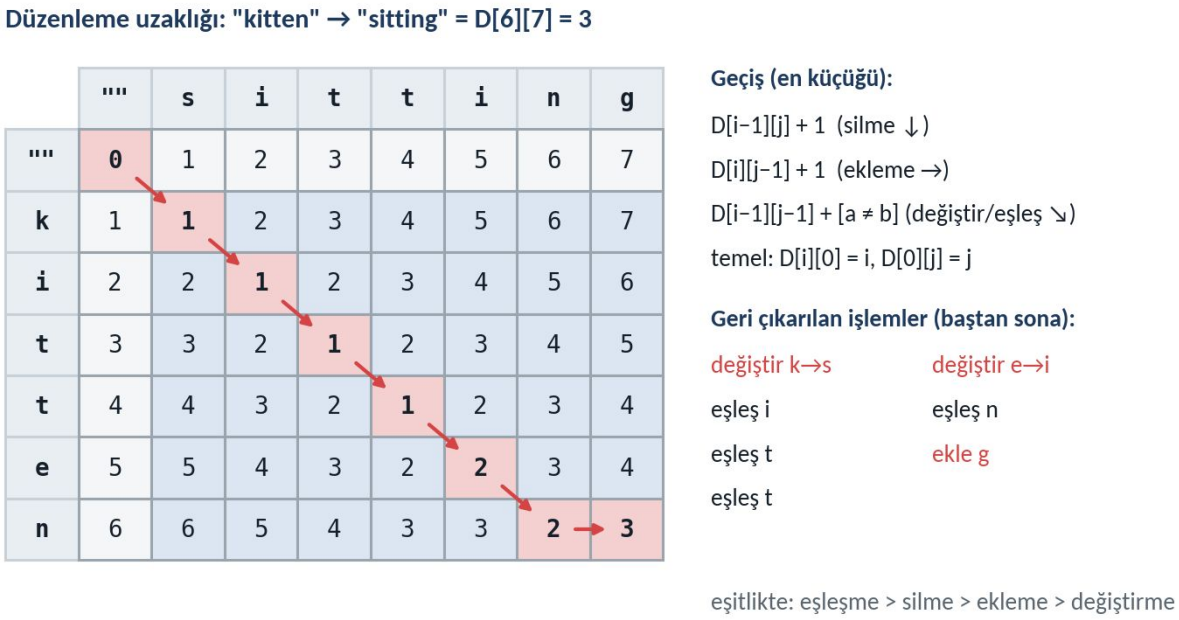
3 · Elle izleme. İlk satır ve sütun boş dizgiye uzaklıklardır (Şekil 13.9). Geri çıkarılan işlemler: değiştir k→s, eşleş i, eşleş t, eşleş t, değiştir e→i, eşleş n, ekle g (eşleşme 4, silme 0, ekleme 1, değiştirme 2).

SÖZDE KOD · 4 · Sözde kod: düzenleme uzaklığı

```
algoritma UZAKLIK(a[0..n−1], b[0..m−1])
    her i için D[i][0] ← i;  her j için D[0][j] ← j
    i ← 1'den n'ye, j ← 1'den m'ye kadar (satır öncelikli)
        sil ← D[i−1][j] + 1;  ekle ← D[i][j−1] + 1
        değiş ← D[i−1][j−1] + (a[i−1] ≠ b[j−1] ise 1 değilse 0)
        D[i][j] ← min(sil, ekle, değiş)
    döndür D[n][m]
```

5 · Doğruluk. En iyi işlem dizisinin **son** işlemi ya a[i−1]'in silinmesi, ya b[j−1]'in eklenmesi ya da a[i−1]'in b[j−1]'e çevrilmesi/eşlenmesidir; kalanı ilgili önekin en iyi çevirisidir (kimlik kartı LCS ile ortak: Tablo 13.6). **6 · Analiz.** Θ(n·m) zaman; yalnız uzaklık için iki satır yeter (Mini-OBS 2 v2.12 öneride bunu kullanır), işlem dizisi için tam tablo.

Şekil 13.9
Örnek 9: "kitten" → "sitting" düzenleme tablosu, geçiş kuralı ve geri çıkarılan işlemler (kırmızı yol)



7 · C gerçekleştirimi. Karakterler önce **kod noktası** dizisine çevrilir (`utf8_coz`); tablo tek bloktur:

C `c/H13_09_duzenleme_uzakligi.c · satır 75–83`

```c
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            int sil = D[(i - 1) * s + j] + 1;
            int ekle = D[i * s + j - 1] + 1;
            int degis = D[(i - 1) * s + j - 1] + (a[i - 1] != b[j - 1]);
            int enk = sil < ekle ? sil : ekle;
            D[i * s + j] = degis < enk ? degis : enk;
        }
    }
```

8 · Üç dil. Üç dilde girdi **bayt** olarak okunur ve UTF-8 elle çözülür (Python'da `bytes` öğeleri 0–255 tamsayıdır); böylece "ü" her dilde tek karakter sayılır:

C++ `H13_09_duzenleme_uzakligi.cpp · satır 83–87`

```cpp
            int sil = D[i - 1][j] + 1;
            int ekle = D[i][j - 1] + 1;
            int degis = D[i - 1][j - 1] + (a[i - 1] != b[j - 1]);
            int enk = sil < ekle ? sil : ekle;
            D[i][j] = degis < enk ? degis : enk;
```

Python `H13_09_duzenleme_uzakligi.py · satır 63–67`

```python
            sil = D[i - 1][j] + 1
            ekle = D[i][j - 1] + 1
            degis = D[i - 1][j - 1] + (a[i - 1] != b[j - 1])
            enk = sil if sil < ekle else ekle
            D[i][j] = degis if degis < enk else enk
```

9 · Test. Türkçe harfler UTF-8'de iki bayttır; bayt düzeyindeki uzaklık her Türkçe harf hatasını iki kez sayar: `gunduz / gündüz: bayt = 4, kod noktası = 2`. “Bunu mu demek istediniz?” Ömer Seyfettin'in üç öyküsünden (“Diyet”, “Ferman”, “Forsa”; Seyfettin, 1918/2022, 1917/2023, 1919/2024) çıkarılmış 3 215 sözcükte en yakın üç öneriyi verir:

ÇIKTI (3. BÖLÜMÜN BAŞI)

```
$ ./H13_09_duzenleme_uzakligi < ../girdiler/H13_09_duzenleme_uzakligi.txt
3) "Bunu mu demek istediniz?" — sözlük: 3215 sözcük (stdin), k = 3
  öneri: kod noktası düzeyinde; eşit uzaklıkta bayt sırası
  gunduz -> gündüz (2), buldu (3), bulduk (3) | bayt düzeyinde 1.: buldu (3)
  sehre -> şehre (1), kere (2), sedire (2) | bayt düzeyinde 1.: kere (2)
  kılç -> kılıç (1), hıç (2), kal (2) | bayt düzeyinde 1.: hıç (2)
```

“gunduz”un bayt düzeyindeki ilk önerisi “buldu”, kod noktası düzeyinde “gündüz”dür: Türkçe metinde uzaklık **kod noktası** üzerinden ölçülmelidir. **10 · Deney.** 7 sorguda 1 178 100 hücre; v2.12 aynı işi 10^5 kayıtlık ad sözlüğünde budamayla ölçer (Bölüm 10). **11 · Varyantlar.** Ağırlıklı işlemler, yer değiştirmeyi tek işlem sayan Damerau–Levenshtein, uzunluk farkı budaması (Atölye 2), hizalamada eşleşme/boşluk puanları (Needleman & Wunsch, 1970).

4.6 0/1 sırt çantası: tam tablo, tek satır ve sözde polinom zaman

1 · Problem. n nesnenin ağırlıkları a_i, değerleri d_i, kapasite W (tamsayı). Her nesne **en çok bir kez** alınır; ağırlık toplamı W'yi aşmayan en değerli küme hangisi (Kleinberg & Tardos, 2006)? **2 · Fikir.** Son nesne ya alınmaz (ilk n − 1 nesne, W) ya alınır (ilk n − 1 nesne, W − a_n, + d_n). Alt problemin iki boyutu vardır. **Reçete:** durum K[i][w] = ilk i nesneyle w kapasitede en büyük değer · geçiş max(K[i−1][w], K[i−1][w − a_i] + d_i) · temel K[0][w] = 0 · sıra i, w artan · yanıt K[n][W] · geri çıkarma i = n … 1: K[i][w] ≠ K[i−1][w] ise i alındı, w −= a_i.

3 · Elle izleme. A(5, 10), B(4, 40), C(6, 30), D(3, 50), W = 10 (Şekil 13.10). Satır D'de K[4][7]: almadan K[3][7] = 40, alarak K[3][4] + 50 = 90. Yanıt K[4][10] = 90; geri çıkarma D'yi alır (w: 10 → 7), C'yi almaz, B'yi alır (7 → 3), A'yı almaz: {B, D}. **Tahmin edin** (şekle bakmadan): Satır C (ağırlık 6, değer 30): almadan K[2][10], alarak K[2][4] + 30. Satır B'nin değerlerini tablodan okuyun.

SÖZDE KOD · 4 · Sözde kod: tam tablo ve tek satır

```
algoritma SIRT-ÇANTASI(a[1..n], d[1..n], W)
    w ← 0'dan W'ye kadar K[0][w] ← 0
    i ← 1'den n'ye, w ← 0'dan W'ye kadar
        K[i][w] ← K[i−1][w]                              ▷ alma
        eğer a[i] ≤ w ve K[i−1][w−a[i]] + d[i] > K[i][w] ise
            K[i][w] ← K[i−1][w−a[i]] + d[i]              ▷ al
    döndür K[n][W]
GERİ-ÇIKAR: w ← W;  i ← n..1:  eğer K[i][w] ≠ K[i−1][w] ise i alındı, w ← w − a[i]
TEK-SATIR: i ← 1..n;  w ← W'den a[i]'ye AZALARAK:  dp[w] ← max(dp[w], dp[w − a[i]]
+ d[i])
```

Şekil 13.10
Örnek 10: 0/1 sırt çantası tablosu (nesne × kapasite); yeşil: "al" seçeneği kazandı, kırmızı: geri çıkarma

0/1 sırt çantası: K[i][w] (nesne × kapasite), W = 10 → K[4][10] = 90

	0	1	2	3	4	5	6	7	8	9	10
i = 0	0	0	0	**0**	0	0	0	0	0	0	0
1 A(5,10)	0	0	0	**0**	0	10	10	10	10	10	10
2 B(4,40)	0	0	0	0	40	40	40	**40**	40	50	50
3 C(6,30)	0	0	0	0	40	40	40	**40**	40	50	70
4 D(3,50)	0	0	0	50	50	50	50	90	90	90	**90**

geçiş: K[i][w] = max(K[i−1][w], K[i−1][w − a_i] + d_i); yeşil: "al" seçeneği kazandı

kırmızı: geri çıkarma — D alındı (w: 10 → 7), C alınmadı, B alındı (7 → 3), A alınmadı

K[i][w] = K[i−1][w] ise nesne ALINMADI sayılır (eşitlik kuralı)

5 · Doğruluk. Değişmez: i. satır dolduğunda K[i][w], ilk i nesnenin w'ye sığan alt kümelerinin en büyük değeridir. Geri çıkarmada K[i][w] = K[i−1][w] ise nesne i **alınmadı** sayılır; v2.12 belgesi § 4.2 bu kuralın "en küçük maskeyi" seçtiğini kanıtlar. **6 · Analiz.** (n + 1)(W + 1) hücre: Θ(n·W) zaman, tam tabloyla Θ(n·W), tek satırla Θ(W) bellek.

Tablo 13.7
0/1 sırt çantası kimlik kartı

ALGORİTMA KİMLİK KARTI **0/1 sırt çantası** *(0/1 knapsack)*	
Strateji	DP (2B, nesne × kapasite); tek satırda kapasite TERS sırada
Girdi önkoşulu	Tamsayı ağırlıklar ve kapasite W; her nesne en çok bir kez
Zaman (en iyi · ortalama · en kötü)	Θ(n·W) · Θ(n·W) · Θ(n·W)
Ek bellek	O(n·W) / O(W)
Özellikler	W = 10: 90 ({B, D}); oran açgözlüsü 160 ↔ DP 220; artan dolaşma 150
Değişmez / doğruluk	i. satır dolunca K[i][w] = ilk i nesnenin w'ye sığan en iyi değeri
Kütüphane karşılığı	C: yok · C++: yok · Python: yok
Ne zaman kullanılır	Kapasite altında seçim (AKTS sınırıyla ders seçimi, v2.12); W küçükse
Ne zaman kullanılmaz	W çok büyükse (W'nin bit uzunluğunda üstel); kesirli sürümde açgözlü

7 · C gerçekleştirimi. Tablo tek bloktur; `secilenler` aynı dosyada K[i·s + w] ≠ K[(i − 1)·s + w] ise nesneyi alır (eşitlikte alınmadı):

C `c/H13_10_sirt_cantasi.c · satır 38–51`

```c
    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            int al_ma = K[(i - 1) * s + w];
```

```
            int en_iyi = al_ma;
            if (a[i - 1] <= w) {
                int al = K[(i - 1) * s + w - a[i - 1]] + d[i - 1];
                if (al > en_iyi) {
                    en_iyi = al;
                }
            }
            K[i * s + w] = en_iyi;
            hucre++;
        }
    }
```

8 · Üç dil. C++'ta `std::vector<std::vector<int>>`, Python'da satırları ayrı listeler (`[[0] * (W + 1) for _ in range(n + 1)]`; Örnek 6'nın tuzağı):

C++ **H13_10_sirt_cantasi.cpp · satır 40–45**

```
            if (a[i - 1] <= w) {
                int al = K[i - 1][w - a[i - 1]] + d[i - 1];
                if (al > en_iyi) {
                    en_iyi = al;
                }
            }
```

Python **H13_10_sirt_cantasi.py · satır 30–33**

```
            if a[i - 1] <= w:
                al = K[i - 1][w - a[i - 1]] + d[i - 1]
                if al > en_iyi:
                    en_iyi = al
```

9 · Test. Kâhin bütün alt kümeleri dener: 12 dersin AKTS sınırı 30 altında DP 117 puanı (29 AKTS: {D01, D05, D06, D09, D10, D11, D12}) 372 hücrede bulur; 2^{12} = 4 096 alt kümeyi deneyen kâhin de 117 bulur. H12'nin oran açgözlüsü burada yanılır: {X, Y} = 160 ↔ DP {Y, Z} = 220; 12 derste oran açgözlüsü 111 puanda kalır.

4.6.1 Tek satıra inmek: kapasite neden ters dolaşılır?

K[i][·] yalnız K[i − 1][·]'e bağlıdır; tek dizi dp[0..W] yeter: i. nesne işlenirken dp "önceki satırdır" ve yerinde güncellenir. dp[w], dp[w − a_i]'yi okur. w **azalarak** dolaşılırsa dp[w − a_i] bu nesneyle henüz güncellenmemiştir (önceki satır); **artarak** dolaşılırsa bu turda güncellenmiş olabilir ve nesne ikinci kez alınır: sonuç **sınırsız** sırt çantasıdır (Örnek 11, Şekil 13.11; Hata Avcısı 4). Doğru döngü (Hata Avcısı 4'ün düzeltilmiş sürümü):

C **hata_avcisi/H13_HA4_artan_dolasma_duzeltilmis.c · satır 18–22**

```
        for (int w = W; w >= a[i]; w--) {  /* ters */
            if (dp[w - a[i]] + d[i] > dp[w]) {
                dp[w] = dp[w - a[i]] + d[i];
            }
        }
```

Şekil 13.11
Örnek 11: tek satır, her nesneden sonra dp[0..W]; üstte ters dolaşma (doğru 0/1, dp[10] = 90), altta artan (yanlış: D birden çok kez, dp[10] = 150)

TERS: w = W, …, a_i — doğru 0/1

w	0	1	2	3	4	5	6	7	8	9	10
+A	0	0	0	0	0	**10**	**10**	**10**	**10**	**10**	**10**
+B	0	0	0	0	**40**	**40**	**40**	**40**	**40**	**50**	**50**
+C	0	0	0	0	40	40	40	40	40	50	**70**
+D	0	0	0	**50**	**50**	**50**	**50**	**90**	**90**	**90**	**90**

D(3, 50), ters: dp[w − 3] hâlâ önceki satırın değeri; dp[10]: 70 → 90

ARTAN: w = a_i, …, W — sınırsız (YANLIŞ)

w	0	1	2	3	4	5	6	7	8	9	10
+A	0	0	0	0	0	**10**	**10**	**10**	**10**	**10**	**20**
+B	0	0	0	0	**40**	**40**	**40**	**40**	**80**	**80**	**80**
+C	0	0	0	0	40	40	40	40	80	80	80
+D	0	0	0	**50**	**50**	**50**	**100**	**100**	**100**	**150**	**150**

D(3, 50), artan: dp[3] = 50 olur, sonra dp[6] = dp[3] + 50 = 100 (D iki kez); dp[9] = 150 (üç)

Bedeli: seçilen nesneler artık geri çıkarılamaz (önceki satırlar silindi). Kazancı: n = 100, W = 100 000'de tam tablo 10 100 101, tek satır 100 001 hücre (Örnek 11). **10 · Deney.** v2.12 belgesi § 8.2'de 10 011 001 hücrede tam tablo 30,1 ms, tek satır 6,5 ms sürer: tek satır L1 önbellekte kalan küçük bir diziyi günceller, tam tablo yeni belleğe yazar.

4.6.2 Sözde polinom zaman

Θ(n·W) polinom görünür ama W bir sayının **değeridir**. Verimlilik girdinin **uzunluğuna** (bit sayısına) göre ölçülür; W'yi yazmak b ≈ $\log_2$ W bit ister, W ≈ 2^b'dir ve Θ(n·W) = Θ(n·2^b): bit uzunluğunda **üstel**. Böyle algoritmalara **sözde polinom** (pseudo-polynomial) denir. Kavram 4'te W'nin bit uzunluğu 8'den 16'ya çıkınca hücre sayısı 5 140'tan 1 310 740'a çıkar (255,0 kat; grafik: sunum ve web sayfası). AKTS gibi küçük kapasitelerde DP çok hızlıdır, W = 10^9 gibi değerlerde pratik değildir. 0/1 sırt çantası için polinom zamanlı bir algoritma bilinmemesi NP-zorlukla ilgilidir: H14'ün konusu. **11 · Varyantlar.** Sınırsız sırt çantası (artan dolaşma doğrudur; çubuk kesme, Alıştırma 13.13), kesirli sırt çantası (açgözlü yeter; H12), aynı dönemde önkoşuluyla birlikte ders seçimi (ağaç DP'si; v2.12 belgesi § 4.4'te kapsam dışı).

4.7 Aynı problem, dört strateji: alt küme toplamı

1 · Problem. Pozitif tamsayılar a_1 … a_n ve hedef T: toplamı T olan bir alt küme var mı, kaç tane? H3 bunu kaba kuvvet ve budamalı geri izlemeyle çözmüştü (aynı üreteç: n = 18, tohum 42). **2 · Fikir.** 0/1 sırt çantasının değersiz hâli. **Reçete:** durum var[i][t] (say[i][t]: kaç alt küme) · geçiş var[i−1][t] VEYA var[i−1][t − a_i]; say toplanır · temel var[0][0] = 1 · sıra i, t artan · yanıt var[n][T] · geri çıkarma i = n … 1. **3 · Elle izleme.** a = {5, 4, 3, 2}, hedef 7 (H3 Örnek 8; Şekil 13.12): `say[4][7] = 2 alt küme; bir çözüm (eşitlikte almadan): {4, 3}`.

4 · Sözde kod. SIRT-ÇANTASI'nın max yerine VEYA ile yazılmışı (4.6). **5 · Doğruluk.** Bir alt küme a_i'yi ya içerir (kalanı ilk i − 1 sayıdan t − a_i yapar) ya içermez; durumlar ayrık olduğundan sayım toplamla yapılır. **6 · Analiz.** Θ(n·T) hücre (sözde polinom); kaba kuvvet $2^{n+1} - 1$ düğüm, budama yalnız sabiti küçültür. **7 · C gerçekleştirimi.** Bool tablo `unsigned char`, sayım `long long`; ikisi tek blokta, `calloc` ile sıfırlı:

C `c/H13_12_alt_kume_toplami.c · satır 70–81`

```c
    for (int i = 1; i <= n; i++) {
        int x = a[i - 1];
        for (int t = 0; t <= T; t++) {
            var_t[i * s + t] = var_t[(i - 1) * s + t];        /* almadan */
            say_t[i * s + t] = say_t[(i - 1) * s + t];
            if (x <= t) {                                     /* alarak */
                var_t[i * s + t] |= var_t[(i - 1) * s + t - x];
                say_t[i * s + t] += say_t[(i - 1) * s + t - x];
            }
            hucre++;
        }
    }
```

Şekil 13.12
Örnek 12: var[i][t] tablosu (a = {5, 4, 3, 2}, hedef 7) ve n = 18'de dört stratejinin iş sayıları (logaritmik çubuklar)

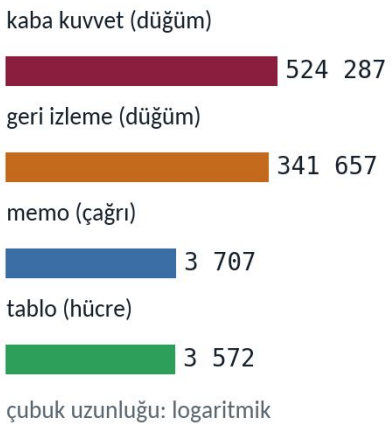
var[i][t]: a = {5, 4, 3, 2}, hedef 7

	0	1	2	3	4	5	6	7
i = 0	1	0	0	0	0	0	0	0
1 (5)	1	0	0	0	0	1	0	0
2 (4)	1	0	0	0	1	1	0	0
3 (3)	1	0	0	1	1	1	0	1
4 (2)	1	0	1	1	1	1	1	1

1 (yeşil): ilk i sayıyla t yapılabilir; say[4][7] = 2 alt küme

var[i][t] = var[i−1][t] VEYA var[i−1][t − a_i]

n = 18 (Örnek 12'nin girdisi)



çubuk uzunluğu: logaritmik

8 · Üç dil. C++'ta `std::vector<std::vector<bool>>` (bit paketli; `|=` yok, `||` yazılır), Python'da `bool` listeleri ve `or`:

C++ `H13_12_alt_kume_toplami.cpp · satır 68–71`

```cpp
            if (x <= t) {                                     // alarak
                var_t[i][t] = var_t[i][t] || var_t[i - 1][t - x];
                say_t[i][t] += say_t[i - 1][t - x];
            }
```

Python H13_12_alt_kume_toplami.py · satır 58–60

```
58              if x <= t:                                  # alarak
59                  var_t[i][t] = var_t[i][t] or var_t[i - 1][t - x]
60                  say_t[i][t] += say_t[i - 1][t - x]
```

9 · Test. Geri izlemenin bulduğu çözüm sayısı DP'nin say[n][T]'siyle aynıdır (n = 18, hedef 187: 2 090 çözüm). **10 · Deney.** n = 18'de kaba kuvvet 524 287 düğüm, budamalı geri izleme 341 657 düğüm, memo 3 707 çağrı, tablo 3 572 hücre; C'de n = 26'da kaba kuvvet 584,7 ms, tablo 0,0057 ms (102 680 kat; Bölüm 6.2). **11 · Varyantlar.** Eşit toplamlı iki parçaya bölme (T = toplam/2), en az farkla bölme, v2.12'nin dört stratejili ders seçimi.

4.7.1 Strateji karşılaştırması ve açgözlü ↔ DP

Şekil 13.13 dört çözümü doğruluk, zaman ve bellek açısından karşılaştırır. **Açgözlü** (büyükten küçüğe sığanı al) hızlıdır ama bu girdi ailesinde ulaşılabilir 2 027 hedefin yalnız 1 461 tanesini (%72,1) tutturdu. Haftanın iki karşı örneği de aynı dersi verir: {1, 3, 4} ile 6 (açgözlü 3, DP 2 para) ve 0/1 sırt çantasında {X, Y} = 160 ↔ {Y, Z} = 220. İkisinde de **en iyi alt yapı** vardır, **açgözlü seçim özelliği** yoktur: DP bütün seçenekleri dener ama her alt problemi bir kez çözer. Kesirli sırt çantasında ve kanonik para sistemlerinde açgözlü seçim özelliği vardır; orada açgözlü yeter ve daha hızlıdır.

Şekil 13.13
Strateji karşılaştırma matrisi: alt küme toplamında kaba kuvvet, budamalı geri izleme, açgözlü ve DP (doğruluk / zaman / bellek)

Aynı problem, dört strateji: alt küme toplamı (n = 18 sayaçları; n = 26 süreleri, C, bu makinede)

Strateji	Doğruluk	Zaman (iş)	Bellek
Kaba kuvvet (H3)	her zaman doğru	Θ(2ⁿ): 524 287 düğüm; 584,7 ms	O(n) yığın
Budamalı geri izleme (H3)	her zaman doğru	en kötü O(2ⁿ): 341 657 düğüm; 386,6 ms	O(n) yığın
Açgözlü (H12)	YANLIŞ olabilir: %72,1 doğru*	O(n log n) (sıralama)	O(1) ek
DP (H13)	her zaman doğru	Θ(n·T): 3 572 hücre; 0,0057 ms	O(n·T); tek satırla O(T)

* n = 10…40, ulaşılabilir 2 027 hedefin 1 461 tanesinde tam tutturdu (büyükten küçüğe sığanı al)

5. Çözümlü Örnekler ve Uygulamalar

Dört örnek, reçeteyi yeni tablo biçimlerine taşır: 2B ızgara (Örnek 5), tablonun bellekte kurulması (Örnek 6), önek DP'si (Örnek 14) ve aralık DP'si (Örnek 15). Son ikisi meydan okumadır; ikili aramalı LIS (Örnek 13) Merak Kutusu'ndadır (Bölüm 13).

5.1 Izgarada yol sayısı ve en küçük maliyetli yol (Örnek 5)

Yol sayısı. Yalnız sağa ya da aşağı gidilen bir ızgarada (i, j) hücresine ya üstünden ya solundan gelinir: **durum** yol[i][j], **geçiş** yol[i − 1][j] + yol[i][j − 1], **temel** ilk satır ve sütun 1, **sıra** satır satır. 3 × 4 ızgarada yanıt yol[2][3] = 10 = C(5, 2) (Şekil 13.14, solda). n × n ızgarada yanıt C(2n − 2, n − 1)'dir ve hızla büyür: n = 30'da tam değer 30 067 266 499 541 040 (64 bite sığar), daha büyük n'de program her toplamadan sonra mod $10^9 + 7$ alır ve tek satır tamponu kullanır (Bölüm 8). Engelli hücre 0 yoldur: engelli 5 × 6 ızgarada 13 yol vardır (engelsiz 126).

En küçük maliyet. Her hücrenin bir maliyeti var; sol üstten sağ alta en ucuz yol? Geçiş m[i][j] = a[i][j] + min(m[i − 1][j], m[i][j − 1]); engel ∞'dur ve ∞'a maliyet eklenmez (Hata Avcısı 5). Geri çıkarma sağ alttan, hangi komşunun küçük olduğuna bakarak (eşitlikte yukarı) yürür: en küçük maliyet 15, yol 10 hücre.

Şekil 13.14
Örnek 5: 3 × 4 ızgarada yol sayısı ve engelli ızgarada en küçük maliyet; hücrede maliyet (gri) ve m[i][j] (kalın), kırmızı: geri çıkarılan yol



5.2 2B tabloyu kurmak: tek blok, satır vektörleri ve Python tuzağı (Örnek 6)

DP tablosu bellekte üç dilde farklı kurulur (Şekil 13.15). C'de `malloc(n * m * sizeof *t)` **tek blok** ayırır ve (i, j) hücresi t[i·m + j]'dir: satırlar bitişiktir (satır öncelikli; AP-1 H11). C++'ta `std::vector<std::vector<int>>` her satırı ayrı bir vektör yapar (satırlar bitişik olmak zorunda değildir). Python'da `[[0] * m] * n` **tuzaktır**: `[0] * m` bir kez oluşur, `* n` aynı listeye n başvuru koyar.

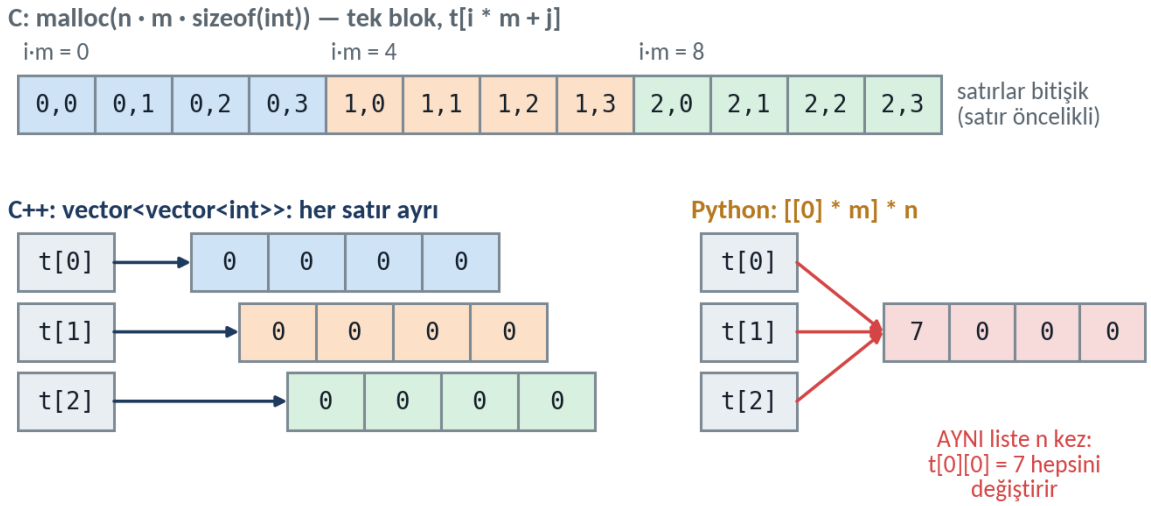
Örnek 6'nın Python sürümünde t[0][0] = 7 yazılınca öteki satırların ilk hücreleri `7 7 7 7 7` olur (`t[0] is t[5]`: True) ve aynı tabloyla Pascal üçgeni C(5, 2) = 13 verir (doğrusu 10): "önceki satır" diye okunan hücre, aynı satırın bu turda değişmiş hâlidir. Bu yüzden Örnek 6'nın çıktısı dillerde **farklıdır** (Δ); C++ karşılığı `Tablo(n, std::vector<int>(m, 0))` her satırı kopyalar:

Python `python/H13_06_tablo_kurma.py · satır 17–24`

```
17  def tablo_kur_tuzak(n, m):
18      """YANLIŞ: [0] * m bir kez oluşur; * n aynı listeye n başvuru koyar"""
19      return [[0] * m] * n
20
21
22  def tablo_kur(n, m):
23      """DOĞRU: liste üreteci her satır için YENİ bir liste oluşturur"""
```

```
24      return [[0] * m for _ in range(n)]
```

Şekil 13.15
*Örnek 6: C'de tek blok (t[i·m + j], satırlar bitişik), C++'ta ayrı satır vektörleri, Python'da [[0] * m] * n ile aynı satırın n kez paylaşılması*



Doğrusu (Python): [[0] * m for _ in range(n)] — her satır yeni liste

5.3 Meydan okuma: kelime bölme, önek DP'si (Örnek 14)

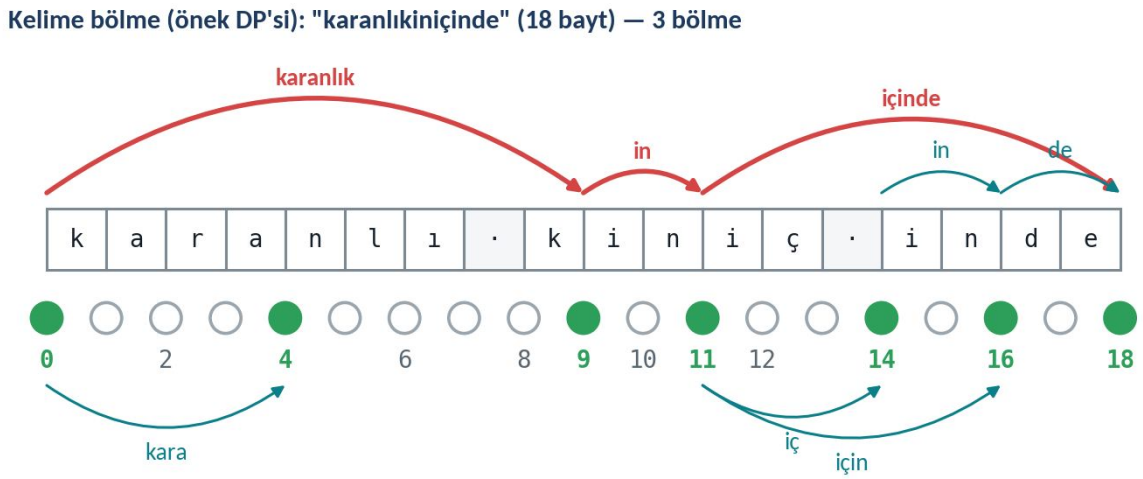
Boşlukları silinmiş "karanlıkiniçinde" bir sözlükle sözcüklere bölünebilir mi, kaç yolla? **Durum:** ok[i] = s'nin ilk i baytlık öneki bölünebilir mi (say[i]: kaç yolla). **Geçiş:** ok[i] = VAR j < i: ok[j] ve s[j..i) sözlükte; say[i] = Σ say[j]. **Temel:** ok[0] = doğru (boş önek). **Sıra:** i artan. **Yanıt:** ok[n], say[n]. **Geri çıkarma:** son[i] (en küçük j). Konumlar **bayttır**: "ı" ve "ç" ikişer bayt olduğundan 16 karakterlik dizgi 18 bayttır ve bazı konumlar bir karakterin ortasına düşer (Şekil 13.16). Sözlük Örnek 9'unkidir (3 215 sözcük); j yalnız en uzun sözcük (20 bayt) kadar geriye bakar:

c `c/H13_14_kelime_bolme.c · satır 68–82`

```c
    for (int i = 1; i <= n; i++) {
        ok[i] = 0;
        say[i] = 0;
        son[i] = -1;
        int j0 = i - en_uzun > 0 ? i - en_uzun : 0;
        for (int j = j0; j < i; j++) {
            if (ok[j] && sozlukte_mi(s + j, i - j)) {
                if (!ok[i]) {
                    son[i] = j;                /* en küçük j */
                }
                ok[i] = 1;
                say[i] += say[j];
            }
        }
    }
```

Program dizginin bölünebildiğini, 3 bölme olduğunu (say[18] = 3; 54 sözlük sorgusu) yazar; seçilen bölme karanlık + in + içinde, öbür bölmeler karanlık + in + için + de ve karanlık + in + iç + in + de. Sözlükte olmayan bir parça bölmeyi engeller: “kılıçlarınıçekti” bölünemez (53 sorgu).

Şekil 13.16
Örnek 14: “karanlıkiniçinde” (18 bayt); yeşil düğüm ok[i], yaylar sözlükteki parçalar s[j..i), kırmızı: son[] zinciri



yeşil düğüm: ok[i] (önek s[0..i) bölünebilir); kırmızı: son[] zinciri — karanlık + in + içinde; · : UTF-8 devam baytı

5.4 Meydan okuma: matris zinciri çarpımı, aralık DP'si (Örnek 15)

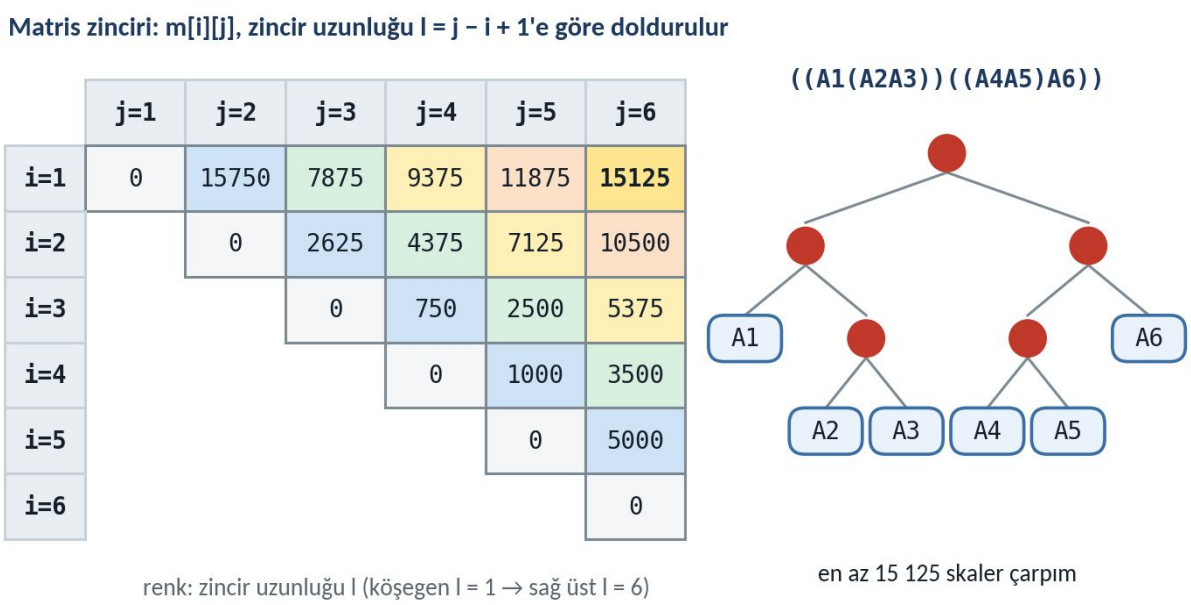
$A_1 \dots A_6$ (A_i boyutu p[i − 1] × p[i], p = {30, 35, 15, 5, 10, 20, 25}; Cormen vd., 2022) çarpılacak; çarpım birleşmeli olduğundan parantezleme sonucu değiştirmez ama skaler çarpım sayısını değiştirir: soldan sağa 40 500, en iyisi 15 125 (2,68 kat). **Durum:** m[i][j] = $A_i \dots A_j$'nin en az maliyeti. **Geçiş:** son çarpım ($A_i \dots A_k$)($A_{k+1} \dots A_j$): m[i][j] = min{m[i][k] + m[k + 1][j] + p[i − 1]·p[k]·p[j] : i ≤ k < j}. **Temel:** m[i][i] = 0. **Sıra:** zincir uzunluğu l = 2 … n — m[i][j], **aşağıdaki** satırlardaki m[k + 1][j]'ye bağlıdır; satır satır (i artan) doldurmak Hata Avcısı 2'dir. **Geri çıkarma:** s[i][j] = en iyi k.

c/H13_15_matris_zinciri.c · satır 37–51

```c
    for (int l = 2; l <= n; l++) {            /* zincir uzunluğu */
        for (int i = 1; i + l - 1 <= n; i++) {
            int j = i + l - 1;
            m[i][j] = LLONG_MAX;
            for (int k = i; k < j; k++) {
                denenen_k++;
                long long q = m[i][k] + m[k + 1][j] +
                              (long long)p[i - 1] * p[k] * p[j];
                if (q < m[i][j]) {            /* eşitlikte küçük k */
                    m[i][j] = q;
                    s[i][j] = k;
                }
            }
        }
    }
```

DP 21 hücrede 35 k dener ($\Theta(n^3)$); kaba kuvvet 243 çağrı yapıp aynı 15 125'i bulur, olası parantezleme sayısı 42'dir (Catalan sayısı; n ile üstel). Parantezleme s tablosundan özyinelemeyle yazılır: ((A1(A2A3)) ((A4A5)A6)) (Şekil 13.17).

Şekil 13.17
Örnek 15: m[i][j] tablosu (renk: zincir uzunluğu l, köşegen l = 1 → sağ üst l = 6) ve s tablosundan geri çıkarılan parantez ağacı



6. Haftanın Deneyi: Kaba Kuvvet, Memoization ve Tablolama

Üç soru: (a) Fibonacci ve alt küme toplamında kaba kuvvet, budamalı geri izleme, memoization ve tablolama çağrı/düğüm/hücre sayısında ve sürede nasıl ayrışır? (b) Python'da `functools.lru_cache` elle yazılmış not tablosuna göre ne kazandırır, neye mal olur? (c) Aynı LCS tablosunu satır ya da sütun öncelikli doldurmak süreyi değiştirir mi? Sayaçlar üç dilde aynıdır; süreler bu makineye özgüdür.

6.1 Ölçüm düzeni

- **Makine:** Intel(R) Xeon(R) Processor @ 2,80 GHz, 2 çekirdek (sanal makine); gcc 13.3.0; Python 3.11.15; C/C++ -std=c17 / -std=c++17 -Wall -Wextra -pedantic -O2 (doldurma deneyinde C ayrıca -O0). 1 ısınma + 5 tekrar, ortanca; duvar saati ve işlemci zamanı birlikte kaydedildi. Ölçüm sırasında makinede başka işler de çalışıyordu (yük ortalaması önce 1,40 1,45 1,15 · sonra 1,98 1,63 1,25).
- **Bütçe ve kestirim:** üstel yöntemlerde öngörülen tek çalıştırma süresi C/C++ 500 ms, Python 700 ms'yi aşarsa ölçülmez, kestirilir. Ölçülmeyen noktalar tablolarda **(k)** ile, grafiklerde içi boş işaret ve kesikli çizgiyle gösterilir; kaba kuvvet süresi düğüm formülüyle ($2^{n+1} - 1$), geri izleme süresi son iki ölçümün oranıyla ölçeklendi.
- **Kendi makinenizde:** `cd kodlar/deney && python3 h13_olcum_calistir.py && python3 h13_cozumle.py` (bu makinede 103,0 s). Süreleriniz farklı olacaktır; çağrı, düğüm ve hücre sayıları aynı kalmalıdır.

6.2 (a) Fibonacci ve alt küme toplamı: üstelden sözde polinoma

Tablo 13.8

Fibonacci: çağrı ve hücre sayıları (üç dilde aynı) ve ortanca süreler (bu makinede); (k) kestirim

n	saf çağrı	memo çağrı	tablo hücre	C saf (ms)	C tablo (ms)	C saf/tablo	Python saf (ms)	Python saf/tablo
20	21 891	39	21	0,058	0,000335	173	3,290	645
25	242 785	49	26	0,660	0,000320	2 062	34,113	5 568
30	2 692 537	59	31	7,552	0,000344	21 952	656,203	96 714
35	29 860 703	69	36	131,219	0,000495	265 089	4 927,948 (k)	710 386 (k)
40	331 160 281	79	41	1 171,984 (k)	0,000352	3 329 500 (k)	54 651,778 (k)	5 795 523 (k)

Saf özyinelemenin çağrı sayısı $2 \cdot F(n + 1) - 1$'dir; $n \to n + 5$'te oran kuramda $\varphi^5 \approx 11{,}09$. Ölçülen süre oranları C'de 11,38, 11,45, 17,38, Python'da 11,72, 10,37, 19,24: eğilim açık, tek tek oranlar gürültülü (son aralıklar makine yükünden etkilendi; Bölüm 6.5). Memo $2n - 1$ çağrı, tablo $n + 1$ hücre yapar; $n = 90$'da bile süre ölçüm çözünürlüğüne yakındır (grafik: sunum ve web sayfası).

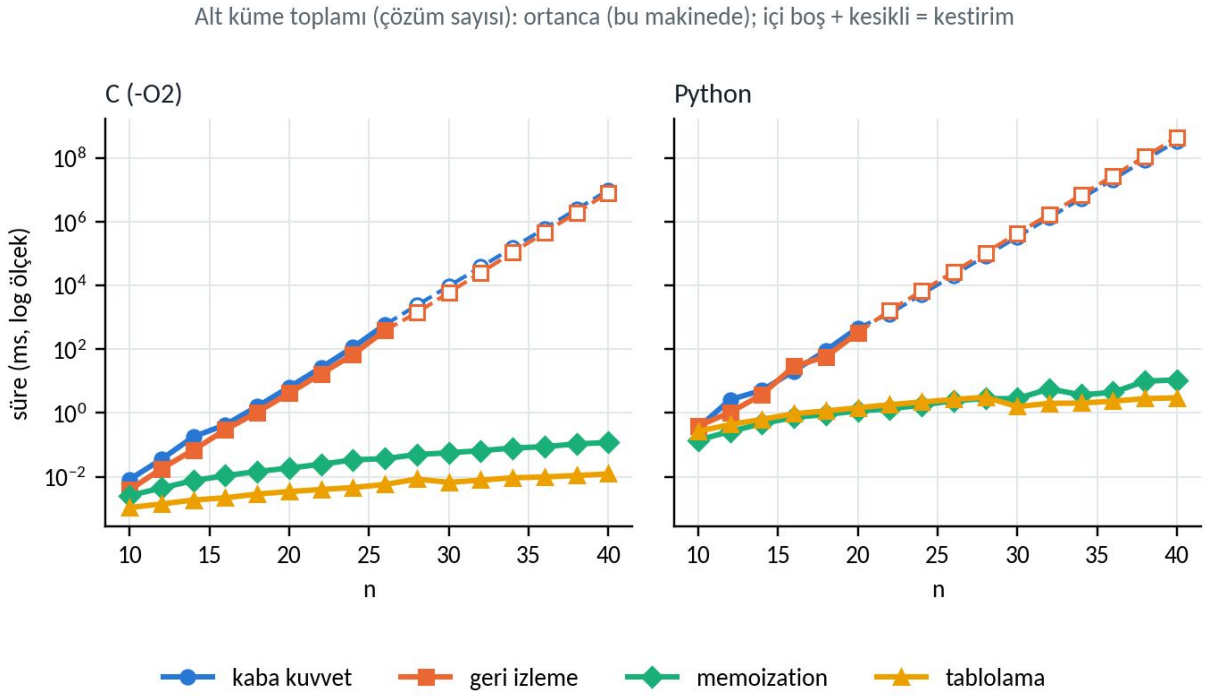
Tablo 13.9

Alt küme toplamı (hedef = toplam/2, tohum 42): iş sayıları (üç dilde aynı) ve C -O2 ortanca süreleri; (k) kestirim (kestirim satırındaki geri izleme düğümü de kestirimdir)

n	çözüm	kaba düğüm	geri düğüm	memo hesap.	tablo hücre	C kaba (ms)	C geri (ms)	C memo (ms)	C tablo (ms)
10	13	2 047	1 269	366	1 056	0,008	0,004	0,0025	0,0011
14	160	32 767	23 693	1 060	2 145	0,179	0,069	0,0076	0,0018
18	2 090	524 287	341 657	2 035	3 572	1,6	1,1	0,0145	0,0028
22	30 660	8 388 607	5 541 603	3 188	5 175	26,4	17,0	0,0244	0,0039
26	456 405	134 217 727	89 005 023	4 556	7 020	584,7	386,6	0,0367	0,0057
30	7 107 232	2 147 483 647	1 600 135 990 (k)	6 005	8 835	9 504,4 (k)	5 936,8 (k)	0,0562	0,0066
40	5 932 172 280	2 199 023 255 551	2 192 869 740 655 (k)	11 833	16 933	9 732 457,6 (k)	8 136 002,0 (k)	0,1180	0,0122

Kaba kuvvet $2^{n+1} - 1$ düğüm açar; budamalı geri izleme $n = 18$'de bu düğümlerin %65,2'si kadarını açar ve $n \to n + 2$'de düğüm sayısı yine 3,57–4,78 kat büyür: budama sabiti küçültür, üstel büyüme kalır. DP'nin hücresi $(n + 1)(T + 1)$'dir ve $T \propto n$ olduğundan n^2'yle büyür (log–log eğim 1,938): sözde polinom. C'de $n = 26$'da kaba/tablo 102 680 kat (ölçülen); $n = 40$'ta kaba kuvvet ≈ 2,7 saat sürerdi (kestirim), tablo 0,0122 ms sürdü. Memo yalnız erişilen durumları hesaplar ($n = 40$'ta 11 833/16 933 hücre): C'de yine de tablodan yavaştır (çağrı maliyeti), Python'da ise $n = 10 \ldots 28$ aralığında **tablodan hızlıdır** — yorumlayıcıda hücre başına maliyet baskındır. “Tablo her zaman hızlıdır” bir kural değildir.

Şekil 13.18
Alt küme toplamı süreleri (log y; ortanca, bu makinede): kaba kuvvet ve geri izleme üstel, memo ve tablo sözde polinom; içi boş + kesikli: kestirim



6.3 (b) Python: lru_cache ile elle yazılmış not tablosu

Fibonacci n = 450'de elle liste 0,106 ms, `lru_cache` 0,154 ms sürdü (tepe bellek 26 792 ↔ 50 216 bayt; ortanca, bu makinede). Varsayılan özyineleme sınırı 1000 iken soğuk önbellekle fib(n)'in çalıştığı en büyük n: elle liste ve sözlükte 996, `lru_cache` ve `cache` ile 498 (bu sürümdeki gözlem; Bölüm 3.9). Alt küme n = 40'ta `lru_cache` elle listeye göre 3,07 kat süre ve 6,32 kat bellek kullanır (2,83 ↔ 8,70 ms; 11 833 ↔ 12 240 not girdisi): karma tabanlı not (sözlük ya da dekoratör) her durum için bir demet anahtar ve sözlük girdisi taşır. Dekoratörlü sürümde not girdisi daha çoktur: "t > hedef" denetimi fonksiyonun içinde olduğu için bu çağrılar da önbelleğe girer. Dekoratör yazımı kolaylaştırır; derin zincirde ve büyük durum uzayında tablo ya da elle liste tercih edilir.

6.4 (c) Doldurma sırası: satır ya da sütun öncelikli

Aynı LCS tablosu (iki rastgele DNA dizisi) satır öncelikli (i dışta) ve sütun öncelikli (j dışta) doldurulur; değer ve hücre sayısı aynıdır. C -O2'de tablo ≈ 1 MB iken iki sıra eşittir (n = 500: 1,07 kat); tablo önbelleği aşınca sütun öncelikli sıra yavaşlar, en çok n = 3 000'de 6,15 kat, sonra oran düşer (n = 6 000: 3,73). -O0'da oran en çok 1,92: optimize edilmemiş kodda hücre başına iş bellek gecikmesini kısmen örter. Python'da oran 0,85–1,08: hücre başına ≈ 100–200 ns yorumlayıcı maliyeti önbellek etkisini gizler (grafik: sunum ve web sayfası). Kat eğrisinin biçimi makineye bağlıdır; genellenebilir olan, iç döngünün bitişik belleği taramasıdır.

6.5 Sınırlılıklar

Ölçüm sırasında makinede başka ajanlar da çalışıyordu; komşularından 1,5 kattan çok sapan 25 nokta yeniden ölçüldü (CSV'nin `not` sütunu) ve yeniden ölçüm bazen daha büyük değer verdi: gürültü tek yönlü değildir. Kestirimler ölçüm değildir. Bellek ölçütleri dillere göre farklıdır (C: `malloc` sarmalayıcı sayacı, C++: vektörlere istenen bayt, Python: `tracemalloc` tepe değeri); diller arası mutlak karşılaştırma yapılmamalıdır. Mikro saniye altındaki süreler ölçüm çözünürlüğüne yakındır.

7. Hata Avcısı

Hatalı ve düzeltilmiş programlar `kodlar/hata_avcisi/` altındadır; hepsi uyarısız derlenir ve **sessizce** yanlış sonuç verir. Belirtiler bu makinede gözlenen çıktılardır; Hata Avcısı 5 tanımsız davranıştır.

Tablo 13.10
Hata Avcısı: hata, belirti, neden ve çözüm

#	Hata	Belirti (bu makinede)	Neden	Çözüm
1	Temel durumun yanlış başlatılması (C)	`(a) hepsi 0 : 0 0 0 0 0 0 0 -> tutar 6 için en az para: 0; (b) enaz[0] = ∞ : ∞ ∞ ∞ ∞ ∞ ∞ ∞ …`	(a) 0 "hiç para gerekmez" der, min hiç güncellenmez; (b) hiçbir tutar 0'dan türetilemez	enaz[0] = 0, ötekiler ∞; toplamadan önce ∞ denetimi
2	Doldurma sırası bağımlılıklara aykırı (Python, matris zinciri)	`satır satır doldurulan tablo: m[1][6] = 9000` (doğrusu 15125)	m[i][j], aşağıdaki m[k + 1][j]'ye bağlı; i artan doldurulunca o hücre henüz 0	zincir uzunluğu l = 2 … n ya da i azalan: alt problem DAG'ının topolojik sırası
3	Sayımda döngü sırası (C++)	`paralar {1, 2, 5}, tutar 5: farklı para kümesi sayısı = 9` (doğrusu 4)	dış döngü tutar: her para her tutarın son adımı olur, 1 + 2 ile 2 + 1 ayrı sayılır	kombinasyon için dış döngü para (Örnek 4)
4	Tek satırda artan dolaşma (C)	`tek satır, artan dolaşma: en büyük değer = 150; dp[9] = 150: D üç kez alınmış (3 + 3 + 3 = 9, 50 * 3)`	dp[w − a_i] bu turda güncellenmiş: nesne yeniden alınır (sınırsız sırt çantası)	w = W … a_i ters dolaş (Örnek 11)
5	∞ = INT_MAX'a 1 eklemek (C)	-O0: `enaz[ 6] = -2147483648`; -O2: `enaz[ 5] = -2147483648`	işaretli taşma tanımsız davranış: INT_MIN'e sarıp "en küçük" seçilebilir; -O0 ve -O2 farklı	enaz[t − c] != SONSUZ denetimi (Örnek 3)
6	Memo anahtarı durumun bir parçası (Python)	`memo anahtarı yalnız i: en büyük değer = 130` (doğrusu 90)	durum (i, w); anahtar yalnız i ise ilk kapasitenin sonucu bütün kapasitelere döner	anahtar = durumun tamamı: memo[(i, w)]

Hata Avcısı 5'e yakından bakış. Ulaşılamayan tutarın ∞'u `INT_MAX`'tır; denetimsiz `enaz[t - c] + 1` taşar:

C `hata_avcisi/H13_HA5_sonsuz_tasmasi.c · satır 25–30`

```c
        for (int i = 0; i < 2; i++) {
            int c = paralar[i];
            if (c <= t && enaz[t - c] + 1 < enaz[t]) {  /* HATA: ∞ + 1 */
                enaz[t] = enaz[t - c] + 1;
            }
        }
```

-O0, -O2 VE UBSAN (SADELEŞTİRİLMİŞ: YOL ÇIKARILDI)

```
-O0 : enaz[ 5] = -2147483648 · enaz[ 6] = -2147483648 · enaz[ 8] = -2147483648
-O2 : enaz[ 5] = -2147483648 · enaz[ 6] = 1 · enaz[ 8] = 2
UBSan (-O1 -fsanitize=undefined):
H13_HA5_sonsuz_tasmasi.c:27:39: runtime error: signed integer overflow: 2147483647 +
1 cannot be represented in type 'int'
```

-O0'da INT_MIN "en küçük" seçilir ve hata 6, 8, 10'a yayılır; -O2'de derleyici farklı kod üretir ve çift tutarlar doğru kalır: aynı kaynak, iki ayrı yanlışlık. Düzeltilmiş sürüm `enaz[ 5] = ∞`, `enaz[ 6] = 1` verir. **Hata Avcısı 2 ve 6** Python'dadır ve hata iletisi yoktur. Hata Avcısı 2'de dış döngü `for i in`

`range(1, n + 1)`'dir (17. satır): m[k + 1][j] okunduğunda o satır henüz doldurulmamıştır. Hata Avcısı 6'da not `if i in memo: return memo[i]` ile okunur (20. satır).

İkinci programın notunda yalnız 4 girdi vardır (`{0: 130, 1: 120, 2: 80, 3: 50}`), oysa durum uzayı (i, w) çiftleridir; düzeltilmiş sürüm 13 farklı durum saklar ve 90 bulur. Kural: sonucu etkileyen her parametre anahtarda olmalıdır.

8. Dil Penceresi: C · C++ · Python

Tablo 13.11
Dil karşılıkları: not tutma, 2B tablo, ∞, büyük sayılar ve özyineleme derinliği

Öğe	C	C++	Python
Not tablosu (memoization)	dizi + "hesaplanmadı" işareti (−1); seyrek durumda elle karma tablosu (Kavram 6)	`std::vector` (−1) ya da `std::unordered_map`	liste (−1), `dict` ya da `functools.cache`/`lru_cache` dekoratörü
"Hesaplanmadı" işareti	geçerli yanıtlarla karışmayan değer (F(i) ≥ 0 → −1; 0 yanlış: Alıştırma 13.9)	aynı; ya da `std::optional<T>`	`None` ya da anahtarın `dict`'te olmaması (`in`)
2B tablo	`malloc(n * m * sizeof *t)`: tek blok, `t[i * m + j]`	`std::vector<std::vector<int>>` (satırlar ayrı) ya da tek blok `std::vector<int>(n * m)`	`[[0] * m for _ in range(n)]`; **tuzak** `[[0] * m] * n` (aynı satır n kez)
∞ (ulaşılamaz)	`INT_MAX` + toplamadan önce denetim (Hata Avcısı 5)	`std::numeric_limits<int>::max()` + denetim	`math.inf` (float) ya da büyük tamsayı; taşma yok
Büyük sayılar (yol sayısı, F(n))	`uint64_t` 2^{64}'te sessizce sarar; mod 10^9+7	aynı (`std::uint64_t`)	tam sonuç (keyfi duyarlıklı `int`); mod isteğe bağlı
Özyineleme derinliği	yığın taşması: uyarısız çökme	aynı	`RecursionError` (sınır 1000); `lru_cache` daha erken (Bölüm 3.9)
Tercih	tablolama; derin zincirde özyineleme yok	tablolama; memo için `unordered_map`	küçük durumda dekoratör; derin/büyük durumda tablo

Büyük sayılar (Kavram 3). İşaretsiz taşma C'de **tanımlıdır** ve uyarı vermez: `uint64_t` ile F(94) = 1293530146158671551 çıkar, doğrusu 1974027421986822316 7'dir (65 bit). 40 × 40 ızgaranın yol sayısı C'de 806736047744338 2000, Python'da 272170148691990320 15600; mod 10^9+7 sürümü üç dilde 929264847 verir. Bu yüzden Örnek 1 F(94)'ü toplamadan **önce** denetler, sayma örnekleri (4, 5) mod alır; Python sürümleri aynı çıktıyı vermek için aynı modu uygular.

Python'da satır paylaşımı (Örnek 6, Bölüm 5.2) dilin değer–başvuru modelinden gelir: `*` listeyi değil başvuruyu çoğaltır. C++'ta `std::vector<std::vector<int>>(n, satir)` her satırı **kopyalar**, paylaşım yoktur (Alıştırma 13.15). C'de satır işaretçileri dizisi (`int **`) de kurulabilir ama tek blok hem bitişik hem tek `free`'dir. **Not tutma araçları:** Kavram 6 aynı alt küme memo'sunu dizi ve karma tablosuyla yazar; seyrek durum uzayında karma yalnız ziyaret edilen 6 005 durumu saklar. Python'da `functools.cache` içte bir `dict` kullanır; anahtar, fonksiyonun **bütün** argümanlarıdır (Hata Avcısı 6'daki tuzak dekoratörde oluşmaz, elle yazılan notta oluşur).

9. Mühendis Gözüyle: Bellek–Zaman Ödünleşimi

- **Önce tablonun boyutunu hesaplayın.** Hücre sayısı × hücre boyu bir bütçedir: n = m = 20 000'de LCS'nin tam tablosu 1,60 GB, iki satırı 160 KB'tır (Kavram 1). Bütçeyi aşan tablo ya hiç ayrılamaz ya da sayfa takasıyla süreyi patlatır.

- **Yalnız değer mi, çözümün kendisi mi?** Değer yetiyorsa satır tamponu (iki satır, tek satır, iki değişken) hem bellek hem çoğu zaman süre kazandırır (önbellekte kalır: Bölüm 4.6.1). Çözüm gerekiyorsa tam tablo ya da seçim dizisi tutulur; ikisi birden gerekiyorsa Hirschberg (1975) böl ve fethetle doğrusal bellekte hizalamayı da çıkarır.
- **İki sürüm, bir kâhin.** Mini-OBS 2 v2.12 ders seçiminde tam tabloyu (küme için) ve tek satırı (değer için) birlikte tutar ve değerlerini karşılaştırır (v2.12 belgesi § 7, karar 1): bellek azaltmanın doğruluğu böyle sınanır.
- **Seyrek mi yoğun mu?** Erişilen durumlar azsa memo + karma tablosu belleği ve işi azaltır (n = 30 alt küme: ayrılan hücrelerin %68,0'ı ziyaret edildi); yoğunsa düz dizi daha hızlı ve küçüktür (Python'da `lru_cache` 6,32 kat bellek: Bölüm 6.3).
- **Sözde polinomun bedeli ölçülebilir:** W ya da S büyüdükçe tablo da büyür; AKTS (W ≤ 1 000) gibi küçük sınırlarda DP rahattır, kuruş cinsinden büyük tutarlarda değil. Karar verirken girdinin büyüklüğünü değil **değerini** sorun (Bölüm 4.6.2).

10. Mini-OBS 2 v2.12: Ders Seçimi (0/1 Sırt Çantası) ve “Bunu mu demek istediniz?”

Bu hafta: Mini-OBS'ye iki dinamik programlama yeteneği ekliyoruz. **Ders seçimi:** AKTS üst sınırı altında tercih puanını en büyük yapan ders kümesini **0/1 sırt çantası** tablosuyla buluyor, seçilen dersleri tablodan **geri çıkarıyor**, eşitlikte belirleyici bir kural uyguluyoruz; aynı problemi v2.2'nin geri izlemesi, dal ve sınır, açgözlü (yanlış) ve kaba kuvvetle çözüp **düğüm ↔ hücre** ve süreleri karşılaştırıyoruz. **“Bunu mu demek istediniz?”:** ad araması sonuç vermezse **düzenleme (Levenshtein) uzaklığı** en küçük adları öneriyoruz — Türkçe büyük/küçük harf duyarsız ve **kod noktası** düzeyinde; aday sayısını **uzunluk farkı** budamasıyla azaltıyoruz. (v2.12 belgesi § 10)

Ölçüm (v2.12 belgesi § 8.1, W = 30): n = 10'dan 50'ye geri izlemenin düğüm sayısı 896 → 147 198 469, DP'nin hücresi 341 → 1 581; açgözlü n = 40'ta 673 (−14) puan bulur (en iyi 687). Bu hafta yazacağınız modüller ders notunun Bölüm 4.5–4.7'sinin doğrudan uygulamasıdır.

10.1 Gereksinimler

- `secim.h/.c`: aday listesi ve tercih dosyası (`kod;puan`; hatada satır numarası, çıkış 1); tam tablo DP + geri çıkarma (eşitlikte ALMA); tek satırlı DP (c azalan); v2.2 ağacında geri izleme (en iyiyi tutan); dal ve sınır (kesirli sırt çantası sınırı); açgözlü (oran sırası); kaba kuvvet (n ≤ 25).
- `ders_secimi`: havuz kipi (v2.2 havuzu + tercihler) ve katalog kipi (v2.10'un alınabilir dersleri; önkoşul denetimi); `--tablo` ile tablo ve geri çıkarma izi; yöntemlerin kümelerinin karşılaştırması; v2.2 düğüm sayısıyla kıyas.
- `oneri.h/.c`: kod noktası düzeyinde Levenshtein (iki satır); Türkçe katlama (v2.9) + `utf8_coz`; ad, soyad ve tam ad terimlerinin sözlüğü (Türkçe sıra); en yakın k (eşitlikte alfabe sırası); uzunluk farkı budaması.
- Menü: sonuç vermeyen ad aramasından sonra en çok üç öneri (eşikli); `ara --oneri`.
- Testler: DP ↔ bütün maskeler (değer, küme, tablo); v2.2 düğümü ↔ geri izleme; Levenshtein ↔ naif özyineleme; öneri ↔ bütün terimlerle tam hesap.
- Ölçüm: n = 10…40, en az iki W için DP, geri izleme (bütçeyi aşanlar kestirim), kaba kuvvet (n ≤ 25); 10^5 kayıtta öneri sorgusu süresi ve budamanın etkisi.

10.2 İpuçları

- Tabloyu tek blokta tutun: `T[i * (W + 1) + c]` (`malloc((n + 1) * (W + 1) * sizeof *T)`); her satır yalnız bir öncekine bakar.
- Geri çıkarmada "T[i][c] == T[i−1][c] ise ALMA" eşitlik kuralını belirler; kuralı yazın ve kaba kuvvette "maske artan, kesin büyükse güncelle" ile aynı kümeyi verdiğini test edin.
- Tek satırlı DP'de c'yi **W'den aşağı** gezin; `--hata-goster sinirsiz`'in karşı örneğini (1 ders, AKTS 1, puan 1, W = 2) elle izleyin.
- Geri izlemenin düğüm sayısını tahmin etmek için çalıştırmayın: "en büyük indisi i, toplamı x olan küme sayısı" DP'si sayıyı kesin verir; büyük n'de süreyi düğüm başına süreyle kestirin ve **kestirim** diye işaretleyin.
- Levenshtein'i UTF-8 baytlarında değil, `utf8_coz` ile çözülmüş kod noktalarında hesaplayın; "ŞAHİN"/"SAHIN" ile sınayın (2, bayt düzeyinde 4).
- Büyük/küçük harf için `toupper` kullanmayın: Türkçe i/İ ve ı/I kuralını v2.9'un `tr_katla`'sı uygular.
- Aynı adı taşıyan kayıtlar için DP'yi tekrarlamayın: önce farklı terimleri ayıklayın (sözlük).
- Uzunluk budamasında "≥ u" mu "> u" mu olacağını eşitlik kuralından çıkarın: alfabe sırasıyla taranıyorsa eşit uzaklıktaki sonraki terim zaten giremez.

10.3 Başlangıç noktası

- **Verilir:** kendi v2.11'iniz (ya da referans v2.11), `secim.h` ve `oneri.h` (bildirimler, sözleşmeler, sahiplik), `ornek_veri/tercihler.csv` ve `tercihler_katalog.csv`, `olcer_dp`'nin ölçüm düzeneği (aday ve sorgu üretimi, bütçe ve kestirim), v2.2'nin `secmeli` modülü, v2.4'ün `utf8_coz`'u ve v2.9'un `tr_katla`'sı, Şekil 1 ve 2'nin örnekleri.
- **Yazılır:** `secim.c` (tablo, geri çıkarma, tek satır, geri izleme, dal-sınır, açgözlü, kaba kuvvet, düğüm sayımı), `oneri.c` (Levenshtein, sözlük, en yakın k, budama), `ders_secimi.c`, menü ve `ara` bütünleştirmesi; testler (birim + kâhin); ölçüm ve yorum.
- **Değiştirilmez:** v1.0 kuralları, menünün sonuç bulan aramaları, v2.2'nin `secim_yap`'ı, v2.10'un katalog modülü, ortak modüller.
- **Meydan okuma:** (a) Levenshtein'de satırın en küçük değeri en kötü önerinin uzaklığını aşınca DP'yi erken bitirin (Ukkonen'in kesme fikri) ve § 8.4'ü tekrarlayın; (b) yukarıdan aşağıya (memoization) sırt çantası: v2.9'un karma tablosunda (i, c) durumlarını tutun, ziyaret edilen durum sayısını (n+1)(W+1) ile karşılaştırın; (c) yer değiştirmeyi (YILMAZ ↔ YLIMAZ) bir düzenleme sayan Damerau–Levenshtein uzaklığı; (d) önkoşul kısıtlı ders seçimi: aynı dönemde bir ders ancak önkoşuluyla birlikte seçilebiliyorsa problem nasıl değişir? (ağaç sırt çantası).

10.4 Kabul ölçütleri ve test senaryoları

1. `make` ve `make test` sıfır uyarıyla derlenir; v1.0'ın 99 doğrulaması ve önceki sürümlerin testleri dahil bütün birim testleri (439) ve `test_kahin` (300 deneme) geçer.
2. `./ders_secimi` şunları basar: `en büyük tercih puanı 315 · AKTS 20 / 20 · 5 ders: BIL331 BIL341 BIL345 BIL355 BIL357`, `satır tamponlu DP … 315 — AYNI`, açgözlü için `puan 295 … EN İYİ DEĞİL`, geri izleme, dal-sınır ve kaba kuvvet için `küme: AYNI`, v2.2 için `3264 düğüm (geri izleme 3264, sayım DP'si 3264: AYNI)`.
3. `./ders_secimi --tablo` geri çıkarma izinde `BIL351 … (EŞİTLİK) → ALINMADI` satırı vardır.

4. `./ders_secimi --katalog ornek_veri/onkosul.txt --alinan BLM101,MAT101,FIZ101,BLM102 --akts 30` 12 aday, 415 puan ve `Önkoşul denetimi: … EVET` basar.
5. `./ara ornek_veri/ogrenciler.csv --oneri 'ilgaz sahin'` ilk öneri olarak `uzaklık 2 · bayt düzeyinde 4 ILGAZ ŞAHİN` ve `Üç yöntem: AYNI` basar; menüde "yilmas" araması `Bunu mu demek istediniz? YILMAZ (soyad · 2 kayıt · uzaklık 2)` satırını üretir.
6. `./test_kahin --hata-goster sinirsiz` ve `--hata-goster bayt` uyuşmazlık ve en küçük karşı örnek basar, 1 ile çıkar; `make asan` ve `make valgrind` temizdir.
7. Ölçüm raporunda: n = 10…40 ve en az iki W için DP hücresi ve süresi, geri izleme düğümü ve süresi (kestirimler işaretli), kaba kuvvet n ≤ 25; 10^5 kayıtta öneri sorgusu süresi; budamasız ↔ budamalı aday sayısı ve yorumu.

Tablo 13.12
Mini-OBS 2 v2.12 test senaryoları (v2.12 belgesi § 10)

#	Girdi	Beklenen çıktı / davranış
1	(AKTS, puan) = (1, 1), (3, 4), (4, 5), (5, 7); W = 7	9 puan: 2. ve 3. ders; açgözlü 8; kesirli 9
2	iki özdeş ders (2, 5); W = 2	5 puan, ilk ders (eşitlikte en küçük maske)
3	(3, 10), (2, 10); W = 3 / W = 5	ilk ders / iki ders, 20 puan
4	W = 0 ya da n = 0	boş küme, 0 puan
5	tek satırlı DP, 1 ders (1, 1), W = 2	1 (c artan gezilirse 2 — hata)
6	örnek havuz, W = 20	315; BIL351'de eşitlik → ALMA; açgözlü 295; v2.2 düğümü 3 264
7	katalog, `--alinan BLM101,MAT101,FIZ101,BLM102`, W = 30	12 aday, 415 puan, önkoşul denetimi EVET
8	d("KITTEN", "SITTING"), d("", "ABC")	3, 3
9	d("ŞAHİN", "SAHIN")	2 (kod noktası), 4 (bayt)
10	" ismail öztürk " katlama	"İSMAİL ÖZTÜRK", 13 kod noktası
11	örnek 8 kayıt, "yilmaz", k = 3	YILMAZ 1, ILGAZ 3, İSMAİL 4; uzunluk farkı 4, kova 5 terimi atlar
12	ad = soyad ("CAN CAN") ve "ALİ CAN"	terim CAN: ad + soyad, 2 kayıt
13	menüde "zzzzzz"	öneri yazılmaz (eşik dışında)

11. Alıştırmalar ve Atölyeler

Türler: A) izleme ve çıktı tahmini, B) karmaşıklık, C) değişmez, D) hata bulma, E) Parsons (Parsons & Haden, 2006), F) kod yazma, G) diller arası okuma. Kodlu alıştırmaların dosyaları `kodlar/alistirmalar/` altındadır: önce **çalıştırmadan** tahmin edin, sonra çalıştırıp karşılaştırın. Kod yazarken önce DP reçetesinin altı satırını yazın; sonucu bir kâhinle (kaba kuvvet) karşılaştırın, C/C++'ı AddressSanitizer ile çalıştırın. Çözümler öğretim üyesindedir.

11.1 İzleme ve çıktı tahmini (A)

Alıştırma 13.1 ★☆☆ *Çıktı tahmini*

Her adımda 1 ya da 3 basamak çıkılan bir merdiven. Programın iki çıktı satırını (i = 0 … 10 ve yol[i])

çalıştırmadan yazın. Durum, geçiş ve temel durum nedir; i < 3 için neden yalnız yol[i − 1] eklenir? yol[0] = 0 alınsaydı ne olurdu?

alistirmalar/H13_A01_merdiven_13.c · satır 14–21

```
14      long long yol[N + 1];
15      yol[0] = 1;
16      for (int i = 1; i <= N; i++) {
17          yol[i] = yol[i - 1];
18          if (i >= 3) {
19              yol[i] += yol[i - 3];
20          }
21      }
```

Alıştırma 13.2 ★★☆ *İzleme*

a = “ANKARA”, b = “KARMA” için LCS tablosunu (7 × 6) doldurun ve (6, 5)'ten geri çıkarın (eşitlikte önce yukarı, Örnek 8'in kuralı). Ortak alt dizi nedir? Aynı biçimde “kalem” → “kelime” düzenleme uzaklığı tablosunu doldurup bir en kısa işlem dizisi yazın (eşitlikte eşleşme > silme > ekleme > değiştirme).

Alıştırma 13.3 ★★☆ *İzleme*

Paralar {1, 5, 6, 9}, tutar 11. (a) enaz[0..11] ve secim[0..11] satırlarını Örnek 3'ün kuralıyla (paralar artan, yalnız kesin küçükte güncelle) doldurun; 11'in ödemesini geri çıkarın. (b) Açgözlü (en büyük sığan para) kaç para verir? (c) Kombinasyon sayımıyla (dış döngü para) yol[11] kaçtır?

11.2 Karmaşıklık (B)

Alıştırma 13.4 ★★☆ *Karmaşıklık*

(a) $n = m = 10^4$ karakterlik iki dizgide LCS'nin tam tablosu (4 baytlık `int`) ve iki satırlık sürümü kaç bayt tutar? (b) $n = 100$ nesne ve $W = 10^6$ kapasitede 0/1 sırt çantası tablosu kaç hücredir; $W = 10^9$ olursa? W'yi yazmak için kaç bit gerekir, hücre sayısı bit sayısına göre nasıl büyür? (c) fib_memo(n)'in $n \geq 2$ için tam $2n - 1$ çağrı yaptığını tümevarımla gösterin.

Alıştırma 13.5 ★★☆ *Karmaşıklık*

İki “gizemli” fonksiyonun iç işlem sayılarını (sayac_a, sayac_b) n = 10, 20, 40 için kestirin; Θ sınıflarını yazın ve programı çalıştırıp doğrulayın. gizemli_b'de not tablosu olmasaydı çağrı sayısı nasıl büyürdü? gizemli_a hangi tanıdık DP'nin döngü iskeletidir?

alistirmalar/H13_A05_gizemli.c · satır 15–42

```
15  static long long gizemli_a(int n)
16  {
17      long long toplam = 0;
18      for (int l = 1; l <= n; l++) {
19          for (int i = 0; i + l <= n; i++) {
20              for (int k = i; k < i + l; k++) {
21                  sayac_a++;
22                  toplam += k - i;
23              }
24          }
25      }
```

```
    return toplam;
}

static long long gizemli_b(int i, int j)
{
    sayac_b++;
    if (i >= j) {
        return 0;
    }
    if (memo_b[i][j] != -1) {
        return memo_b[i][j];
    }
    long long x = gizemli_b(i + 1, j);
    long long y = gizemli_b(i, j - 1);
    memo_b[i][j] = (x > y ? x : y) + 1;
    return memo_b[i][j];
}
```

11.3 Değişmez yazma (C)

Alıştırma 13.6 ★★☆ *Değişmez yaz*

Örnek 9'un `uzaklik` fonksiyonu için dış döngünün (i) her adımı sonunda D[i][0..m] hakkında doğru olan değişmezi yazın. Başlatmayı (D[i][0] = i, D[0][j] = j neden doğru?), korunmayı (en iyi işlem dizisinin son işlemi üç durumdan biridir) ve sonlanmadan D[n][m]'nin doğruluğunu çıkarın.

Alıştırma 13.7 ★★★ *Değişmez yaz*

Tek satırlı 0/1 sırt çantasında i. nesne işlenirken iç döngü w = W … a_i **azalarak** gidiyor. w'nin her adımının başında dp[w'] için (w' > w ve w' ≤ w ayrı ayrı) doğru olan değişmezi K[i][·] ve K[i − 1][·] cinsinden yazın. Değişmezden ters dolaşmanın doğruluğunu çıkarın; artan dolaşmada hangi yarısı bozulur (Örnek 11, Hata Avcısı 4)?

11.4 Hata bulma (D)

Alıştırma 13.8 ★★☆ *Hata bul*

LCS uzunluğu bazı girdilerde yanlış çıkıyor; programdaki dört çiftten ikisi tesadüfen doğru. Hatalı satırı bulun, her yanlış değeri açıklayın (hangi karakterler karşılaştırılıyor, hangi bellek okunuyor?) ve düzeltin.

alistirmalar/H13_A08_hata_bul_lcs.c · satır 25–36

```
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (a[i] == b[j]) {
                L[i][j] = L[i - 1][j - 1] + 1;
            } else if (L[i - 1][j] >= L[i][j - 1]) {
                L[i][j] = L[i - 1][j];
            } else {
                L[i][j] = L[i][j - 1];
            }
        }
```

```
35      }
36      return L[n][m];
```

Alıştırma 13.9 ★★☆ *Hata bul*

Engelli 12 × 12 ızgarada sağ alt köşeye giden yol sayısı memoization ile hesaplanıyor (yalnız sağa ya da aşağı). Sonuçlar doğru, ama ikinci ızgarada (hedefin iki komşusu engel) çağrı sayısı beklenenin çok üstünde. Hatalı satırı bulun, nedenini açıklayın, düzeltin; düzeltilmiş sürümün iki ızgarada kaç çağrı yapacağını önceden kestirin (Bölüm 8, “hesaplanmadı” işareti).

alistirmalar/H13_A09_hata_bul_memo.c · satır 22–36

```
22  static long long yol(int i, int j)
23  {
24      cagri++;
25      if (i >= R || j >= C || engel[i][j]) {
26          return 0;
27      }
28      if (i == R - 1 && j == C - 1) {
29          return 1;
30      }
31      if (memo[i][j] != 0) {
32          return memo[i][j];
33      }
34      memo[i][j] = yol(i + 1, j) + yol(i, j + 1);
35      return memo[i][j];
36  }
```

11.5 Parsons (E)

Alıştırma 13.10 ★★☆ *Parsons*

O(n²) LIS fonksiyonunun karıştırılmış satırları; ikisi fazladır (bu hâliyle derlenmez). Sırayı ve girintiyi kurun (a = {5, 2, 8, 6, 3, 6, 9, 7} için uzunluk 4); fazla satırlar hangi girdide yanlış sonuç verir?

alistirmalar/H13_A10_parsons_lis.c · satır 8–27

```
            if (a[j] <= a[i] && uz[j] + 1 > uz[i]) {
    }
        if (uz[i] > uz[son]) {
static int lis_n2(const int a[], int n, int uz[], int ebeveyn[])
                ebeveyn[i] = j;
        uz[i] = 1;
    return son;
            son = i;
        for (int j = 0; j < i; j++) {
                uz[i] = uz[j];
{
    int son = 0;
            if (a[j] < a[i] && uz[j] + 1 > uz[i]) {
        ebeveyn[i] = -1;
                uz[i] = uz[j] + 1;
    for (int i = 0; i < n; i++) {
}
        }
            }
        }
```

11.6 Kod yazma (F)

Alıştırma 13.11 ★☆☆ *Kod yaz*

Ucuz merdiven. Her basamağın bir maliyeti var (m[0..n − 1]); 0. ya da 1. basamaktan başlayıp her adımda 1 ya da 2 basamak çıkarak merdivenin üstüne (n) çıkın; bastığınız basamakların maliyet toplamını en küçük yapın ve basılan basamakları geri çıkarın. Sınayın: {10, 15, 20}; {1, 100, 1, 1, 1, 100, 1, 1, 100, 1}; {5, 5, 5, 5}. Kâhin: bütün adım dizileri (n ≤ 20). C, C++ ve Python.

Alıştırma 13.12 ★★☆ *Kod yaz*

Üçgende en büyük yol. n satırlık sayı üçgeninde tepeden tabana, her adımda altındaki iki komşudan birine inerek en büyük toplamı ve yolu bulun. Hangi yön (yukarıdan mı, aşağıdan mı) daha az özel durum ister? Sınayın: 3 / 7 4 / 2 4 6 / 8 5 9 3 ve ortak üreteçle (tohum 13, değerler 0 … 99) n = 15; kâhin: 2^{n-1} yolun hepsi.

Alıştırma 13.13 ★★☆ *Kod yaz*

Çubuk kesme (Cormen vd., 2022). k uzunluklu parçanın fiyatı p[k] = 1, 5, 8, 9, 10, 17, 17, 20, 24, 30 (k = 1 … 10). n uzunluğundaki çubuğu kesip satarak en büyük geliri r[n] ve parçaları bulun (n = 4, 7, 9, 10). Bu bir **sınırsız** sırt çantasıdır: tek satırlı sürümde kapasite hangi yönde dolaşılmalıdır, neden?

Alıştırma 13.14 ★★★ *Kod yaz*

En uzun palindrom alt dizi. s'nin tersten de aynı okunan en uzun alt dizisi. (a) LCS(s, ters(s)) ile; (b) aralık DP'siyle (P[i][j] = s[i..j]'nin yanıtı; Örnek 15'in doldurma sırası). Palindromun kendisini geri çıkarın; iki yöntemin uzunlukları her girdide aynı mı? Sınayın: “BBABCBCAB”, “CHARACTER”, “ABCDE”, “KAYAK”, boş dizgi.

11.7 Diller arası okuma (G)

Alıştırma 13.15 ★★☆ *Diller arası okuma*

Python programı üç 2B tabloyu farklı kuruyor; C++ eşi (`alistirmalar/H13_A15_satir_paylasimi.cpp`, 28–36. satırlar) benzer işlemleri `std::vector` ile yapıyor. İki programın çıktısını çalıştırmadan yazın. Hangi satırlar paylaşılıyor, hangileri kopya? Bunun DP tablosu için anlamı nedir (Örnek 6)?

alistirmalar/H13_A15_satir_paylasimi.py · satır 10–19

```
10      a = [[0] * 3] * 2
11      b = [[0] * 3 for _ in range(2)]
12      c = [a[0][:], a[1][:]]
13      a[0][1] = 5
14      b[0][1] = 5
15      c[1][2] = 7
16      print("a =", a)
17      print("b =", b)
18      print("c =", c)
19      print(a[0] is a[1], b[0] is b[1], c[0] is a[0])
```

Alıştırma 13.16 ★★☆ *Diller arası okuma*

C programı Pascal üçgenini tek satırla, 32 bitlik işaretsiz tamsayıyla hesaplıyor; Python eşi (`alistirmalar/H13_A16_buyuk_sayi.py`) aynı döngüyü Python tamsayısıyla çalıştırıyor. C(n, n/2) ilk hangi n'de 2^{32} − 1'i aşar? İki programın hangi satırları aynı, hangileri farklı çıkar; C'deki değerler nasıl elde edilir? İç döngü neden k = n … 1 (ters) dolaşıyor?

alistirmalar/H13_A16_buyuk_sayi.c · satır 18–26

```c
    uint32_t satir[MAKS_N + 1] = {1};
    for (int n = 1; n <= 36; n++) {
        for (int k = n; k >= 1; k--) {        /* tek satır: k TERS */
            satir[k] = satir[k] + satir[k - 1];
        }
        if (n >= 32) {
            printf("C(%d, %d) = %u\n", n, n / 2, (unsigned)satir[n / 2]);
        }
    }
```

11.8 Atölye 1 · DP reçetesi kartları (eşli, ≈ 25 dk)

ALGORİTMA ATÖLYESİ · **Altı problem, altı reçete — kod yazmadan**

Her eş altı kart alır; her kartta bir problem vardır. Kod yazmadan, kartın arkasına reçetenin altı satırını yazın: **durum** (hücre neyi anlatır?), **geçiş**, **temel durum**, **hesaplama sırası**, **yanıtın yeri**, **geri çıkarma**. Sonra kartları değişip birbirinizinkini küçük bir girdiyle elle sınayın.

- **Kart 1:** her adımda 1, 2 ya da 3 basamak; n. basamağa kaç yolla çıkılır?
- **Kart 2:** {1, 2, 5} paralarıyla tutar 5 kaç farklı **para kümesiyle** ödenir (sıra önemsiz)?
- **Kart 3:** sıralı evlerde para var; yan yana iki ev soyulamıyorsa en çok ne toplanır?
- **Kart 4:** bir dizinin en uzun **azalan** alt dizisi.
- **Kart 5:** iki dizgiyi alt dizi olarak içeren **en kısa ortak üst dizi**.
- **Kart 6:** boşluksuz bir dizgi sözlükle sözcüklere kaç yolla bölünür?

Tartışma: hangi kartlarda tablo 1B, hangilerinde 2B? Hangisi bilinen bir problemin kılık değiştirmiş hâli? Kart 1, 2 ve 6'nın yanıtları Örnek 2, 4 ve 14'ün dosya başlığındaki **DP** satırıyla karşılaştırılabilir.

11.9 Atölye 2 · Yazım düzeltici (gruplar, ≈ 30 dk)

ALGORİTMA ATÖLYESİ · **Türkçe kelime listesiyle "Bunu mu demek istediniz?"**

Başlangıç programı `alistirmalar/H13_A17_yazim_duzeltici.c` sözlüğü standart girdiden, sorguları komut satırından okur ve her sorgu için düzenleme uzaklığı en küçük k sözcüğü yazar (kod noktası düzeyinde, iki satırlık tamponla; `--bayt` ile bayt düzeyinde). Çalıştırma: `./H13_A17_yazim_duzeltici gunduz kilic padisah sehirde zzzzzz < ../girdiler/H13_09_duzenleme_uzakligi.txt`.

- **Gözlem:** "gunduz" ve "sehirde" önerilerini kod noktası ve bayt düzeyinde karşılaştırın; hangisi Türkçe yazım hatasına uygun?
- **Budama:** $|\text{len}(a) - \text{len}(b)| \le d(a, b)$: liste dolunca uzunluk farkı en kötü önerinin uzaklığından küçük olmayan sözcüğün DP'sini atlayın; önerilerin değişmediğini (kâhin: budamasız sürüm), atlanan sözcük ve hücre sayısını gösterin.
- **Eşik:** öneriyi yalnız uzaklık $m/3 + 1$'i (m: sorgunun kod noktası sayısı) aşmıyorsa yazın; "zzzzzz" ne yazar? Mini-OBS 2 v2.12'nin menüsü aynı eşiği kullanır (v2.12 belgesi § 4.6).
- **Tartışma:** eşitlikte bayt sırası mı, sözcüğün sıklığı mı? Gerçek bir denetleyici başka neyi kullanırdı?

11.10 Atölye 3 · Strateji turnuvası (dört grup, ≈ 30 dk)

ALGORİTMA ATÖLYESİ · **Alt küme toplamında dört yaklaşım: kaba kuvvet, geri izleme, açgözlü, DP**

Her grup bir stratejiyi savunur ve Örnek 12'nin girdi üretecini (tohum 42, hedef = toplam/2) kullanır. Ölçüm $n = 10, 12, \ldots, 40$ için yapılır; kaba kuvvet $n \le 25$'te durdurulur, ötesi düğüm formülüyle ($2^{n+1} - 1$) ve son ölçümün düğüm başına süresiyle **kestirilir** ve tabloda işaretlenir.

- **Tahmin turu:** ölçmeden önce her grup $n = 20, 30, 40$ için kendi süresini ve doğru yanıt verip vermeyeceğini yazar.
- **Ölçüm turu:** `kodlar/deney/` sürücüsüyle (ya da Örnek 12 ile) süreleri ve iş sayılarını (düğüm, çağrı, hücre) toplayın; açgözlünün kaç hedefi tutturduğunu sayın.
- **Değerlendirme:** doğruluk / zaman / bellek matrisini (Şekil 13.13) kendi ölçümlerinizle doldurun. Kazanan hangi ölçüte göre kazandı? Hedef 10^9 olsaydı sıralama değişir miydi (sözde polinom)?

12. Kendini Değerlendirme

- ☐ Bir problemde örtüşen alt problemleri ve en iyi alt yapıyı gösterip DP'ye uygunluğunu gerekçelendirebiliyorum (ÖÇ 13.1).
- ☐ Özyinelemeli bir tanımı önce memoization'a, sonra tablolamaya çevirip çağrı sayılarını karşılaştırabiliyorum (ÖÇ 13.2).
- ☐ Bozuk para, LIS, LCS, düzenleme uzaklığı ve 0/1 sırt çantası için reçetenin altı satırını yazabiliyorum (ÖÇ 13.3).
- ☐ Hesaplama sırasını alt problem DAG'ının topolojik sırası olarak seçebiliyor, yanlış sıranın belirtisini tanıyabiliyorum.
- ☐ Tablodan çözümün kendisini (para, alt dizi, işlem dizisi, nesne kümesi) geri çıkarabiliyorum (ÖÇ 13.4).
- ☐ Satır tamponuyla bellek azaltabiliyor, tek satırlı sırt çantasında dolaşma yönünü gerekçelendirebiliyorum (ÖÇ 13.4).
- ☐ Kaba kuvvet, geri izleme, açgözlü ve DP'yi aynı problemde doğruluk, zaman ve bellek açısından karşılaştırabiliyorum (ÖÇ 13.5).

☐ Sözde polinom zamanı açıklayıp ∞ taşması, satır paylaşımı ve eksik memo anahtarı gibi tuzaklardan kaçınabiliyorum.

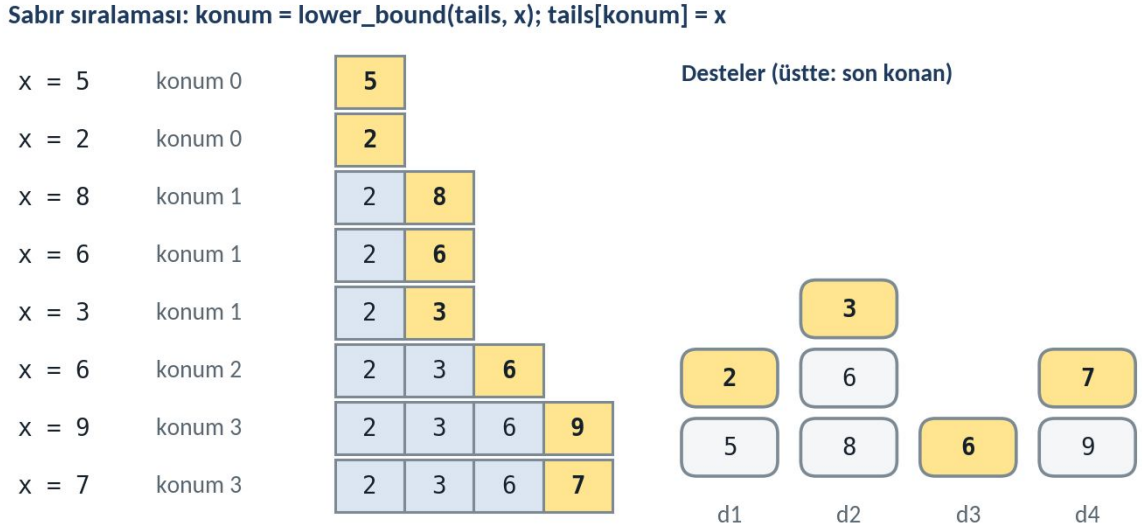
13. İleri Bakış ve Merak Kutusu

İLERİ BAKIŞ · DP nerede yeniden karşımıza çıkacak?

- **H14:** Held ve Karp (1962) gezgin satıcı problemini "ziyaret edilen şehir kümesi + son şehir" durumlu bir DP ile çözdü; küme bir **bit maskesidir**, tablo 2^n ile büyür. Sözde polinom süre ve NP-zorluk H14'ün konusudur.
- **Biyoinformatik:** Needleman ve Wunsch (1970) hizalaması düzenleme uzaklığının puanlı hâlidir: eşleşme ödül, uyuşmazlık ve boşluk ceza alır.
- **Pekiştirmeli öğrenme:** Bellman denklemi bir durumun değerini, eylemler üzerinden "anlık ödül + sonraki durumun (beklenen) değeri" toplamının en büyüğü olarak yazar (Bellman, 1957). **Graflar ve diziler:** Bellman–Ford (Kleinberg & Tardos, 2006), Floyd–Warshall (Cormen vd., 2022) ve gizli Markov modellerindeki Viterbi algoritması da DP'dir.
- **İleri desenler:** aralık DP'si (Örnek 15, Alıştırma 13.14), bit maskesi DP'si (Held–Karp), ağaç DP'si (önkoşuluyla birlikte ders seçimi: v2.12 belgesi § 4.4).

13.1 Merak Kutusu: LIS'i O(n log n)'de bulmak — sabır sıralaması (Örnek 13)

Şekil 13.19
Örnek 13: sabır sıralaması — her adımda konum = lower_bound(tails, x) ve tails dizisi (solda), desteler (sağda); deste sayısı = LIS uzunluğu = 4



deste sayısı = LIS uzunluğu = 4; dizinin kendisi tails_idx + ebeveynle geri çıkarılır
(tails genel olarak bir alt dizi değildir: Örnek 13'ün rastgele dizisi)

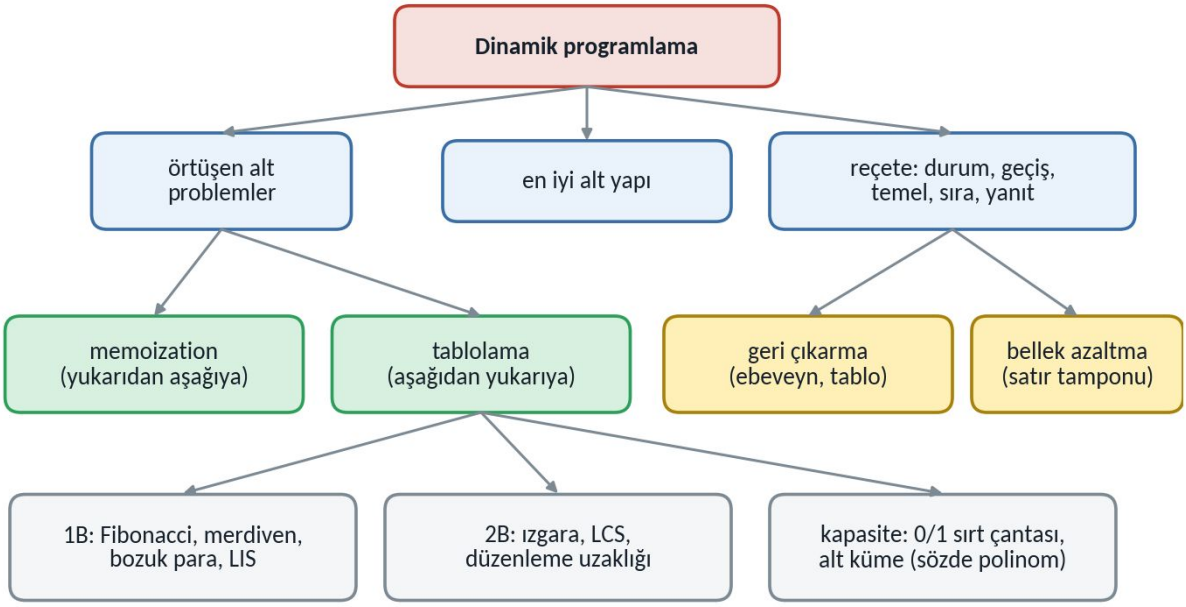
Kartları sırayla destelere koyalım: her kart, üstü ondan **küçük olmayan** en soldaki destenin üstüne konur, yoksa sağa yeni deste açılır. tails[k] = k + 1 uzunluklu artan alt dizilerin bitebileceği en küçük değerdir ve **sıralıdır**; doğru deste H4'ün ikili aramasıyla (`lower_bound`) bulunur (Şekil 13.19). Deste sayısı LIS uzunluğudur: a = {5, 2, 8, 6, 3, 6, 9, 7} için 4 (11 yoklama). Alt dizinin kendisi tails_idx ve ebeveyn dizileriyle geri çıkarılır: 2 3 6 7 — Örnek 7'nin bulduğu 2 3 6 9 ile aynı uzunlukta, başka bir doğru yanıt. n = 10^5'te 858 287 yoklama yeter; O(n^2) sürüm 4 999 950 000 karşılaştırma isterdi.

> MERAK KUTUSU • **Soru**
>
> Son tails dizisi neden genel olarak bir artan **alt dizi** değildir? Örnek 13'ün rastgele dizisinde (n = 16) son tails ile geri çıkarılan alt diziyi karşılaştırın; hangi öğeler farklı ve neden?

14. Özet, Kavram Haritası ve Terim Sözlüğü

- DP iki koşul ister: **örtüşen alt problemler** (aynı alt problem tekrar tekrar sorulur) ve **en iyi alt yapı** (en iyi çözüm alt problemlerin en iyi çözümlerinden kurulur). Böl ve fethette alt problemler ayrıktır; açgözlü tek seçeneği dener, DP hepsini dener ama her alt problemi bir kez çözer.
- Reçete: durum, geçiş, temel durum, hesaplama sırası, yanıtın yeri, geri çıkarma. Hesaplama sırası alt problem DAG'ının topolojik sırasıdır; en iyileme DP'si o DAG'da en kısa ya da en uzun yolu, sayma DP'si yolların sayısını bulur.
- Memoization (yukarıdan aşağıya) yalnız gereken durumları hesaplar ama özyinelemelidir; tablolama (aşağıdan yukarıya) bütün tabloyu döngüyle doldurur ve satır tamponuna kolay iner. Fibonacci: saf 25 çağrı ↔ memo 11 çağrı (n = 6).
- 1B (bozuk para, LIS), 2B (LCS, düzenleme uzaklığı, ızgara) ve kapasite boyutlu (0/1 sırt çantası, alt küme toplamı) tablolar; Θ(n·W) sözde polinomdur. Tek satırda kapasite ters dolaşılır; iki satırla bellek azalır ama geri çıkarma kaybolur.

Şekil 13.20
Hafta 13 kavram haritası



Tablo 13.13
Terim sözlüğü (Yol Haritası Bölüm 6.8 ile uyumlu)

Türkçe	İngilizce	Not
dinamik programlama; örtüşen alt problemler; en iyi alt yapı	dynamic programming; overlapping subproblems; optimal substructure	iki koşul

Türkçe	İngilizce	Not
durum; geçiş; temel durum; hesaplama sırası	state; transition (recurrence); base case; evaluation order	DP reçetesi
memoization (not alma); tablolama	memoization; tabulation	yukarıdan aşağıya / aşağıdan yukarıya
geri çıkarma	reconstruction (traceback)	“geri izleme” (backtracking) H3'ün yöntemi
satır tamponu; bellek azaltma	rolling array; space optimization	iki satır, tek satır
en uzun artan alt dizi; en uzun ortak alt dizi	longest increasing subsequence (LIS); longest common subsequence (LCS)	alt dizi ardışık değil
düzenleme (Levenshtein) uzaklığı	edit (Levenshtein) distance	silme, ekleme, değiştirme
0/1 sırt çantası; sözde polinom zaman	0/1 knapsack; pseudo-polynomial time	W'nin bit uzunluğunda üstel

15. Kaynaklar

Bellman, R. (1952). On the theory of dynamic programming. *Proceedings of the National Academy of Sciences*, *38*(8), 716–719. https://doi.org/10.1073/pnas.38.8.716

Bellman, R. (1957). *Dynamic programming*. Princeton University Press.

Bellman, R. (1984). *Eye of the hurricane: An autobiography*. World Scientific. https://doi.org/10.1142/0076

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Dreyfus, S. (2002). Richard Bellman on the birth of dynamic programming. *Operations Research*, *50*(1), 48–51. https://doi.org/10.1287/opre.50.1.48.17791

Held, M., & Karp, R. M. (1962). A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics*, *10*(1), 196–210. https://doi.org/10.1137/0110015

Hirschberg, D. S. (1975). A linear space algorithm for computing maximal common subsequences. *Communications of the ACM*, *18*(6), 341–343. https://doi.org/10.1145/360825.360861

Hunt, J. W., & McIlroy, M. D. (1976). *An algorithm for differential file comparison* (Computing Science Technical Report No. 41). Bell Laboratories. https://www.cs.dartmouth.edu/doug/diff.pdf

Kleinberg, J., & Tardos, É. (2006). *Algorithm design*. Pearson/Addison-Wesley.

Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, *10*(8), 707–710. (Özgün eser 1965'te yayımlandı)

Needleman, S. B., & Wunsch, C. D. (1970). A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology*, *48*(3), 443–453. https://doi.org/10.1016/0022-2836(70)90057-4

Office of the Secretary of Defense Historical Office. (t.y.). *Charles E. Wilson*. Erişim tarihi 22 Ekim 2026, https://history.defense.gov/Multimedia/Biographies/Article-View/Article/571268/charles-e-wilson/

Parsons, D., & Haden, P. (2006). Parson’s programming puzzles: A fun and effective learning tool for first programming courses. In *Proceedings of the 8th Australasian Conference on Computing Education* (Vol. 52, pp. 157–163). Australian Computer Society.

Seyfettin, Ö. (2022). *Diyet*. Vikikaynak. https://tr.wikisource.org/w/index.php?title=Diyet&oldid=151645 (Özgün eser 1918'de yayımlandı)

Seyfettin, Ö. (2023). *Ferman*. Vikikaynak. https://tr.wikisource.org/w/index.php?title=Ferman&oldid=163811 (Özgün eser 1917'de yayımlandı)

Seyfettin, Ö. (2024). *Forsa*. Vikikaynak. https://tr.wikisource.org/w/index.php?title=Forsa&oldid=167612 (Özgün eser 1919'da yayımlandı)

Wagner, R. A., & Fischer, M. J. (1974). The string-to-string correction problem. *Journal of the ACM*, *21*(1), 168–173. https://doi.org/10.1145/321796.321811

Ek A. Dil Karşılıkları

Bu ekte örnek merdiveninin on beş basamağının C++ ve Python sürümlerinden DP çekirdeği (tabloyu dolduran fonksiyon) dosyadaki satır numaralarıyla verilir; öbür sürümler (Örnek 1'de tablo ve iki değişken, Örnek 4'te permütasyon) ve yardımcı fonksiyonlar (geri çıkarma, parantezleme, UTF-8 çözme, `alt_sinir`, yazdırma) tam dosyalardadır: `kodlar/cpp/` ve `kodlar/python/`. Hardal satırlar C sürümünden **anlam olarak** farklılaşan satırlardır (`kodlar/ornekler.json` → `farklar`); fonksiyon dışındaki farklar not kutusunda açıklanır. Fonksiyon ve değişken adları üç dilde aynıdır. Örnek 14 ve 15 meydan okumadır; Örnek 6'nın çıktısı dillerde farklıdır (Δ), öbürlerininki birebir aynıdır.

Tablo 13.14
Derleme ve çalıştırma komutları (X: örneğin kök adı, ör. H13_03_bozuk_para_en_az)

Dil	Derleme / çalıştırma	Not
C	`gcc -std=c17 -Wall -Wextra -pedantic -O2 X.c [ortak_rastgele.c] -o X -lm · ./X`	ortak_rastgele.c yalnız #include eden örneklerde
C++	`g++ -std=c++17 -Wall -Wextra -pedantic -O2 X.cpp -o X · ./X`	ortak başlıklar (.hpp) yalnızca #include
Python	`python3 X.py (Windows: py X.py)`	Python 3.10 ve üstü; yalnızca standart kütüphane
Girdi	`./H13_09_duzenleme_uzakligi < ../girdiler/H13_09_duzenleme_uzakligi.txt (Örnek 14 de sözlüğü standart girdiden okur)`	girdi dosyaları kodlar/girdiler/ altında

A.1 Örnek 1 · Fibonacci: memoization

C++ **cpp/H13_01_fibonacci.cpp · satır 38–51**

```cpp
38  static long long fib_memo(int i)
39  {
40      cagri++;
41      if (memo[i] != -1) {
42          return memo[i];                    // not alınmış: yeniden çözme
43      }
44      if (i < 2) {
45          memo[i] = i;
46      } else {
47          memo[i] = fib_memo(i - 1) + fib_memo(i - 2);
48      }
49      hesaplanan++;
50      return memo[i];
51  }
```

Python **python/H13_01_fibonacci.py · satır 30–41**

```python
30  def fib_memo(i):
31      """2) Memo (yukarıdan aşağıya): memo[i] == -1 "hesaplanmadı" demektir"""
32      global cagri, hesaplanan
33      cagri += 1
34      if memo[i] != -1:
35          return memo[i]              # not alınmış: yeniden çözme
```

```
    if i < 2:
        memo[i] = i
    else:
        memo[i] = fib_memo(i - 1) + fib_memo(i - 2)
    hesaplanan += 1
    return memo[i]
```

DİL PENCERESİ · Farklılaşan satırlar

Memo C'de statik dizi ve −1 işaretiyle, C++'ta `std::vector<long long>` ile, Python'da listeyle tutulur; Python'da aynı işi `functools.cache` dekoratörü yapar (kavram K02). C/C++'ta `uint64_t` toplamadan ÖNCE `onceki > UINT64_MAX - simdiki` ile denetlenir; Python tamsayısı taşmadığı için sınır `(1 << 64) - 1` sabitiyle elle konur. İki değişkenli döngü Python'da çoklu atamayla yazılır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 25, 55, 82, 84; Python 14, 18, 60, 70.)

A.2 Örnek 2 · Merdiven çıkma

C++ cpp/H13_02_merdiven.cpp · satır 28–35

```
static void merdiven_tablo(int n)
{
    yol[0] = 1;                              // temel durum
    yol[1] = 1;
    for (int i = 2; i <= n; i++) {
        yol[i] = yol[i - 1] + yol[i - 2]; // son adım 1 ya da 2
    }
}
```

Python python/H13_02_merdiven.py · satır 22–27

```
def merdiven_tablo(n):
    """yol[0..n] tablosunu doldurur (1 ya da 2 basamak)"""
    yol[0] = 1                       # temel durum
    yol[1] = 1
    for i in range(2, n + 1):
        yol[i] = yol[i - 1] + yol[i - 2]    # son adım 1 ya da 2
```

DİL PENCERESİ · Farklılaşan satırlar

Listelemedeki o anki yol C'de sabit dizi + sayaçla, C++'ta `std::vector` üzerinde `push_back`/`pop_back`, Python'da listede `append`/`pop` ile (yığın gibi) tutulur. yol[n] n = 40'a dek `long long`'a sığar; büyük n için üç dilde de her toplamadan sonra mod $10^9 + 7$ alınır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 24, 51, 53; Python 18, 35, 41.)

A.3 Örnek 3 · Bozuk para: en az para

C++ cpp/H13_03_bozuk_para_en_az.cpp · satır 32–47

```
static void en_az_para(const std::vector<int>& paralar, int S)
{
```

```
    enaz[0] = 0;                            // temel durum
    secim[0] = 0;
    for (int t = 1; t <= S; t++) {
        enaz[t] = SONSUZ;                   // henüz ulaşılamadı
        secim[t] = 0;
        for (int c : paralar) {
            if (c <= t && enaz[t - c] != SONSUZ &&    // taşma koruması
                enaz[t - c] + 1 < enaz[t]) {
                enaz[t] = enaz[t - c] + 1;
                secim[t] = c;
            }
        }
    }
}
```

Python `python/H13_03_bozuk_para_en_az.py · satır 24–35`

```
def en_az_para(paralar, S):
    """paralar ARTAN sıralı; enaz[0..S] ve secim[0..S] doldurulur"""
    enaz[0] = 0                     # temel durum
    secim[0] = 0
    for t in range(1, S + 1):
        enaz[t] = SONSUZ            # henüz ulaşılamadı
        secim[t] = 0
        for c in paralar:
            if (c <= t and enaz[t - c] != SONSUZ
                    and enaz[t - c] + 1 < enaz[t]):
                enaz[t] = enaz[t - c] + 1
                secim[t] = c
```

DİL PENCERESİ · Farklılaşan satırlar

∞ üç dilde aynı tamsayı sabitidir: C'de `INT_MAX`, C++'ta `std::numeric_limits<int>::max()`, Python'da 2147483647 (`math.inf` yazdırılınca `inf` çıkar, çıktı farklılaşır). `enaz[t - c] != SONSUZ` denetimi C/C++'ta `INT_MAX + 1` işaretli taşmasını (tanımsız davranış) önler; Python'da taşma olmaz ama aynı denetim tutarlılık için korunur. Açgözlü paraları C'de indisle, C++'ta ters yineleyiciyle, Python'da `reversed` ile büyükten küçüğe dolaşır. Eşitlik kuralı üç dilde aynı: paralar artan denenir, yalnız kesin küçükte güncellenir (eşitlikte küçük para kalır). (Gösterilen fonksiyonların dışındaki fark satırları: C++ 26, 54, 64; Python 18, 41, 45.)

A.4 Örnek 4 · Bozuk para: kaç farklı yol (kombinasyon)

C++ `cpp/H13_04_bozuk_para_yollar.cpp · satır 39–56`

```
static long long kombinasyon(const std::vector<int>& paralar, int S,
                             bool satirlari_yaz)
{
    yol[0] = 1;                             // temel durum
    for (int t = 1; t <= S; t++) {
        yol[t] = 0;
    }
    for (int c : paralar) {                 // DIŞ: para
        for (int t = c; t <= S; t++) {      // İÇ: tutar
```

```
            yol[t] = (yol[t] + yol[t - c]) % MOD;
        }
        if (satirlari_yaz) {
            std::cout << "  para " << c << " sonrası:";
            yaz_satir(S);
        }
    }
    return yol[S];
}
```

Python `python/H13_04_bozuk_para_yollar.py · satır 27–38`

```python
def kombinasyon(paralar, S, satirlari_yaz):
    """Kombinasyon: dış döngü para. c parası işlenirken yol[t], yalnız ilk
    paralarla (c dahil) yapılan ödemeleri sayar: sıra önemsiz."""
    yol[0] = 1                      # temel durum
    for t in range(1, S + 1):
        yol[t] = 0
    for c in paralar:               # DIŞ: para
        for t in range(c, S + 1):   # İÇ: tutar
            yol[t] = (yol[t] + yol[t - c]) % MOD
        if satirlari_yaz:
            print(f"  para {c} sonrası:" + yaz_satir(S))
    return yol[S]
```

DİL PENCERESİ · Farklılaşan satırlar

Döngü sırası tuzağı üç dilde aynıdır: dış döngü para → kombinasyon, dış döngü tutar → permütasyon. Sayımlar mod $10^9 + 7$ ile `long long` aralığında tutulur; Python tamsayısı taşmadığı için mod yalnız aynı çıktı için uygulanır. Seçilen paralar C'de sabit dizi + sayaçla, C++'ta vektörle, Python'da listeyle yığın gibi tutulur. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 27, 95; Python 20, 54, 64.)

A.5 Örnek 5 · Izgara: yol sayısı ve en küçük maliyet

C++ `cpp/H13_05_izgara.cpp · satır 103–123`

```
static int en_kucuk_maliyet(std::vector<std::vector<int>>& m)
{
    m.assign(R, std::vector<int>(C, 0));
    for (int i = 0; i < R; i++) {
        for (int j = 0; j < C; j++) {
            if (a[i][j] == ENGEL) {
                m[i][j] = SONSUZ;
                continue;
            }
            if (i == 0 && j == 0) {
                m[i][j] = a[0][0];              // temel durum
                continue;
            }
            int ust = i > 0 ? m[i - 1][j] : SONSUZ;
            int sol = j > 0 ? m[i][j - 1] : SONSUZ;
            int enk = ust <= sol ? ust : sol;  // eşitlikte yukarı
            m[i][j] = enk == SONSUZ ? SONSUZ : enk + a[i][j];
```

```
        }
    }
    return m[R - 1][C - 1];
}
```

Python python/H13_05_izgara.py · satır 80–95

```python
def en_kucuk_maliyet():
    """(3) en küçük maliyet; m[i][j] = SONSUZ: engel ya da ulaşılamaz"""
    m = [[0] * C for _ in range(R)]
    for i in range(R):
        for j in range(C):
            if a[i][j] == ENGEL:
                m[i][j] = SONSUZ
                continue
            if i == 0 and j == 0:
                m[i][j] = a[0][0]   # temel durum
                continue
            ust = m[i - 1][j] if i > 0 else SONSUZ
            sol = m[i][j - 1] if j > 0 else SONSUZ
            enk = ust if ust <= sol else sol    # eşitlikte yukarı
            m[i][j] = SONSUZ if enk == SONSUZ else enk + a[i][j]
    return m[R - 1][C - 1], m
```

DİL PENCERESİ · **Farklılaşan satırlar**

C'de ızgara sabit boyutlu 2B dizidir (`long long yol[R][C]`, satır öncelikli ve bitişik); C++'ta `std::vector<std::vector<long long>>`, Python'da satırları ayrı listeler (`[[0] * c for _ in range(r)]`, Örnek 6). C++17 yapısal bağlaması (`const auto& [i, j]`) yol hücrelerini açar. Binom kâhininde C/C++ `/`, Python `//` kullanır (her adım tam bölünür). Geri çıkarmada eşitlikte üç dilde de yukarıdan gelinir. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 26, 46, 63, 233; Python 36, 49, 60, 181.)

A.6 Örnek 6 · 2B tablo kurma (Δ)

C++ cpp/H13_06_tablo_kurma.cpp · satır 29–38

```cpp
static void pascal_doldur(Tablo& t, int n)
{
    for (int i = 0; i < n; i++) {
        t[i][0] = 1;                                // C(i, 0) = 1
        t[i][i] = 1;                                // C(i, i) = 1
        for (int j = 1; j < i; j++) {
            t[i][j] = t[i - 1][j - 1] + t[i - 1][j];
        }
    }
}
```

Python python/H13_06_tablo_kurma.py · satır 27–33

```python
def pascal_doldur(t, n):
    """Pascal üçgeni: t[i][j] = C(i, j), 0 <= j <= i < n"""
    for i in range(n):
```

```
30          t[i][0] = 1                 # C(i, 0) = 1
31          t[i][i] = 1                 # C(i, i) = 1
32          for j in range(1, i):
33              t[i][j] = t[i - 1][j - 1] + t[i - 1][j]
```

DİL PENCERESİ · Farklılaşan satırlar

Bölüm 1 dile özgüdür (Δ). C'de tablo tek `malloc` bloğudur ve `t[i * m + j]` ile indislenir (satırlar bitişik, satır öncelikli). C++'ta `std::vector<std::vector<int>>` her satırı ayrı bir vektör olarak kurar. Python'da `[[0] * m] * n` AYNI satır listesine n başvuru üretir: `t[0][0] = 7` bütün satırları değiştirir ve Pascal üçgeni yanlış çıkar (C(5, 2) = 13); doğru biçim `[[0] * m for _ in range(n)]`. Bölüm 2 üç dilde aynıdır; paylaşım `is` ile gösterilir, `id()` yazdırılmaz. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 20, 25; Python 19, 24, 48.)

ÇIKTILAR, 1. BÖLÜM (Δ; 2. BÖLÜM, PASCAL ÜÇGENİ, ÜÇ DİLDE AYNI)

```
$ ./H13_06_tablo_kurma  ·  $ python3 H13_06_tablo_kurma.py
[C]
1) 2B tablo, C: tek malloc bloğu, t[i * m + j] (n = 6, m = 6)
  malloc(n * m * sizeof(int)): 36 hücre TEK parça
  hücre (i, j) -> indis i * m + j: (0,0)->0 (0,5)->5 (1,0)->6 (5,5)->35
  satır başları: 0 6 12 18 24 30  (satır i, satır i-1'in hemen ardından)
  t[0 * m + 0] = 7: öteki satırların ilk hücreleri: 0 0 0 0 0
  (her satır ayrı hücrelerdir; paylaşım yok)

[C++]
1) 2B tablo, C++: std::vector<std::vector<int>> t(n, std::vector<int>(m, 0)) (n = 6,
m = 6)
  t.size() = 6 satır, t[0].size() = 6 sütun; her satır AYRI bir vektör
  (satırlar bellekte bitişik olmak zorunda değil; tek blok: std::vector<int> t(n *
m), t[i * m + j])
  t[0][0] = 7: öteki satırların ilk hücreleri: 0 0 0 0 0
  (her satır ayrı hücrelerdir; paylaşım yok)

[Python]
1) 2B tablo, Python: TUZAK t = [[0] * m] * n (n = 6, m = 6)
  t[0][0] = 7: öteki satırların ilk hücreleri: 7 7 7 7 7
  t[0] is t[5]: True  (AYNI satır nesnesi 6 kez paylaşılıyor)
  bu tabloyla Pascal üçgeni (YANLIŞ):
       1
       1    5
       1    5   13
       1    5   13   22
       1    5   13   22   23
       1    5   13   22   23    1
  C(5, 2) = 13 (doğrusu 10): satır i-1 diye okunan, aynı satırın bu turda değişmiş
hâli
  doğru biçim: t = [[0] * m for _ in range(n)]
  t[0][0] = 7: öteki satırların ilk hücreleri: 0 0 0 0 0;  t[0] is t[5]: False
```

A.7 Örnek 7 · LIS O(n²)

C++ cpp/H13_07_lis_n2.cpp · satır 28–48

```cpp
static int lis_n2(const std::vector<int>& a, std::vector<int>& uz,
                  std::vector<int>& ebeveyn)
{
    const int n = static_cast<int>(a.size());
    uz.assign(n, 1);                          // temel: yalnız a[i]
    ebeveyn.assign(n, -1);
    int son = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++) {
            karsilastirma++;
            if (a[j] < a[i] && uz[j] + 1 > uz[i]) {
                uz[i] = uz[j] + 1;
                ebeveyn[i] = j;
            }
        }
        if (uz[i] > uz[son]) {                // eşitlikte küçük i
            son = i;
        }
    }
    return son;
}
```

Python python/H13_07_lis_n2.py · satır 19–34

```python
def lis_n2(a, uz, ebeveyn):
    """uz ve ebeveyn doldurulur; dönen: en uzun alt dizinin son indisi"""
    global karsilastirma
    n = len(a)
    uz[:] = [1] * n                # temel: yalnız a[i]
    ebeveyn[:] = [-1] * n
    son = 0
    for i in range(n):
        for j in range(i):
            karsilastirma += 1
            if a[j] < a[i] and uz[j] + 1 > uz[i]:
                uz[i] = uz[j] + 1
                ebeveyn[i] = j
        if uz[i] > uz[son]:        # eşitlikte küçük i
            son = i
    return son
```

DİL PENCERESİ · Farklılaşan satırlar

C'de `uz` ve `ebeveyn` çağıranın dizileridir; C++'ta başvuruyla (`std::vector<int>&`) geçen vektörler `assign` ile, Python'da `uz[:] = …` dilim atamasıyla yerinde doldurulur. Geri çıkarılan indisler C'de elle, C++'ta `std::swap` ile, Python'da `list.reverse()` ile ters çevrilir. Ortak LCG'nin `rastgele_dizi`'si C'de diziyi doldurur, C++/Python'da yeni dizi döndürür. Eşitlikte üç dilde de en küçük j (yalnız kesin büyükte güncelleme) ve en küçük son indis seçilir. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 58, 140; Python 44, 100.)

A.8 Örnek 8 · LCS tablosu

C++ cpp/H13_08_lcs.cpp · satır 29–45

```cpp
static Tablo lcs_tablo(const Dizi& a, const Dizi& b)
{
    const std::size_t n = a.size(), m = b.size();
    Tablo L(n + 1, std::vector<int>(m + 1, 0));   // L[0][j] = L[i][0] = 0
    for (std::size_t i = 1; i <= n; i++) {
        for (std::size_t j = 1; j <= m; j++) {
            if (a[i - 1] == b[j - 1]) {
                L[i][j] = L[i - 1][j - 1] + 1;
            } else if (L[i - 1][j] >= L[i][j - 1]) {
                L[i][j] = L[i - 1][j];
            } else {
                L[i][j] = L[i][j - 1];
            }
        }
    }
    return L;
}
```

Python python/H13_08_lcs.py · satır 18–30

```python
def lcs_tablo(a, b):
    """L[i][j] tablosu; her satır ayrı liste (Örnek 6)"""
    n, m = len(a), len(b)
    L = [[0] * (m + 1) for _ in range(n + 1)]  # L[0][j] = L[i][0] = 0
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                L[i][j] = L[i - 1][j - 1] + 1
            elif L[i - 1][j] >= L[i][j - 1]:
                L[i][j] = L[i - 1][j]
            else:
                L[i][j] = L[i][j - 1]
    return L
```

> DİL PENCERESİ · **Farklılaşan satırlar**
>
> Aynı DP hem karakterlere hem satırlara uygulanır: C'de satır diff'i için `strcmp`'li ikinci bir döngü yazılır; C++'ta `lcs_tablo` şablonu (`template <class Dizi>`) `std::string` ve `std::vector<std::string>` ile, Python'da aynı fonksiyon `str` ve `list` ile çalışır. C'de tablo tek blok (`L[i * (m + 1) + j]`), C++/Python'da satır satır. Eşitlikte yön üç dilde aynı: önce yukarı, sonra sol. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 28, 53, 96; Python 53, 69.)

A.9 Örnek 9 · Düzenleme uzaklığı

C++ cpp/H13_09_duzenleme_uzakligi.cpp · satır 71–92

```cpp
static int uzaklik(const std::vector<int>& a, const std::vector<int>& b)
{
    const std::size_t n = a.size(), m = b.size();
    D.assign(n + 1, std::vector<int>(m + 1, 0));
```

```
    for (std::size_t i = 0; i <= n; i++) {
        D[i][0] = static_cast<int>(i);          // i silme
    }
    for (std::size_t j = 0; j <= m; j++) {
        D[0][j] = static_cast<int>(j);          // j ekleme
    }
    for (std::size_t i = 1; i <= n; i++) {
        for (std::size_t j = 1; j <= m; j++) {
            int sil = D[i - 1][j] + 1;
            int ekle = D[i][j - 1] + 1;
            int degis = D[i - 1][j - 1] + (a[i - 1] != b[j - 1]);
            int enk = sil < ekle ? sil : ekle;
            D[i][j] = degis < enk ? degis : enk;
        }
    }
    hucre += static_cast<long long>((n + 1) * (m + 1));
    return D[n][m];
}
```

Python `python/H13_09_duzenleme_uzakligi.py · satır 52–69`

```
def uzaklik(a, b):
    """D tablosunu doldurur (satır öncelikli)"""
    global D, hucre
    n, m = len(a), len(b)
    D = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        D[i][0] = i                 # i silme
    for j in range(m + 1):
        D[0][j] = j                 # j ekleme
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            sil = D[i - 1][j] + 1
            ekle = D[i][j - 1] + 1
            degis = D[i - 1][j - 1] + (a[i - 1] != b[j - 1])
            enk = sil if sil < ekle else ekle
            D[i][j] = degis if degis < enk else enk
    hucre += (n + 1) * (m + 1)
    return D[n][m]
```

> DİL PENCERESİ · **Farklılaşan satırlar**
>
> Üç dilde de girdi BAYT olarak okunur ve UTF-8 elle çözülür (kod noktası düzeyi); Python'da `bytes` öğeleri 0–255 tamsayıdır, C++'ta baytlar `unsigned char`'a çevrilir. Sözlük bayt sırasıyla sıralanır (C `qsort` + `strcmp`, C++ `std::sort`, Python `list.sort`; sıralama burada yalnız araçtır); eşit uzaklıkta sözlükte önce gelen seçildiği için öneriler üç dilde aynıdır. Kod noktası C/C++'ta elle UTF-8'e kodlanarak, Python'da `chr` ile yazdırılır. Geri çıkarmada eşitlik önceliği üç dilde aynı: eşleşme > silme > ekleme > değiştirme. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 38, 64, 250, 262; Python 49, 99, 169, 176.)

A.10 Örnek 10 · 0/1 sırt çantası

C++ `cpp/H13_10_sirt_cantasi.cpp · satır 31–51`

```cpp
static Tablo sirt_cantasi(const std::vector<int>& a,
                          const std::vector<int>& d, int W)
{
    const std::size_t n = a.size();
    Tablo K(n + 1, std::vector<int>(W + 1, 0));   // temel: nesne yok
    for (std::size_t i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            int al_ma = K[i - 1][w];
            int en_iyi = al_ma;
            if (a[i - 1] <= w) {
                int al = K[i - 1][w - a[i - 1]] + d[i - 1];
                if (al > en_iyi) {
                    en_iyi = al;
                }
            }
            K[i][w] = en_iyi;
            hucre++;
        }
    }
    return K;
}
```

Python `python/H13_10_sirt_cantasi.py · satır 21–36`

```python
def sirt_cantasi(a, d, W):
    """(n+1) x (W+1) tablo; her satır ayrı liste (Örnek 6)"""
    global hucre
    n = len(a)
    K = [[0] * (W + 1) for _ in range(n + 1)]  # temel: nesne yok
    for i in range(1, n + 1):
        for w in range(W + 1):
            al_ma = K[i - 1][w]
            en_iyi = al_ma
            if a[i - 1] <= w:
                al = K[i - 1][w - a[i - 1]] + d[i - 1]
                if al > en_iyi:
                    en_iyi = al
            K[i][w] = en_iyi
            hucre += 1
    return K
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> C'de tablo tek `malloc` bloğudur (`K[i * (W + 1) + w]`; sahiplik çağıranda, `free`), C++'ta `std::vector<std::vector<int>>`, Python'da satırları ayrı listeler. Seçim C'de çıktı dizisine, C++'ta `std::vector<bool>&`'e yazılır; Python iki değer (liste, ağırlık) döndürür. Açgözlünün oran karşılaştırması üç dilde tamsayıyla (`d_i * a_j > d_j * a_i`) yapılır, kayan nokta kullanılmaz. Geri çıkarmada K[i][w] = K[i−1][w] ise nesne üç dilde de alınmadı sayılır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 54, 195; Python 49, 132.)

A.11 Örnek 11 · 0/1 sırt çantası: tek satır

C++ cpp/H13_11_sirt_tek_satir.cpp · satır 47–66

```cpp
static int tek_satir_ters(const Dizi& a, const Dizi& d,
                          const std::vector<std::string>& ad, int W,
                          bool satirlari_yaz, Dizi& dp)
{
    dp.assign(W + 1, 0);                        // temel
    for (std::size_t i = 0; i < a.size(); i++) {
        for (int w = W; w >= a[i]; w--) {       // TERS
            if (dp[w - a[i]] + d[i] > dp[w]) {
                if (static_cast<int>(i) == iz_nesne) {
                    guncelleme_yaz(w, dp[w], dp[w - a[i]] + d[i], w - a[i]);
                }
                dp[w] = dp[w - a[i]] + d[i];
            }
        }
        if (satirlari_yaz) {
            satir_yaz("+" + ad[i], dp);
        }
    }
    return dp[W];
}
```

Python python/H13_11_sirt_tek_satir.py · satır 29–40

```python
def tek_satir_ters(a, d, ad, W, satirlari_yaz):
    """0/1: w TERS sırada; satirlari_yaz ise her nesneden sonra satır"""
    dp = [0] * (W + 1)              # temel
    for i in range(len(a)):
        for w in range(W, a[i] - 1, -1):        # TERS
            if dp[w - a[i]] + d[i] > dp[w]:
                if i == iz_nesne:
                    guncelleme_yaz(w, dp[w], dp[w - a[i]] + d[i], w - a[i])
                dp[w] = dp[w - a[i]] + d[i]
        if satirlari_yaz:
            satir_yaz("+" + ad[i], dp)
    return dp[W], dp
```

> DİL PENCERESİ · **Farklılaşan satırlar**
>
> Ters dolaşma C/C++'ta `for (int w = W; w >= a[i]; w--)`, Python'da `range(W, a[i] - 1, -1)`; artan sürüm `range(a[i], W + 1)`. Son satırın tam tabloyla karşılaştırması C'de döngüyle, C++'ta vektör `==`, Python'da liste `==` ile yapılır. Python fonksiyonları (değer, satır) demeti döndürür. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 107, 159; Python 48, 109.)

A.12 Örnek 12 · Alt küme toplamı

C++ cpp/H13_12_alt_kume_toplami.cpp · satır 55–75

```cpp
static void dp_doldur(int T)
{
    const std::size_t n = a.size();
```

```
    var_t.assign(n + 1, std::vector<bool>(T + 1, false));
    say_t.assign(n + 1, std::vector<long long>(T + 1, 0));
    var_t[0][0] = true;                          // boş küme: toplam 0
    say_t[0][0] = 1;
    hucre = T + 1;                               // satır 0
    for (std::size_t i = 1; i <= n; i++) {
        int x = a[i - 1];
        for (int t = 0; t <= T; t++) {
            var_t[i][t] = var_t[i - 1][t];                    // almadan
            say_t[i][t] = say_t[i - 1][t];
            if (x <= t) {                                     // alarak
                var_t[i][t] = var_t[i][t] || var_t[i - 1][t - x];
                say_t[i][t] += say_t[i - 1][t - x];
            }
            hucre++;
        }
    }
}
```

Python `python/H13_12_alt_kume_toplami.py · satır 45–61`

```
def dp_doldur(T):
    global var_t, say_t, hucre
    n = len(a)
    var_t = [[False] * (T + 1) for _ in range(n + 1)]
    say_t = [[0] * (T + 1) for _ in range(n + 1)]
    var_t[0][0] = True              # boş küme: toplam 0
    say_t[0][0] = 1
    hucre = T + 1                   # satır 0
    for i in range(1, n + 1):
        x = a[i - 1]
        for t in range(T + 1):
            var_t[i][t] = var_t[i - 1][t]              # almadan
            say_t[i][t] = say_t[i - 1][t]
            if x <= t:                                 # alarak
                var_t[i][t] = var_t[i][t] or var_t[i - 1][t - x]
                say_t[i][t] += say_t[i - 1][t - x]
            hucre += 1
```

DİL PENCERESİ · **Farklılaşan satırlar**

Bool tablo C'de `unsigned char` dizisidir (`calloc`; sahiplik modülde, `dp_birak` ile bırakılır), C++'ta bit paketli `std::vector<std::vector<bool>>`, Python'da `bool` listeleri. Sayım tablosu `long long` / Python tamsayısı; n ≤ 18'de en büyük sayım 2090, taşma yok. Python'da toplam bile `sum` yerine açık döngüyle hesaplanır. Geri çıkarmada var[i−1][t] doğruysa üç dilde de a_i alınmaz (eşitlikte almadan). (Gösterilen fonksiyonların dışındaki fark satırları: C++ 51, 94, 151; Python 114.)

A.13 Örnek 13 · LIS O(n log n): sabır sıralaması

C++ `cpp/H13_13_lis_nlogn.cpp · satır 57–83`

```
static int lis_nlogn(const Dizi& a, Dizi& tails, Dizi& tails_idx,
                     Dizi& ebeveyn, bool iz)
```

```
{
    const int n = static_cast<int>(a.size());
    tails.assign(n, 0);
    tails_idx.assign(n, 0);
    ebeveyn.assign(n, -1);
    int L = 0;
    for (int i = 0; i < n; i++) {
        int k = alt_sinir(tails, L, a[i]);
        tails[k] = a[i];                        // yeni deste ya da üstüne
        tails_idx[k] = i;
        ebeveyn[i] = k > 0 ? tails_idx[k - 1] : -1;
        if (k == L) {
            L++;
        }
        if (iz) {
            std::cout << "  x = " << a[i] << " -> konum " << k
                      << ": tails ";
            liste_yaz(tails, L);
            std::cout << ", tails_idx ";
            liste_yaz(tails_idx, L);
            std::cout << ", ebeveyn[" << i << "] = " << ebeveyn[i] << '\n';
        }
    }
    return L;
}
```

Python **python/H13_13_lis_nlogn.py · satır 40–59**

```
def lis_nlogn(a, iz):
    """Sabır sıralaması; iz ise her adımı yazar.
    Dönen: (L, tails, tails_idx, ebeveyn)"""
    n = len(a)
    tails = [0] * n
    tails_idx = [0] * n
    ebeveyn = [-1] * n
    L = 0
    for i in range(n):
        k = alt_sinir(tails, L, a[i])
        tails[k] = a[i]             # yeni deste ya da üstüne
        tails_idx[k] = i
        ebeveyn[i] = tails_idx[k - 1] if k > 0 else -1
        if k == L:
            L += 1
        if iz:
            print(f"  x = {a[i]} -> konum {k}: tails {liste_yaz(tails, L)}, "
                  f"tails_idx {liste_yaz(tails_idx, L)}, "
                  f"ebeveyn[{i}] = {ebeveyn[i]}")
    return L, tails, tails_idx, ebeveyn
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> lower_bound H4'teki elle yazılmış ikili aramadır; `std::lower_bound` ve `bisect.bisect_left` yalnız kâhindir. C'de büyük n için diziler `malloc` ile ayrılıp bırakılır; C++/Python'da vektör/liste. Python'da orta nokta `//` ile bulunur; fonksiyon dört değeri demet olarak döndürür. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 46, 161; Python 27, 37.)

A.14 Örnek 14 · Kelime bölme (meydan okuma)

C++ cpp/H13_14_kelime_bolme.cpp · satır 41–62

```cpp
static bool kelime_bol(const std::string& s)
{
    const int n = static_cast<int>(s.size());
    ok.assign(n + 1, false);
    son.assign(n + 1, -1);
    say.assign(n + 1, 0);
    ok[0] = true;                              // boş önek
    say[0] = 1;
    for (int i = 1; i <= n; i++) {
        int j0 = i - en_uzun > 0 ? i - en_uzun : 0;
        for (int j = j0; j < i; j++) {
            if (ok[j] && sozlukte_mi(s, j, i - j)) {
                if (!ok[i]) {
                    son[i] = j;                // en küçük j
                }
                ok[i] = true;
                say[i] += say[j];
            }
        }
    }
    return ok[n];
}
```

Python python/H13_14_kelime_bolme.py · satır 35–52

```python
def kelime_bol(s):
    """Önek DP'si: ok, son, say doldurulur"""
    global ok, son, say
    n = len(s)
    ok = [False] * (n + 1)
    son = [-1] * (n + 1)
    say = [0] * (n + 1)
    ok[0] = True                    # boş önek
    say[0] = 1
    for i in range(1, n + 1):
        j0 = i - en_uzun if i - en_uzun > 0 else 0
        for j in range(j0, i):
            if ok[j] and sozlukte_mi(s, j, i - j):
                if not ok[i]:
                    son[i] = j      # en küçük j
                ok[i] = True
                say[i] += say[j]
    return ok[n]
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Sözlük araması C'de sıralı dizide elle ikili aramadır (H4; parça `memcmp` ile karşılaştırılır); C++'ta `std::unordered_set<std::string>`, Python'da `set` (H10'un karma kümesi; rehber 14.1). Sorgu sayısı üç dilde aynıdır, çünkü sayaç arama yönteminden bağımsız olarak her sorguda artar. Dizgi üç dilde UTF-8 bayt dizisidir; konumlar bayttır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 25, 33, 116; Python 20, 29, 108.)

A.15 Örnek 15 · Matris zinciri (meydan okuma)

C++ **cpp/H13_15_matris_zinciri.cpp · satır 36–57**

```cpp
static long long matris_zinciri(const Dizi& p)
{
    const int n = static_cast<int>(p.size()) - 1;
    m.assign(n + 1, std::vector<long long>(n + 1, 0));   // m[i][i] = 0
    s.assign(n + 1, std::vector<int>(n + 1, 0));
    for (int l = 2; l <= n; l++) {             // zincir uzunluğu
        for (int i = 1; i + l - 1 <= n; i++) {
            int j = i + l - 1;
            m[i][j] = SONSUZ;
            for (int k = i; k < j; k++) {
                denenen_k++;
                long long q = m[i][k] + m[k + 1][j] +
                              static_cast<long long>(p[i - 1]) * p[k] * p[j];
                if (q < m[i][j]) {             // eşitlikte küçük k
                    m[i][j] = q;
                    s[i][j] = k;
                }
            }
        }
    }
    return m[1][n];
}
```

Python **python/H13_15_matris_zinciri.py · satır 24–40**

```python
def matris_zinciri(p):
    """Aralık DP'si: m[1..n][1..n], s[1..n][1..n]"""
    global m, s, denenen_k
    n = len(p) - 1
    m = [[0] * (n + 1) for _ in range(n + 1)]     # m[i][i] = 0
    s = [[0] * (n + 1) for _ in range(n + 1)]
    for l in range(2, n + 1):       # zincir uzunluğu
        for i in range(1, n - l + 2):
            j = i + l - 1
            m[i][j] = SONSUZ
            for k in range(i, j):
                denenen_k += 1
                q = m[i][k] + m[k + 1][j] + p[i - 1] * p[k] * p[j]
                if q < m[i][j]:     # eşitlikte küçük k
                    m[i][j] = q
                    s[i][j] = k
```

```
40      return m[1][n]
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Parantezleme C'de özyinelemeyle doğrudan yazdırılır; C++'ta `std::string` birleştirmesiyle, Python'da f-dizgisiyle döndürülür. Skaler çarpım C/C++'ta `(long long)` dönüşümüyle 64 bitte hesaplanır (ara çarpım `int`'te taşmasın); Python tamsayısı taşmaz. ∞ C'de `LLONG_MAX`, C++'ta `std::numeric_limits<long long>::max()`, Python'da aynı değer `(1 << 63) - 1`. Eşitlikte üç dilde de en küçük k tutulur. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 33, 59; Python 18, 45.)